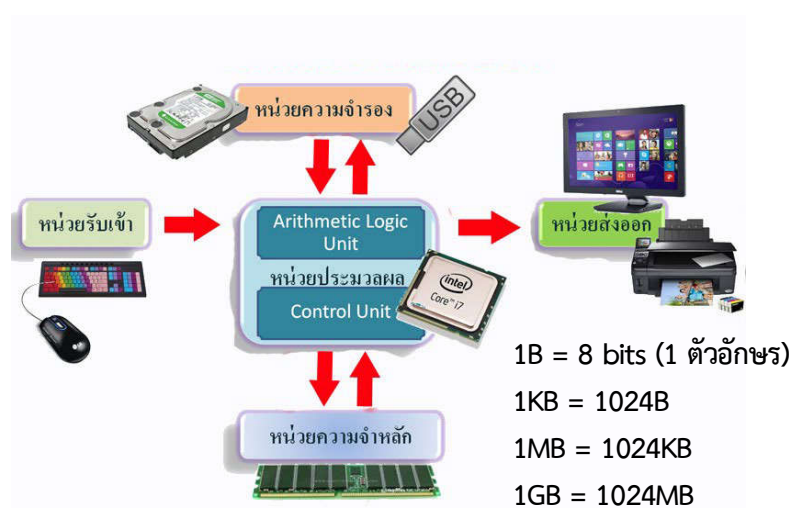


หน่วยที่ 1 คอมพิวเตอร์และการโปรแกรม ตัวแปร ฟังก์ชันการรับและแสดงผลข้อมูล ตัวดำเนินการ

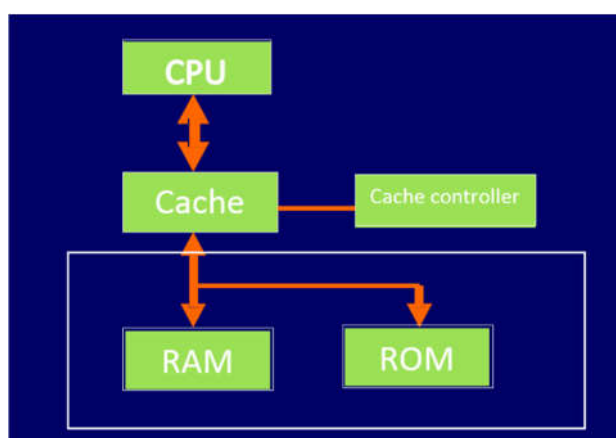
คอมพิวเตอร์และฟังก์งาน

คอมพิวเตอร์เป็นอุปกรณ์ที่มนุษย์ประดิษฐ์มาเพื่ออำนวยความสะดวกในด้านต่างๆ ไม่ว่าจะเป็นด้านการคำนวณการเก็บข้อมูล การตัดสินใจ และอื่นๆในอดีตคอมพิวเตอร์ถูกนำมาใช้ในงานด้านวิทยาศาสตร์เป็นส่วนใหญ่ แต่ปัจจุบันคอมพิวเตอร์ถูกพัฒนาขีดความสามารถสูงขึ้น มีการนำไปประยุกต์ใช้ในหน่วยงานต่างๆมากมาย เช่น ราชการ ธุรกิจ การแพทย์ การทหาร เป็นต้น ซึ่งการเรียนรู้ขั้นตอนการทำงานของคอมพิวเตอร์ทำให้เราสามารถเลือกซื้อคอมพิวเตอร์ได้ตรงตามความต้องการ โดยหลักการทำงานของคอมพิวเตอร์จะเป็นตามที่โปรแกรมกำหนดไว้ โดยตัวเครื่อง หรือที่เรียกว่า ฮาร์ดแวร์(Hardware) จะมีส่วนประกอบที่สำคัญพื้นฐาน 5 หน่วย คือ

1. หน่วยรับข้อมูล (input unit)
2. หน่วยประมวลผลกลาง (central processing unit)
3. หน่วยความจำหลัก (main memory unit)
4. หน่วยความจำรอง (secondary storage)
5. หน่วยแสดงผล (output unit)



โครงสร้างบนเมนบอร์ด



หน่วยความจำในระบบคอมพิวเตอร์

1. หน่วยรับข้อมูล (Input)

อุปกรณ์รับข้อมูล (Input Devices)

- เป็นเครื่องมือในการรับข้อมูล และคำสั่งจากผู้ใช้
- เช่น Keyboard, Mouse, Microphone

2. หน่วยประมวลผลกลาง (Central Processing Unit)

CPU ทำหน้าที่หลักในการประมวลผลข้อมูล และควบคุมการทำงานทั้งหมดของระบบ เป็นเสมือนสมองของคอมพิวเตอร์ในส่วนคำนวณ เรียกว่า Microprocessor วางอยู่บนแผงวงจรหลัก ซึ่งประกอบด้วยวงจรที่เชื่อมต่อระหว่างหน่วยประมวลผล กับหน่วยอื่นๆ

3. หน่วยความจำหลัก (Main Memory)

เป็นที่เก็บโปรแกรมและข้อมูลที่อยู่ระหว่างการประมวลผล โดยเก็บใน RAM (Random Access Memory) แบบชั่วคราว

ความจุของหน่วยความจำ มีหน่วยวัด ดังนี้

- 1 kBs (Kilobytes) = 2^{10} = 1024 Bytes
- 1 MBs (Megabytes) = 2^{20} = 1024 KBs (= 1,048,576 Bytes)
- 1 GBs (Gigabytes) = 2^{30} = 1024 MBs
- 1 TBs (Terabytes) = 2^{40} = 1024 GBs

4. หน่วยความจำสำรอง (Secondary Storages)

เก็บโปรแกรมและข้อมูลแบบถาวร ทั้งที่กำลังประมวลผลและยังไม่ถูกประมวลผลในขณะนั้น ชนิดของอุปกรณ์ที่ใช้เป็นหน่วยความจำสำรอง เช่น

- Hard disk
- CD
- Flash Memory

5. หน่วยแสดงผล (Output)

อุปกรณ์แสดงผลข้อมูล (Output Devices)

- เป็นเครื่องมือในการส่งผลการทำงานกลับมายังผู้ใช้
- เช่น จอภาพ เครื่องพิมพ์ ลำโพง

ประเภทของ Software

Software คือโปรแกรมชุดคำสั่ง ที่ผู้เขียนสร้างขึ้นเพื่อให้คอมพิวเตอร์ทำงานอย่างเป็นขั้นตอน และได้ผลลัพธ์ตามที่ต้องการ Software แบ่งได้ 2 ประเภท

1. โปรแกรมระบบ (Operating System: OS)
 - เช่น DOS, Windows, UNIX, ...
2. โปรแกรมประยุกต์ (Application Program)

- Software package เช่น Word, Spreadsheet, ...
- Developed program เช่น โปรแกรมใช้งานเฉพาะ ถูกพัฒนาโดยใช้โปรแกรมภาษา (Programming language เช่น Pascal, C, ...)

โปรแกรม OS ทำหน้าที่ในการควบคุมการทำงานของระบบคอมพิวเตอร์ เพื่อให้อุปกรณ์ต่างๆทำงานร่วมกันอย่างมีประสิทธิภาพ ตั้งแต่เริ่มเปิดเครื่องเป็นโปรแกรมสื่อกลางระหว่างอุปกรณ์ Hardwar และโปรแกรมประยุกต์ Software

โปรแกรมภาษา ใช้ในการพัฒนาโปรแกรมประยุกต์สำหรับงานเฉพาะตามที่ใช้ต้องการ ประเภทของโปรแกรมภาษา

1. ภาษาระดับต่ำ (Low-Level Language)
เช่น ภาษาเครื่อง (Machine language)
2. ภาษาระดับกลาง (Middle-Level Language)
เช่น ภาษาแอสเซมบลี (Assembly Language)
3. ภาษาระดับสูง (High-Level Language)
เช่น Pascal, Fortran, C, JAVA, ...

ภาษาเครื่อง (Machine Language)

เป็นภาษาที่คอมพิวเตอร์เข้าใจ ซึ่งเขียนเป็นรหัสเลขฐาน 2 (0/1) และคำสั่งมีความเกี่ยวข้องกับอุปกรณ์ของคอมพิวเตอร์โดยตรง แต่มนุษย์เข้าใจภาษาเครื่องได้ยาก ดังนั้นการเขียนโปรแกรมด้วยภาษาเครื่องจึงยากมาก

```
00010100101101010101010101010100010
11101101010101010101110010100010110
00101001010100101111010111011101010
1001010010110101010101010101010110
01101001001100101111010111010100010
0001000101011101010100010101011010
10101001010100101101011101011101011
00010100101101010101010101010100010
```

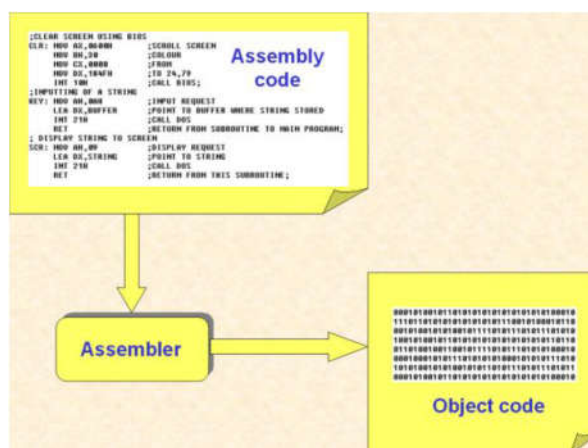
Object code



ภาษาแอสเซมบลี

ภาษาแอสเซมบลี (Assembly Language) เป็นภาษาที่เขียนโดยใช้คำสั่งที่มนุษย์เข้าใจ (English-like statements) แทนการใช้รหัสเลขฐาน 2

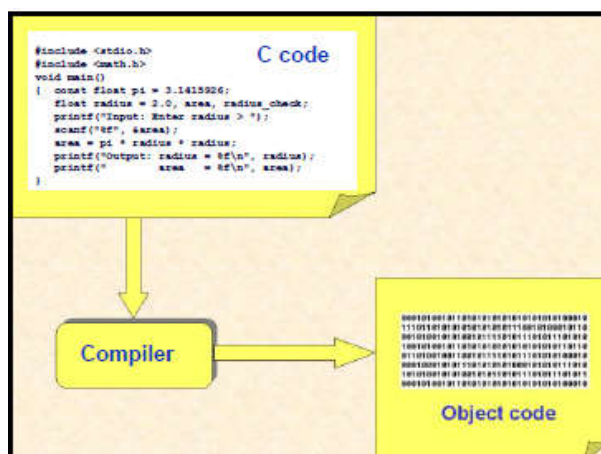
- แต่ออกแบบมาเฉพาะสำหรับคอมพิวเตอร์แต่ละแบบ
- และผู้เขียนโปรแกรมยังต้องทราบข้อมูลที่เกี่ยวข้องกับอุปกรณ์ของคอมพิวเตอร์ ใช้แอสเซมเบอร์ (Assembler) ในการแปลภาษาแอสเซมบลี ให้เป็นภาษาเครื่อง



ภาษาระดับสูง

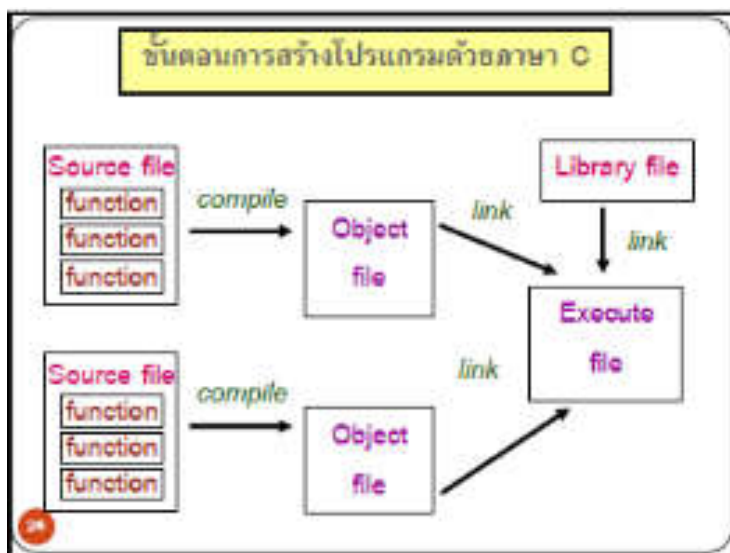
ภาษาระดับสูง (High-Level Languages) ใช้ภาษาที่มนุษย์เข้าใจ (English-like language) เช่น Basic, Pascal, C, JAVA, ... ใช้ คอมไพเลอร์ (Compiler) หรือ Interpreter ในการแปลภาษาระดับสูงให้เป็นภาษาเครื่อง

- Compiler แปลทั้งโปรแกรม (เช่น Pascal, C, ...)
- Interpreter แปลทีละบรรทัด (เช่น Basic, ...)



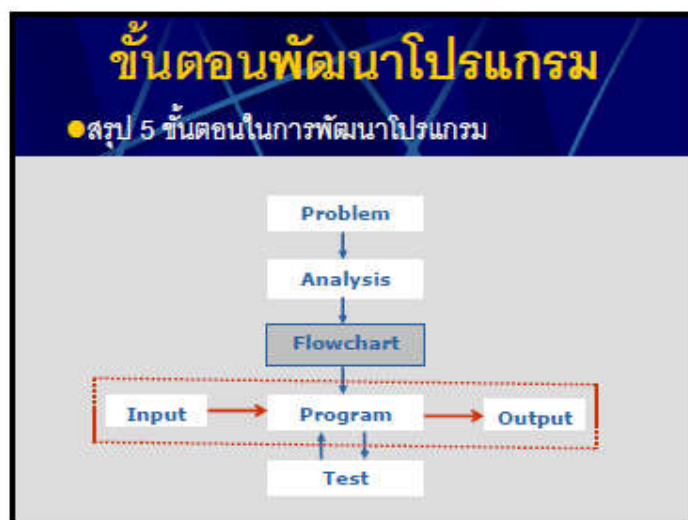
ภาษาสำหรับนักพัฒนาโปรแกรม

- วิซวลเบสิก (VISUAL Basic) พัฒนาโดยบริษัทไมโครซอฟต์ ปัจจุบันใช้เทคโนโลยี .net
- ภาษาจาวา (JAVA) พัฒนาโดยบริษัท Sun Microsystem
- ภาษาซีชาร์ป (C#) พัฒนาโดยไมโครซอฟต์ ปัจจุบันใช้เทคโนโลยี .net



การพัฒนาโปรแกรม การพัฒนาโปรแกรม ประกอบด้วย 5 ขั้นตอน

1. กำหนดและวิเคราะห์ปัญหา (State problem & Problem analysis)
2. เขียนผังงาน (Flowchart)
3. เขียนโปรแกรม (Program Development)
4. ทดสอบและแก้ไขโปรแกรม (Testing & debugging)
5. ทำเอกสารและบำรุงรักษาโปรแกรม (Document & maintenance)

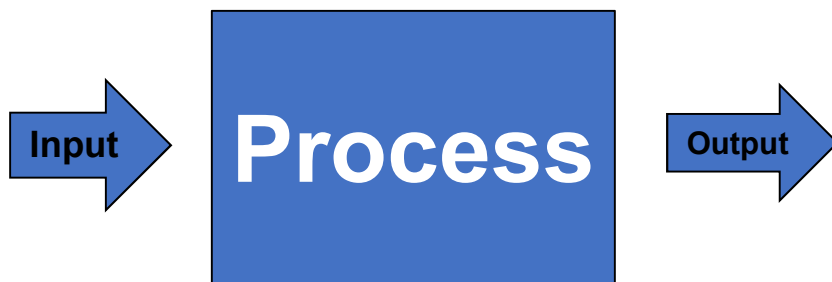


กำหนด-วิเคราะห์ปัญหา

กำหนดขอบเขตของปัญหา ให้ชัดเจนว่า จะให้คอมพิวเตอร์ทำอะไร (What?) วิ

เคราะห์ปัญหา (Problem analysis) กำหนด

- Input: ลักษณะของข้อมูลเข้า
- Process: วิธีการประมวลผล (How?)
- Output: ลักษณะของผลลัพธ์ที่ต้องการ



การเขียนผังงาน

ผังงาน (Flowchart) เป็นแผนภาพ ที่ใช้อธิบายลำดับการทางานของโปรแกรมเป็นขั้นตอน ประโยชน์ของ Flowchart

1. อธิบายลำดับขั้นตอนการทำงานของโปรแกรม
2. ทำให้ตรวจสอบข้อผิดพลาดของโปรแกรมได้ง่าย
3. ทำให้ผู้อื่นสามารถศึกษาการทำงานและแก้ไข โปรแกรมได้ง่าย

ประเภทของผังงาน

- ผังงานระบบ (System Flowchart) แสดงขั้นตอนการทำงานภายในระบบงาน โดยจะกล่าวอย่างกว้างๆ
- ผังงานโปรแกรม (Program Flowchart) แสดงขั้นตอนของคำสั่ง ที่ใช้ในโปรแกรม

ลักษณะในผังงาน

สัญลักษณ์พื้นฐาน ที่นิยมใช้ใน Flowchart



การเริ่มต้น และการสิ้นสุดการทางาน



ลูกศรแสดงทิศทางการทางาน และการไหลของข้อมูล



การประมวลผล หรือการคำนวณ (การปฏิบัติงานอย่างใดอย่างหนึ่ง)



การรับข้อมูล หรือแสดงผล (ไม่ระบุชนิดอุปกรณ์)



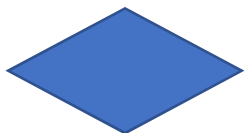
การเตรียมการ (กำหนดชื่อข้อมูล หรือค่าเริ่มต้นต่างๆ)



การแสดงผลทางจอภาพ



การแสดงผลทางเครื่องพิมพ์ (เป็นเอกสาร)



การตรวจสอบเงื่อนไข (เพื่อการตัดสินใจเมื่อมีทางเลือก)



จุดเชื่อมต่อผังงาน

รูปแบบของผังงาน

- แบบลำดับ (Sequence)
- แบบมีทางเลือก (Selection)
- แบบทำซ้ำ (Looping)

การเขียนผังงานกับโครงสร้างควบคุมแบบลำดับ

ลักษณะของผังงานที่มีโครงสร้างควบคุมแบบลำดับจะเป็นลักษณะการทำงานเป็นลำดับเรียงกันไปจากบนลงล่าง ไม่มีการย้อนกลับไปทำลำดับที่ผ่านมาได้ ในผังงานเราใช้สัญลักษณ์ Process Symbol แทนโครงสร้างควบคุมแบบลำดับ ดังนี้

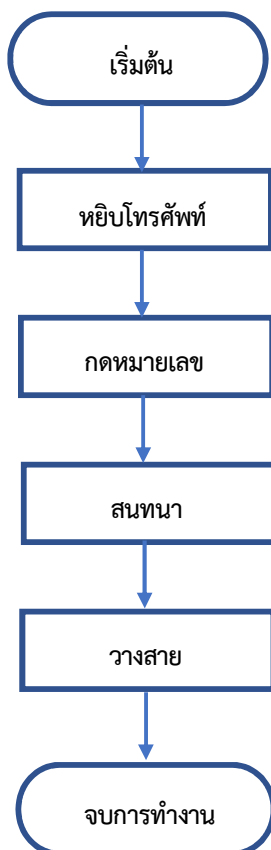


ตัวอย่างโครงสร้างผังงานแบบลำดับ

สถานการณ์ ต้องการเขียนผังงานแสดงขั้นตอนการโทรศัพท์
หากเขียนไล่เรียงขั้นตอนการถ่ายภาพออกมาเป็นข้อๆ จะได้ดังนี้

1. หยิบโทรศัพท์
2. กดหมายเลข
3. สนทนา
4. วางสาย

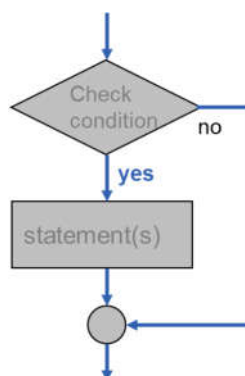
จากข้างต้น สามารถนำมาเขียนให้อยู่ในรูปผังงานได้ ดังนี้



การเขียนผังงานกับโครงสร้างควบคุมแบบการตัดสินใจ

รูปแบบของการเขียนผังงานแบบการตัดสินใจ จะขึ้นกับเงื่อนไข (Condition) ในการตัดสินใจว่ามีค่าความจริงเป็นจริงหรือเท็จ และโปรแกรมก็จะข้ามไปทำงานในกลุ่มของคำสั่งตามเงื่อนไขนั้น ๆ รูปแบบของคำสั่งการตัดสินใจโดยทั่วไป มีดังนี้

รูปแบบที่ 1 ทางเลือกทางเดียว (Single Alternative IF) เป็นลักษณะของคำสั่งที่มีทางเลือกทางเดียว คือ ถ้าเงื่อนไขในการตรวจสอบเป็นจริงจึงทำคำสั่ง (Statement) หรือกลุ่มคำสั่ง (Statement Block) แต่ถ้าเงื่อนไขการตรวจสอบเป็นเท็จก็ให้ข้ามไปทำคำสั่งถัดไป

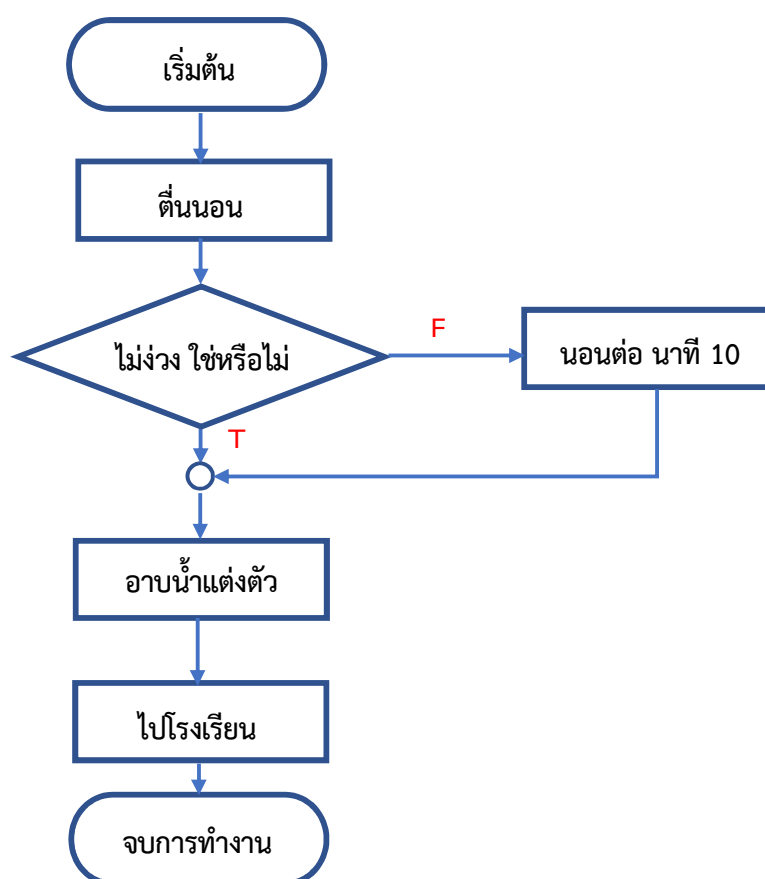


ตัวอย่างโครงสร้างผังงานแบบการตัดสินใจ (ทางเลือกทางเดียว)

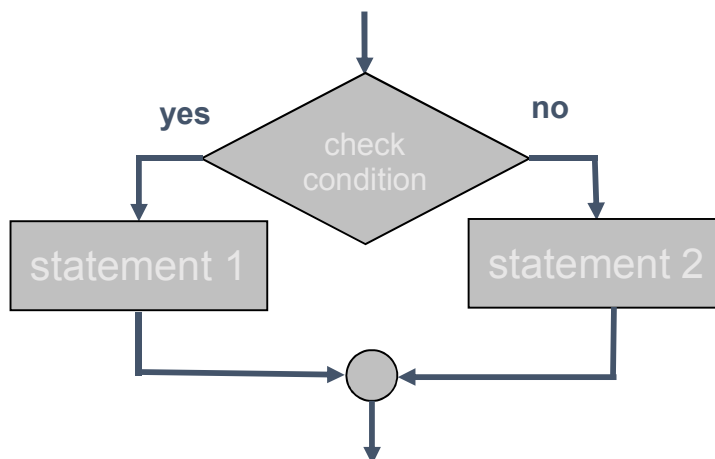
สถานการณ์ ต้องการเขียนผังงานแสดงขั้นตอนการตื่นนอนไปโรงเรียน โดยมีเงื่อนไขในการตื่น คือ ถ้ายังง่วงให้นอนต่ออีก นานี้ 10หากเขียนไล่เรียงขั้นตอนการตื่นนอนไปโรงเรียนออกมาเป็นข้อๆ จะได้ดังนี้

1. ตื่นนอน
2. ตรวจสอบว่าไม่่ง่วงนอน ใชหรือไม่
 - 2.1 ไม่ใช่ นอนต่อ นานี้ 10
3. อาบน้ำแต่งตัว
4. ไปโรงเรียน

จากข้างต้น สามารถนำมาเขียนให้อยู่ในรูปผังงานได้ ดังนี้



รูปแบบที่ 2 ทางเลือกสองทาง (Double Alternative) เป็นรูปแบบที่มีทางเลือกสองทางเลือก ขึ้นอยู่กับค่าที่ใช้ในการตรวจสอบเงื่อนไข โดยใช้คำสั่ง ELSE เพื่อเป็นการแบ่งแยกเงื่อนไขของทางเลือก คือ เมื่อเงื่อนไขในการตรวจสอบมีค่าความจริงเป็นจริง ก็จะไปทำคำสั่งหลังคำว่า THEN แล้วจึงทำคำสั่งถัดจากคำสั่ง END IF และถ้าเงื่อนไขในการตรวจสอบมีค่าความจริงเป็นเท็จ ก็จะไปทำคำสั่งหลังคำว่า ELSE แล้วจึงทำคำสั่งถัดจากคำสั่ง END IF ต่อไป

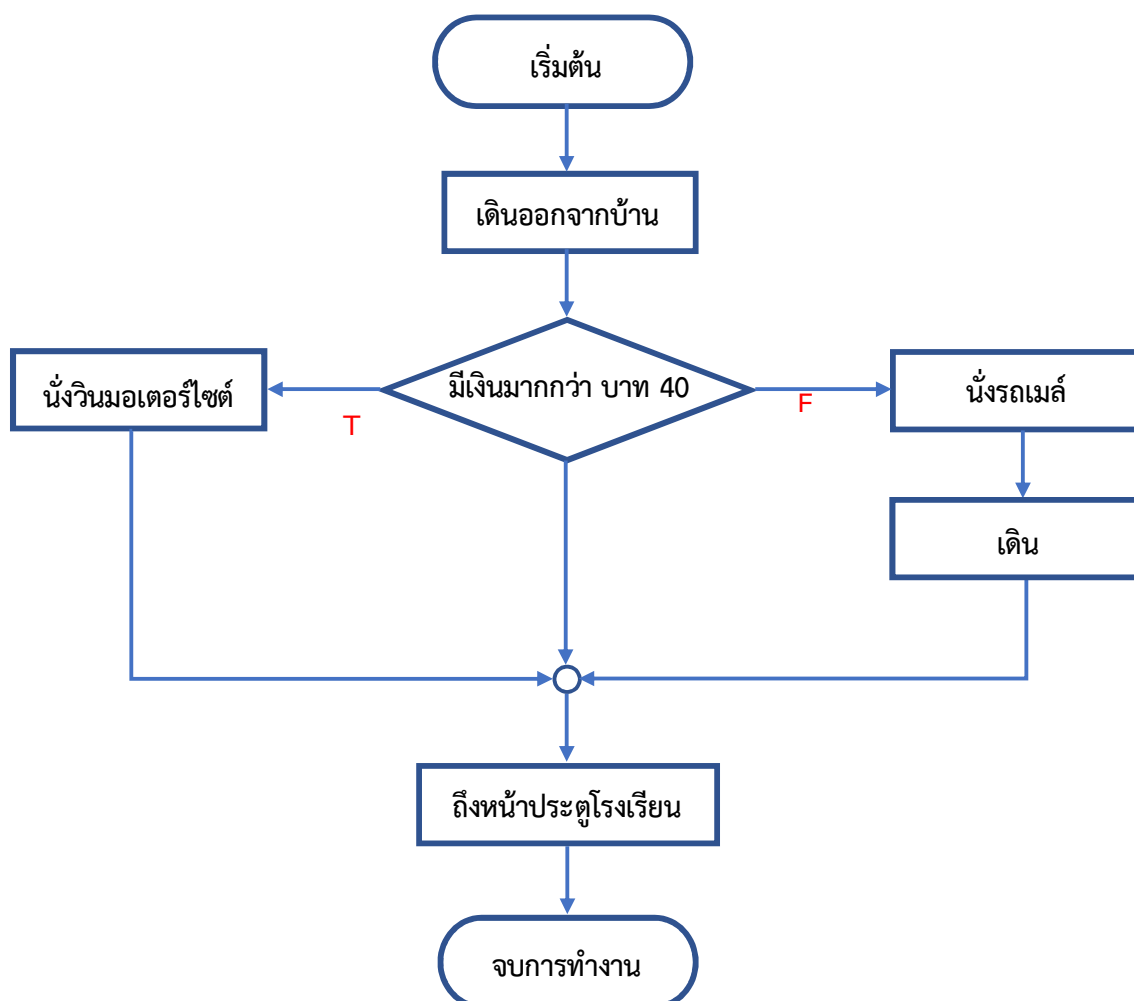


ตัวอย่างโครงสร้างผังงานแบบการตัดสินใจ (ทางเลือกสองทาง)

สถานการณ์ ต้องการเขียนผังงานแสดงขั้นตอนการเดินทางไปโรงเรียน โดยมีเงื่อนไขในการเดินทาง คือ ถ้ามีเงินมากกว่า บาท ให้นำเงินมอเตอร์ไซด์ถึงหน้าประตูโรงเรียน ถ้าน้อยกว่า ให้นำรถเมล์ และเดินถึงหน้าประตูโรงเรียน 40 หากเขียนไล่เรียงขั้นตอนการตัดสินใจไปโรงเรียนออกมาเป็นข้อๆ จะได้ดังนี้

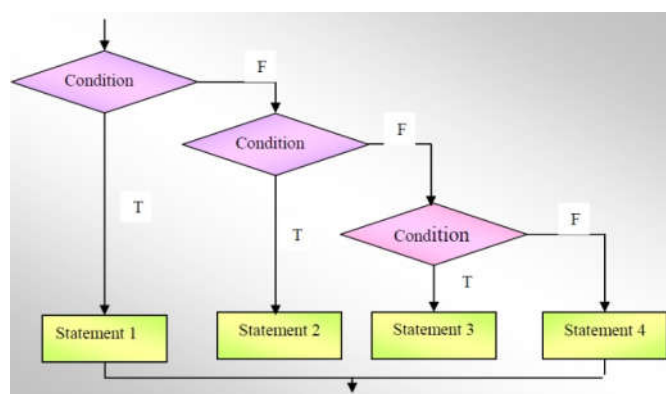
1. เดินออกจากบ้าน
2. ตรวจสอบว่ามีเงินมากกว่า บาท ใช่หรือไม่ 40
 - 2.1 ใช่ นำเงินมอเตอร์ไซด์
 - ไม่ใช่ นำรถเมล์ และเดิน 2.2
3. ถึงหน้าประตูโรงเรียน

จากข้างต้น สามารถนำมาเขียนให้อยู่ในรูปผังงานได้ ดังนี้



รูปแบบที่ 3 ทางเลือกหลายทาง (Multiple Alternative) โครงสร้างควบคุมแบบการตัดสินใจชนิด ทางเลือกหลายทางยังสามารถแบ่งย่อยได้อีก 2 ลักษณะ คือ

1. ทางเลือกหลายทางแบบ IF ซ้อน IF (Nested IF statement)



ตัวอย่างโครงสร้างผังงานแบบการตัดสินใจ ทางเลือกหลายทาง แบบ)IF ซ้อน IF(

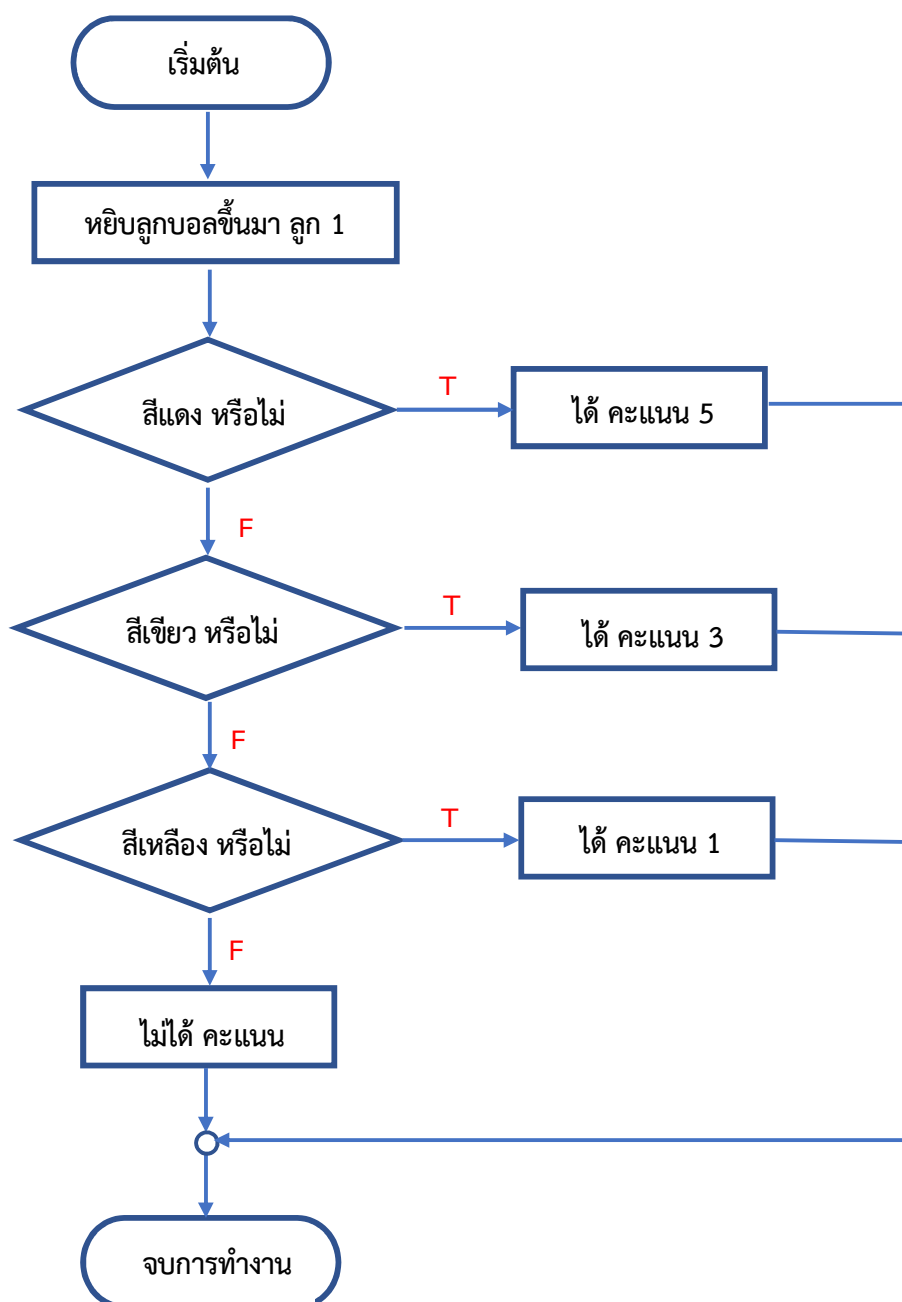
สถานการณ์ ต้องการเขียนผังงานแสดงขั้นตอนการหยิบลูกบอลในภาชนะหนึ่ง ซึ่งมีลูกบอล ลูก คือ สีแดง สีเขียว สี 4 โดยลูกบอลที่หยิบขึ้นมาจะมีคะแนน ดังนี้ เหลือง และสีดำ

- ถ้าหยิบได้ สีแดง ได้ 5 คะแนน
- ถ้าหยิบได้ สีเขียว ได้ 3 คะแนน
- ถ้าหยิบได้ สีเหลือง ได้ 1 คะแนน
- ถ้าหยิบได้ สีดำ ไม่ได้รับคะแนน

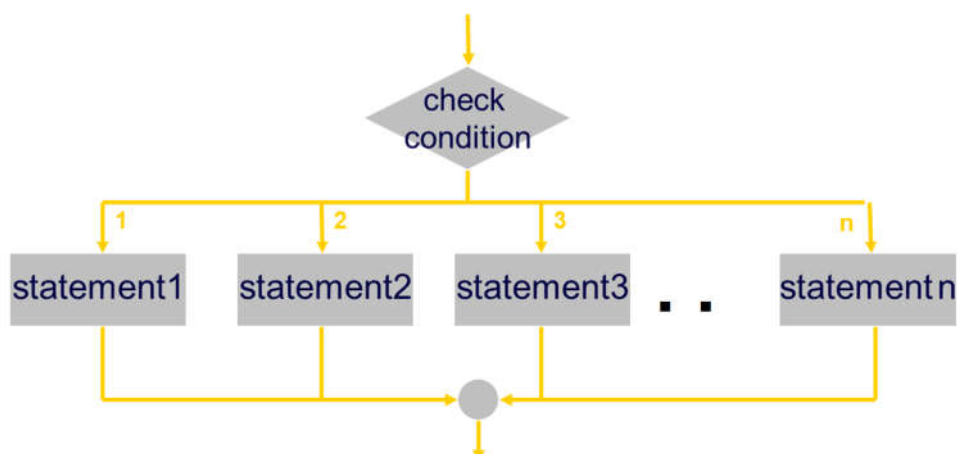
หากเขียนไล่เรียงขั้นตอนการตัดสินใจไปโรงเรียนออกมาเป็นข้อๆ จะได้ดังนี้

1. หยิบลูกบอลขึ้นมา ลูก 1
2. ตรวจสอบเงื่อนไข ใช้สีแดงหรือไม่
 - คะแนน 5 ใช้ ได้ 2.1
 - 3 ไม่ใช่ ไปที่ข้อ 2.2
3. ตรวจสอบเงื่อนไข ใช้สีเขียวหรือไม่
 - ใช้ 3.1ได้ คะแนน 3
 - 4 ไม่ใช่ ไปที่ ข้อ 3.2
4. ตรวจสอบเงื่อนไข ใช้สีเหลืองหรือไม่
 - ใช้ 3.1ได้ คะแนน 1
 - ไม่ใช่ ไม่ได้รับคะแนน 3.2

จากข้างต้น สามารถนำมาเขียนให้อยู่ในรูปผังงานได้ ดังนี้



2. ทางเลือกหลายทางแบบ Case (Case Control Structure)



ตัวอย่างโครงสร้างผังงานแบบการตัดสินใจ ทางเลือกหลายทางแบบ)Case(

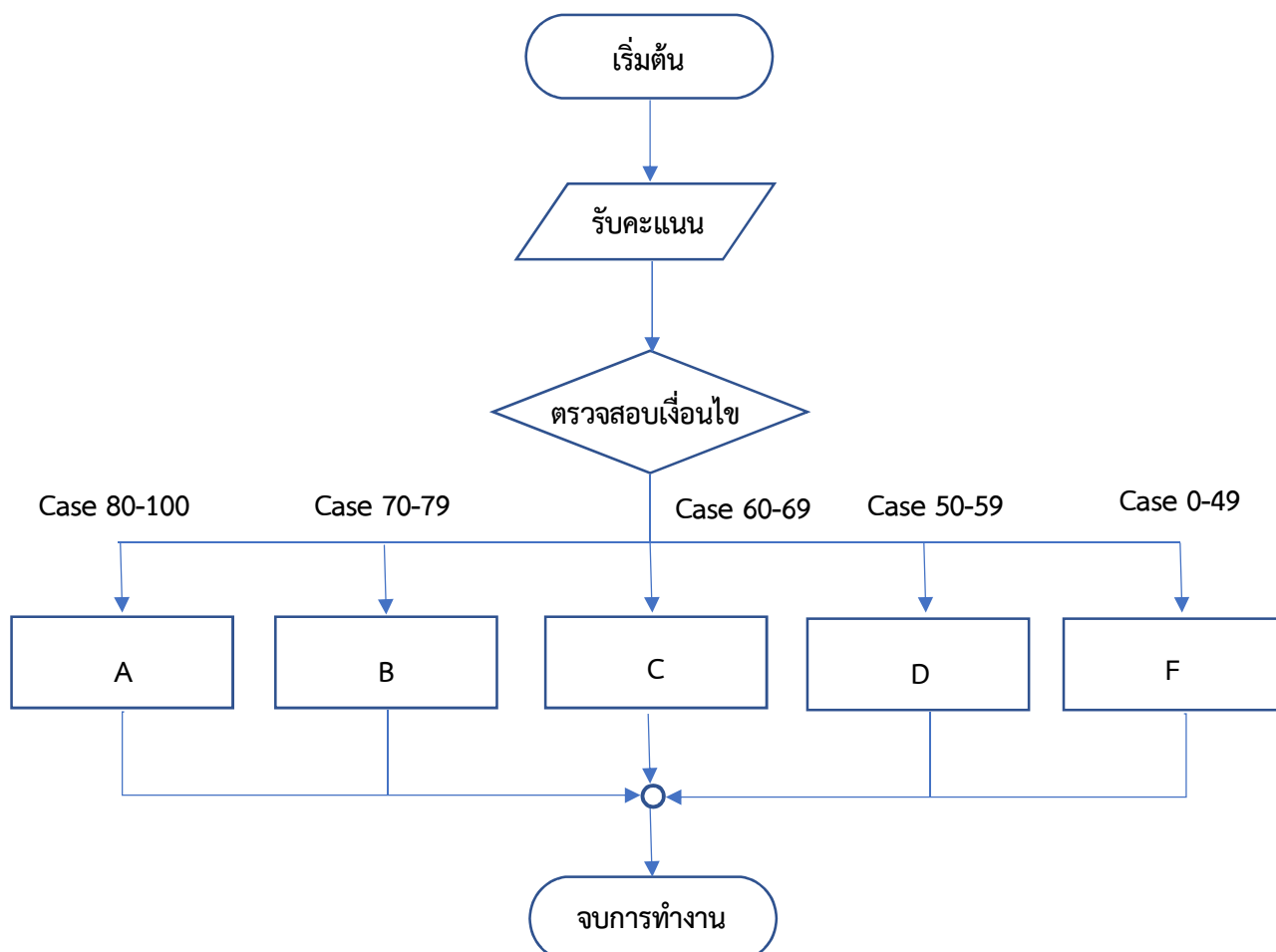
สถานการณ์ ต้องการเขียนผังงานแสดงขั้นตอนการตัดเกรดนักเรียน โดยมีเงื่อนไข ดังนี้

- ถ้าคะแนน 80-100 ได้ เกรด A
- ถ้าคะแนน 70-79 ได้ เกรด B
- ถ้าคะแนน 60-69 ได้ เกรด C
- ถ้าคะแนน 50-59 ได้ เกรด D
- ถ้าคะแนน 0-49 ได้ เกรด F

หากเขียนไล่เรียงขั้นตอนการตัดสินใจไปโรงเรียนออกมาเป็นข้อๆ จะได้ดังนี้

1. รับคะแนนนักเรียน
2. ตรวจสอบคะแนนนักเรียนตามเงื่อนไข
 - 100-80 คะแนน 2.1 ได้ เกรด A
 - 2.2 คะแนน 70-79 ได้ เกรด B
 - 2.3 คะแนน 60-69 ได้ เกรด C
 - 2.4 คะแนน 50-59 ได้ เกรด D
 - 2.5 คะแนน 0-49 ได้ เกรด F

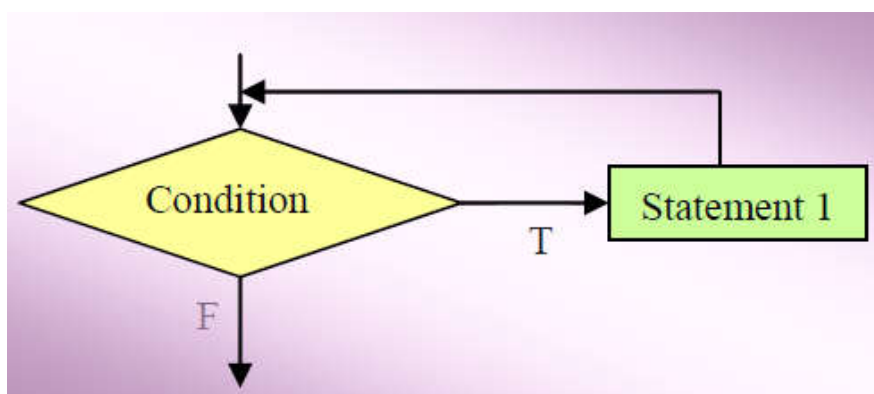
จากข้างต้น สามารถนำมาเขียนให้อยู่ในรูปผังงานได้ ดังนี้



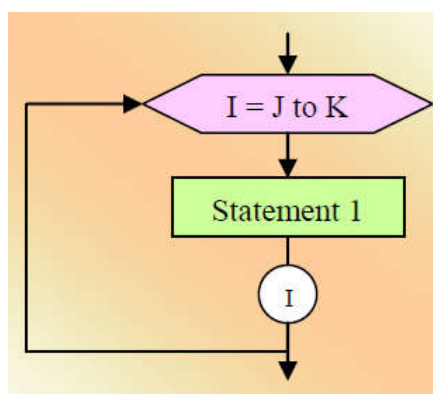
การเขียนผังงานกับโครงสร้างควบคุมแบบการทำซ้ำ

รูปแบบการเขียนโครงสร้างการทำซ้ำหรือการวนรอบนั้น สามารถใช้รูปแบบได้ 2 รูปแบบ ได้แก่

1. รูปแบบ DO...WHILE ในการเปรียบเทียบเงื่อนไขของการทำซ้ำ ถ้าเงื่อนไขเป็นจริงก็จะทำคำสั่งภายในลูป (Loop) และวนกลับมาเปรียบเทียบอีกครั้งเพื่อทำงานต่อไป จนกระทั่งเงื่อนไขในการเปรียบเทียบเป็นเท็จจึงจะข้ามไปทำคำสั่งถัดไป

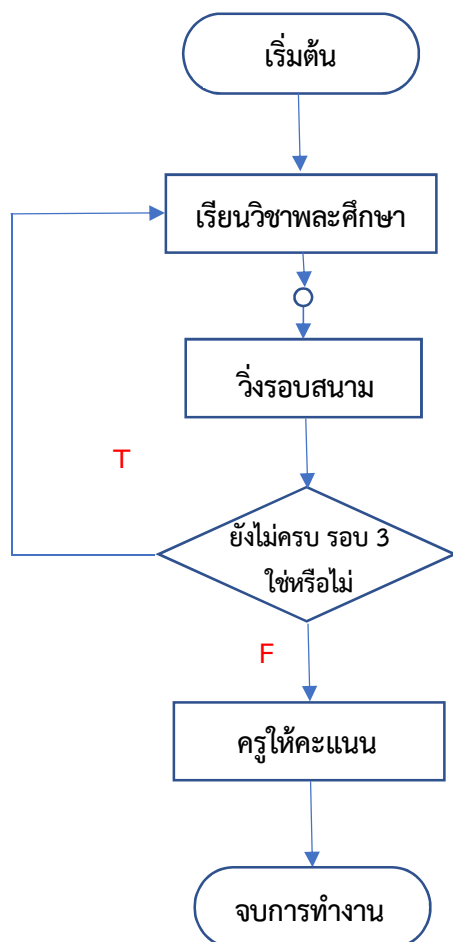


2. รูปแบบ FOR Loop หากทราบจำนวนที่จะวนรอบที่แน่นอน ควรจะใช้ FOR Loop โดยมี Loop Index เป็นตัวควบคุมการวนรอบ หากตรวจสอบจำนวนรอบจะกระทำโดยอัตโนมัติ การสิ้นสุดการวนรอบจะกระทำเมื่อเงื่อนไขเป็นเท็จ



ตัวอย่างโครงสร้างผังงานแบบการทำซ้ำ)FOR(

สถานการณ์ ต้องการเขียนผังงานแสดงการวิ่งรอบสนามในการเรียนวิชาพลศึกษา โดยมีเงื่อนไขว่า ถ้าวิ่งรอบสนามครบ ครูสามารถให้คะแนน จึงหยุดการทำงาน รอบ 3
จากข้างต้น สามารถนำมาเขียนให้อยู่ในรูปผังงานแบบ FOR ได้ ดังนี้



แบบฝึกหัดการเขียนผังงาน

1. ให้นักเรียนเขียนผังงานแสดงขั้นตอนการบวกเลข 2 จำนวน โดยรับข้อมูล 2 ค่า และแสดงผลบวก
2. ให้นักเรียนเขียนผังงานแสดงขั้นตอนการคำนวณหาพื้นที่วงกลม โดยมีการรับค่าข้อมูล เป็นรัศมี และแสดงค่าพื้นที่วงกลม
3. ให้นักเรียนเขียนผังงานแสดงขั้นตอนการหารเลข 2 จำนวน โดยโปรแกรมต้องตรวจสอบได้ว่าตัวหารเป็น “0” หรือไม่
4. ให้นักเรียนเขียนผังงานแสดงขั้นตอนการทำงานของโปรแกรม โดยให้โปรแกรมรับคะแนนนักเรียน ตรวจสอบ และแสดงผล ถ้าผ่าน โดยมีเงื่อนไข ดังนี้
 - คะแนน ≥ 50 ผ่าน
 - คะแนน < 50 ตก
5. ให้นักเรียนเขียนผังงานแสดงขั้นตอนสถานภาพการเรียน โดยมีเงื่อนไข ดังนี้
 - เกรดเฉลี่ย < 1.00 แสดงสถานะ Retried
 - เกรดเฉลี่ย < 2.00 แสดงสถานะ Probation
 - เกรดเฉลี่ย ≥ 2.00 แสดงสถานะ Normal
6. ให้นักเรียนเขียนผังงานแสดงขั้นตอนการคูณเลข ตั้งแต่ เลข 1 – 10 และแสดงผลบวก
7. ให้นักเรียนเขียนผังงานแสดงขั้นตอนการหาผลรวมของตัวเลข 1 – 100 เฉพาะที่เป็นเลขคู่และแสดงผลบวก

โครงสร้างภาษาซี

ให้นักเรียนศึกษาตัวอย่างโปรแกรมต่อไปนี้ครับ นักเรียนอาจจะลองเขียนโปรแกรมนีกลงในโปรแกรม Dev-C++ หรือ CodeBlocks เพื่อศึกษาผลลัพธ์ด้วยก็ได้

```
บรรทัดที่ 1  #include<stdio.h>
บรรทัดที่ 2  #include<stdlib.h>
บรรทัดที่ 3  int main()
บรรทัดที่ 4  {
บรรทัดที่ 5      printf("Hello World!");
บรรทัดที่ 6      system("pause");
บรรทัดที่ 7      return 0;
บรรทัดที่ 8  }
```

ผลลัพธ์จากการรันโปรแกรม

Hello World!

โปรแกรมด้านบนนี้เป็นโปรแกรมภาษาซี ซึ่งเมื่อรันโปรแกรมแล้วจะแสดงผลคำว่า Hello World! โดยบรรทัดที่ 1 เป็นการเรียกใช้ไฟล์ stdio.h ซึ่งเป็นไฟล์ที่บรรจุฟังก์ชัน printf ที่ถูกเรียกใช้ในบรรทัดที่ 4 บรรทัดที่ 2 เป็นการเรียกใช้ไฟล์ stdlib.h ซึ่งเป็นไฟล์ที่บรรจุฟังก์ชัน system() ที่ถูกเรียกใช้ในบรรทัดที่ 6 บรรทัดที่ 3 ฟังก์ชัน int main() เป็นฟังก์ชันหลักสำหรับเขียนโปรแกรม โดยทุกโปรแกรมจะต้องมีฟังก์ชันหลักนี้เสมอ

บรรทัดที่ 4 วงเล็บเปิดสำหรับเริ่มเขียนโปรแกรม โปรแกรมที่เขียนจะต้องเขียนภายในวงเล็บนี้
 บรรทัดที่ 5 เรียกใช้ฟังก์ชัน printf เพื่อแสดงคำว่า Hello World!
 บรรทัดที่ 6 ฟังก์ชันที่สั่งให้โปรแกรมแสดงผลหน้าจอค้างไว้
 บรรทัดที่ 7 คืนค่า 0 เพื่อบอกว่าจบโปรแกรม
 บรรทัดที่ 8 วงเล็บปิด เพื่อแสดงว่าจบฟังก์ชัน main

ตัวอย่างโปรแกรม

แสดงโครงสร้างของโปรแกรม โดยโปรแกรมนี้นประกอบด้วยฟังก์ชัน main() และ ฟังก์ชัน sum()
 ฟังก์ชัน main() ทำหน้าที่รับค่ามาเก็บไว้ในตัวแปร a และตัวแปร b แล้วส่งค่าของตัวแปรทั้งสองไปยังฟังก์ชัน sum() เพื่อคำนวณหาผลรวม เมื่อคำนวณผลรวมแล้ว จะส่งค่าของผลรวมกลับไปยังฟังก์ชัน main() จากนั้นฟังก์ชัน main() จะแสดงค่าของผลรวม

#include<stdio.h>	//คำสั่งตัวประมวลผลก่อน
int a, b, c;	คำสั่งประกาศกรอบคลุม//
int sum(int x, int y);	คำสั่งแบบฟังก์ชัน//
int main()	//ฟังก์ชัน main()
{	//เริ่มต้นฟังก์ชัน main()
scanf("%d", &a);	//คำสั่งรับค่าตัวแปร
scanf("%d", &b);	//คำสั่งรับค่าตัวแปร
c = sum(a, b);	//เรียกฟังก์ชัน sum()
printf("\n%d + %d = %d",a, b, c);	//แสดงผล
}	//จบฟังก์ชัน main()
int sum (int x, int y)	//ฟังก์ชัน sum()
{	//เริ่มต้นฟังก์ชัน sum()
return (x + y);	คำสั่งรวมค่าและส่งค่ากลับ//
}	//จบฟังก์ชัน sum()

ตัวแปร (variables)

คอมพิวเตอร์มีส่วนประกอบที่สำคัญส่วนหนึ่งคือ หน่วยความจำ ซึ่งเปรียบได้กับสมองของมนุษย์ทำหน้าที่เก็บข้อมูล ในขณะที่ประมวลผล ในการประมวลผลแต่ละครั้งมักต้องใช้ข้อมูลจำนวนมาก ซึ่งจำเป็นจะต้องเก็บไว้ในหน่วยความจำ เป็นเก็บแล้วจะต้องทราบตำแหน่งที่นำข้อมูลเข้าไปเก็บไว้ภายในของหน่วยความจำด้วย เพื่อให้สามารถนำข้อมูล เหล่านั้นกลับมาประมวลผลได้ ดังนั้นตัวแปรจึงมีหน้าที่สำคัญที่ช่วยในการเก็บข้อมูลแต่ละประเภทที่ผู้ใช้ป้อนเข้าสู่ โปรแกรม

ชนิดข้อมูล (data types)

ข้อมูลที่ใช้ในโปรแกรมมีหลายชนิด ซึ่งนักเขียนโปรแกรมต้องเลือกใช้ตามความเหมาะสมกับการใช้งาน ข้อมูลมี ขนาดที่แตกต่างกันไปตามชนิดข้อมูล นอกจากนี้แล้ว ชนิดข้อมูลยังอาจมีขนาดที่แตกต่างกันโดยขึ้นกับเครื่อง คอมพิวเตอร์และตัวแปรโปรแกรมที่ใช้ในการประมวลผล แต่โดยทั่วไปแล้วในไมโครคอมพิวเตอร์ ชนิดข้อมูลมีการใช้ใน โปรแกรมและขนาดดังนี้

ชนิดข้อมูล	การใช้ในโปรแกรม	คำสั่งในโปรแกรม	ขนาดข้อมูล (ไบต์)	ช่วงข้อมูล
character	char	%c	1	-128 ถึง 127
integer	int	%d	2	-32768 ถึง 32767
long integer	long	%ld	4	-2147483648 ถึง 2147483647
unsigned character	unsigned char	%c	1	0 ถึง 255
unsigned integer	unsigned int	%d	2	0 ถึง 65535
unsigned long	unsigned long	%ld	4	0 ถึง 4294967295
single-precision floating-point	Float	%f	4	1.2×10^{-38} ถึง 3.4×10^{38}

กฎการตั้งชื่อตัวแปร

การตั้งชื่อตัวแปรมีข้อกำหนด ดังนี้

ประกอบด้วย -a ถึง z, 0 ถึง 9 และ _ เท่านั้น

อักขระตัวแรกต้องเป็น -a ถึง z และ _

- ห้ามใช้ชื่อเฉพาะ

ตัวพิมพ์ใหญ่ ตัวพิมพ์เล็ก มีความหมายที่แตกต่างกัน -

มีความยาวได้สูงสุด -31 ตัวอักษร

การประกาศตัวแปร

การประกาศตัวแปรทำได้โดย เขียนข้อความสั่งขึ้นต้นด้วยชนิดข้อมูล ตามด้วยชื่อตัวแปร และจบข้อความสั่งประกาศตัวแปรด้วยเครื่องหมายอัฒภาค (;) ดังนี้

ชนิดข้อมูล ชื่อตัวแปร;

ถ้าต้องการประกาศตัวแปรชนิดเดียวกันหลายตัว ต้องคั่นระหว่างตัวแปรด้วยเครื่องหมายจุลภาค (,) และจบด้วยเครื่องหมายอัฒภาค (;) ดังนี้

ชนิดข้อมูล ชื่อตัวแปร1, ชื่อตัวแปร2;

เช่น

```
int count;           //ประกาศตัวแปรชื่อ count ใช้เก็บข้อมูลชนิด integer
int m, n;            //ประกาศตัวแปรชื่อ m และ n ใช้เก็บข้อมูลชนิด integer
int num = 10;        //ประกาศตัวแปรชื่อ num และเก็บค่าไว้ในตัวแปรดังกล่าว 10
char str = 'a';      //ประกาศตัวแปรชื่อ str และเก็บค่าอักขระ a ไว้ในตัวแปรดังกล่าว
```

ตัวดำเนินการคำนวณ

ตัวดำเนินการคำนวณพื้นฐาน มีดังนี้

ตัวดำเนินการ บวก +

ตัวดำเนินการ ลบ -

ตัวดำเนินการ คูณ *

ตัวดำเนินการหาร /

) ตัวดำเนินการมอดุลัส % (Modulus) หรือหารแล้วเอาเฉพาะเศษนั่นเอง

ในการทำงานของตัวดำเนินการคอมพิวเตอร์จะมีลำดับการทำงานดังนี้

1. () คอมพิวเตอร์จะทำการคำนวณในวงเล็บก่อนเสมอ
2. * / % จากนั้นจะทำการคูณ หาร มอดุลัส เรียงจากซ้ายไปขวา เจอเครื่องหมายใดก่อน ทำก่อน
3. + - และทำการบวก ลบ เรียงจากซ้ายไปขวา

จงหาลำดับจากการคำนวณต่อไปนี้

1. $x = 4 + 5 * 3$

ตอบ $x = \dots\dots\dots$

2. $y = 3 + 6 / 2 + 1$

ตอบ $y = \dots\dots\dots$

3. $z = 5 \% 2$

ตอบ $z = \dots\dots\dots$

4. $m = (5 \% 3) + (7 + 3) * 2$

ตอบ $m = \dots\dots\dots$

5. $n = 10 / 5 + 7 * 2 - 6 / 3 / 2$

ตอบ $n = \dots\dots\dots$

ความรู้เพิ่มเติม

การเขียนโปรแกรมที่มีการแก้โจทย์ปัญหาทางคณิตศาสตร์ และมีการใช้ฟังก์ชันต่างๆ ทางคณิตศาสตร์ นักเรียนสามารถใช้ฟังก์ชันทางคณิตศาสตร์ในภาษาซีช่วยในการเขียนโปรแกรมได้ โดยมีฟังก์ชันที่ควรรู้จักดังนี้

ฟังก์ชัน `sqrt()` ใช้สำหรับหารากที่สอง มีวิธีใช้คือ

`x = sqrt(ค่าที่ต้องการหารากที่สอง);`

ฟังก์ชัน `sin()` ใช้สำหรับหาค่ามุม `sin` มีวิธีใช้ดังนี้

`x = sin(ค่ามุม);`

เป็นต้น

หมายเหตุ นักเรียนจะสามารถใช้ฟังก์ชันทางคณิตศาสตร์ได้ นักเรียนจะต้องทำการ include ไฟล์ `math.h` ก่อนเท่านั้น

โจทย์ฝึกสมองประลองฝีมือ...

จงเขียนโปรแกรมคอมพิวเตอร์ เพื่อแก้ปัญหาด้านสูตรการเคลื่อนที่แนวเส้นตรงต่อไปนี้

1. $s = (u + v) / 2 * t$
2. $v = u + a * t$
3. $s = (u * t) + (1 / 2 * a * t * t)$

ตัวดำเนินการเปรียบเทียบ

ตัวดำเนินการเปรียบเทียบที่นิยมใช้กันมีดังนี้

== ตัวดำเนินการเท่ากัน

ตัวดำเนินการไม่เท่ากัน !=

>= ตัวดำเนินการมากกว่าหรือเท่ากับ

<= ตัวดำเนินการน้อยกว่าหรือเท่ากับ

> ตัวดำเนินการมากกว่า

< ตัวดำเนินการน้อยกว่า

ตัวอย่างเช่น หากนักเรียนต้องการเปรียบเทียบว่า เท่ากันหรือไม่ จะเขียนได้ 20 และ 10 ดังนี้

10== 20 ซึ่งถ้าเท่ากันจริง โปรแกรมจะคืนค่า 0 แต่ถ้าไม่เท่ากันโปรแกรมจะคืนค่า 1

หรือต้องการเปรียบเทียบว่า จริงหรือไม่ จะเขียนได้ดังนี้ 20 มากกว่าหรือเท่ากับ 10

10>= 20 ซึ่งถ้าจริง โปรแกรมจะคืนค่า 0 แต่ถ้าไม่จริงโปรแกรมจะคืนค่า 1

ตัวอย่างโปรแกรมเพื่อตรวจสอบว่า จริงหรือไม่ 20 มากกว่าหรือเท่ากับ 10

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int x;
    x = 10>=20;
    printf("%d\n",x);
    system("pause");
    return 0;
}
```

ผลลัพธ์

0

หมายความว่า การเปรียบเทียบ 10>= 10 เป็นเท็จ หรือ 20 ไม่ได้มากกว่าหรือเท่ากับ นั่นเอง 20

Text Formatting

คำสั่ง printf() นอกจากจะใช้แสดงผลข้อความธรรมดาๆ แล้ว ยังสามารถจัดรูปแบบให้กับข้อความได้อีกด้วย ซึ่งชื่อเต็มๆของ printf ก็คือ print formatted นั่นเอง

ตัวอย่างโค้ด

```
printf("2 + 2 = %d", 4);
printf("%d + %d = %d", 2, 3, 5);
```

ผลลัพธ์

```
2 + 2 = 4
2 + 3 = 5
```

จะเห็นได้ว่า ข้อความในคำสั่ง printf ด้านบน "2 + 2 = " แสดงผลออกมาตามปกติ มีเพียง %d เท่านั้น ที่ถูกเปลี่ยนไปเป็น 4 นั่นเป็นเพราะคำสั่ง printf จะนำอาร์กิวเมนต์ด้านหลัง มาแทนที่ %d ในอาร์กิวเมนต์แรกตามลำดับ ทั้งนี้ ต้องใช้ให้ถูกประเภทด้วย ซึ่ง %d ใช้สำหรับ เลขจำนวนเต็ม (Integer) ฐานสิบ หากเป็นเลขทศนิยมหรือเลขจำนวนจริง (Floating number) ต้องใช้ %f แทน ซึ่งประเภทต่างๆ ดูได้ตามตารางข้างล่างนี้

Code	Format	Example
%c	ตัวอักษร หรือ อักขระ 1 ตัว	a
%d	เลขจำนวนเต็มฐานสิบ	475
%e	สัญกรณ์ทางวิทยาศาสตร์ (scientific notation) ใช้ตัว e	4.7523e+2
%E	สัญกรณ์ทางวิทยาศาสตร์ (scientific notation) ใช้ตัว E	4.7523E+2
%f	เลขทศนิยม หรือจำนวนจริง	475.23
%g	เป็น %e หรือ %f ตามความเหมาะสม	475.23
%G	เป็น %E หรือ %f ตามความเหมาะสม	475.23
%o	เลขจำนวนเต็มฐานแปด	653
%x	เลขจำนวนเต็มฐานสิบหก ใช้ตัวอักษรตัวเล็ก	1ab
%X	เลขจำนวนเต็มฐานสิบหก ใช้ตัวอักษรตัวใหญ่	1AB
%s	ข้อความ หรือสายอักขระ (ตัวอักษรหลายตัวเรียงต่อกัน)	ABCD
%u	เลขจำนวนเต็มบวก และศูนย์ (Unsigned integer)	385
%%	เครื่องหมาย '%'	

คำสั่งรับข้อมูลและคำสั่งแสดงผล

.1 ฟังก์ชันprintf()

ฟังก์ชัน printf() เป็นฟังก์ชันจากคลังที่มาพร้อมกับตัวแปลโปรแกรมภาษาซี ใช้สำหรับการแสดงผล มีรูปแบบดังนี้

```
printf("สายอักขระควบคุม", ตัวแปร);
```

โดยที่ สายอักขระควบคุมประกอบด้วย 3 ส่วน คือ

ตัวอักขระที่จะแสดง -

รูปแบบการแสดงผล ขึ้นต้นด้วยเครื่องหมายเปอร์เซ็นต์ (%)

- ลำดับหลัก (escape sequence)

ตัวแปร คือ ชื่อของตัวแปรที่จะแสดงผล

ตัวอย่างการใช้งาน

ตัวอย่างที่ การแสดงผลข้อความ มีรูปแบบการใช้งานดังนี้ 1

```
printf("Nattapon Buaurai"); // จะแสดงผลคำว่า Nattapon Buaurai
printf("Hello World");      // จะแสดงผลคำว่า Hello World
```

ตัวอย่างที่ และตัวเลขที่ถูกเก็บไว้ในตัวแปร การแสดงข้อความ 2

```
int a = 50; // ประกาศตัวแปรจำนวนเต็ม a มีค่าเท่ากับ 50
printf("My number is %d", a); // จะแสดงผลคำว่า My number is 50
```

```
float x = 125.75; // ประกาศตัวแปรตัวเลขทศนิยม //x มีค่าเท่ากับ 125.75
printf("My salary is %f dollars", x); // จะแสดงผลคำว่า My salara is 125.75 dollars
```

.2 ฟังก์ชัน scanf()

ฟังก์ชัน scanf() เป็นฟังก์ชันที่ใช้ในการรับข้อมูลจากแป้นพิมพ์ โดยจะบอกเลขที่อยู่ของตัวแปรในหน่วยความจำ แล้วจึงนำค่าที่รับมาเก็บไว้ตามที่อยู่นั้น โดยมีรูปแบบดังนี้

```
scanf("%รูปแบบ", &ตัวแปร);
```

โดยที่ &ตัวแปร หมายถึง เลขที่อยู่ (address) ของตัวแปรที่จะรับค่ามาเก็บในหน่วยความจำ

ตัวอย่างการใช้งาน

ตัวอย่างที่ การรับค่าตัวเลขจำนวนเต็ม 1

```
scanf("%d", &a); // รับค่าตัวเลขจำนวนเต็มเก็บไว้ที่ตัวแปร a

scanf("%d %d",&number1, &number2); // รับค่าตัวเลขจำนวนเต็ม ตัว เก็บไว้ที่ 2
// ตัวแปร number1 และ number2 ตามลำดับ

scanf("%f", &salary); // รับค่าตัวเลขทศนิยมเก็บไว้ที่ตัวแปร salary

scanf("%c",&name); // รับค่าตัวอักษรเก็บไว้ที่ตัวแปร name
```

ในการคำนวณทางคณิตศาสตร์สำหรับภาษาซีนั้น มีฟังก์ชันมาตรฐานที่นักเรียนสามารถนำมาใช้งานได้ โดยฟังก์ชันทางคณิตศาสตร์จะอยู่ในไฟล์ `math.h` ซึ่งหากนักเรียนจะเรียกใช้ฟังก์ชันทางคณิตศาสตร์ต่างๆ นักเรียนจะต้องเรียกใช้ไฟล์ `math.h` เสียก่อน โดยฟังก์ชันทางคณิตศาสตร์ที่นิยมใช้ได้แก่

.1 ฟังก์ชัน `abs(int)`

ฟังก์ชัน `abs(int)` เป็นฟังก์ชันสำหรับหาค่าสมบูรณ์ของจำนวนเต็มใดๆ โดยส่งค่ากลับเป็นค่าสมบูรณ์ของจำนวนเต็มนั้นๆ มีวิธีการใช้งานดังนี้

```
int x, y=-5;
x = abs(y);
printf("Absolute of %d is %d\n",y, x)
```

ผลลัพธ์ที่ได้จะแสดงคำว่า

Absolute of -5 is 5

2. ฟังก์ชัน `sqrt(int)`

ฟังก์ชัน `sqrt(int)` เป็นฟังก์ชันสำหรับหาค่ารากที่สองของจำนวนจริงใดๆ แล้วคืนค่าเป็นจำนวนจริงของรากที่สองของค่านั้น มีวิธีการใช้งานดังนี้

```
float x, y=8;
x = sqrt(y);
printf("Sqrt of %f is %.2f\n",y,x);
```

ผลลัพธ์ที่ได้จะแสดงคำว่า

Sqrt of 8 is 2.83

ทั้งนี้ฟังก์ชันทางคณิตศาสตร์ที่อยู่ในไฟล์ `math.h` ยังมีอีกหลายฟังก์ชัน โดยแต่ละฟังก์ชันมีหลักการใช้งานเช่นเดียวกับตัวอย่างการใช้งานข้างต้น

สำหรับฟังก์ชันอื่นๆ ที่บรรจุอยู่ในไฟล์ math.h ได้แก่

ฟังก์ชัน	การใช้งาน
<code>sqrt(int x)</code>	หาค่ารากที่สองของ x
<code>pow(double x, double y)</code>	หาค่ายกกำลัง x ด้วย y
<code>exp(x)</code>	หาค่าเอกซ์โปเนนเชียลของ x
<code>log(x)</code>	หาค่าลอการิธึมธรรมชาติของ x
<code>log10(x)</code>	หาค่าลอการิธึมสิบของ x
<code>abs(x)</code>	หาค่าสัมบูรณ์ของ x (เมื่อ x เป็นเลขจำนวนเต็ม)
<code>fabs(x)</code>	หาค่าสัมบูรณ์ของ x (เมื่อ x เป็นเลขทศนิยม)
<code>ceil(x)</code>	คืนค่าเลขจำนวนเต็มของ x โดยปัดเศษขึ้น
<code>floor(x)</code>	คืนค่าเลขจำนวนเต็มของ x โดยปัดเศษลง
<code>fmod(x,y)</code>	หาค่า x^y
<code>sin(x)</code>	หาค่า $\sin$ ของ x
<code>cos(x)</code>	หาค่า $\cos$ ของ x
<code>tan(x)</code>	หาค่า $\tan$ ของ x
<code>rand()</code>	สุ่มเลขจำนวนเต็มระหว่าง 0 ถึง RAND_MAX

ตัวอย่างโปรแกรมสำหรับให้นักเรียนพิมพ์ตาม

Program-1

- โปรแกรมแสดงการรับข้อมูล 2 ค่า (X, Y) และแสดงผลบวก (X+Y) ทางจอภาพ

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{ int X, Y, sum;
```

```
printf("Enter X: "); scanf("%d", &X);
```

```
printf("Enter Y: "); scanf("%d", &Y);
```

```
sum = X + Y;
```

```
printf("sum = %d\n", sum);
```

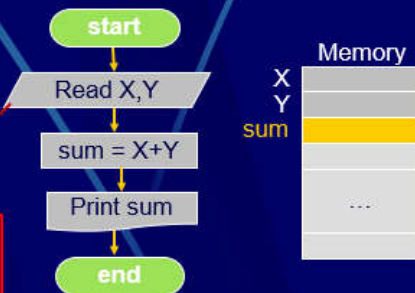
```
getch(); /* get 1 character from keyboard */
```

```
}
```

- printf หมายถึง ฟังก์ชันแสดงผลลัพธ์ (Output) ในภาษา C

- scanf หมายถึง ฟังก์ชันรับข้อมูล (Input) ในภาษา C

- %d หมายถึง ชนิดของข้อมูลแบบ Integer (หรือ Decimal)



Program-2

- โปรแกรมแสดงการรับข้อมูลค่าความยาว (มีหน่วยเป็นฟุต) คำนวณการเปลี่ยนหน่วยเป็นนิ้ว

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int feet, inches;
```

```
printf("Enter number (in feet) ");
```

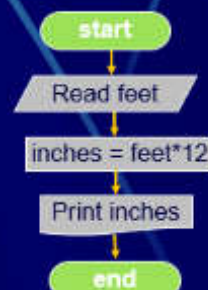
```
scanf("%d", &feet);
```

```
inches = feet*12;
```

```
printf("= %d inches\n", inches);
```

```
getch();
```

```
}
```



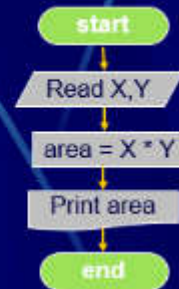
ผลลัพธ์

Enter number (in feet) 2
= 24 inches

Program-3

- โปรแกรมแสดงการรับข้อมูลค่าความกว้าง ความยาวของสี่เหลี่ยม คำนวณพื้นที่และแสดงผล

```
#include <stdio.h>
void main()
{
    int X, Y;
    float area;
    printf("Enter width (X): ");
    scanf("%d", &X);
    printf("Enter length (Y): ");
    scanf("%d", &Y);
    area = X * Y;
    printf("Area = %f\n", area);
    getch();
}
```



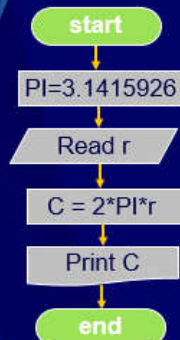
ผลลัพธ์

Enter width (X): 5
 Enter length (Y): 10
 Area = 50.0

Program-4

- โปรแกรมแสดงการรับข้อมูลค่ารัศมีของวงกลม คำนวณ เส้นรอบวง ($2\pi r$) และแสดงผล

```
#include <stdio.h>
void main()
{
    float r, C, PI = 3.1415926;
    printf("Enter radius: ");
    scanf("%f", &r);
    C = 2*PI*r;
    printf("Circumference = %f\n", C);
    getch();
}
```



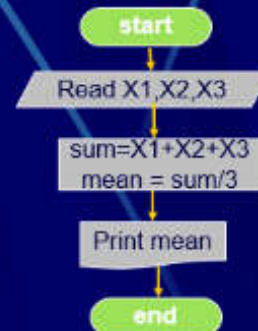
ผลลัพธ์

Enter radius 2
 Circumference = 12.566370

Program-5

- โปรแกรมแสดงการรับข้อมูล 3 ค่า คำนวณค่าเฉลี่ย และแสดงค่าเฉลี่ย ทางจอภาพ

```
#include <stdio.h>
void main()
{
    int X1, X2, X3;
    float sum, mean;
    printf("Enter X1,X2,X3: ");
    scanf("%d,%d,%d", &X1,&X2,&X3);
    sum = X1+X2+X3;
    mean = sum/3;
    printf("Mean = %f\n", mean);
    getch();
}
```



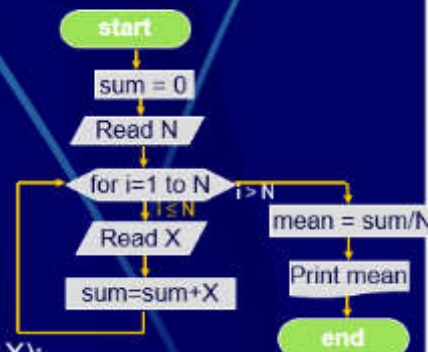
ผลลัพธ์

Enter X1,X2,X3: 5,2,10
Mean = 5.666667

Program-6

- โปรแกรมแสดงการรับข้อมูล N ค่า คำนวณค่าเฉลี่ย และแสดงค่าเฉลี่ย ทางจอภาพ

```
#include <stdio.h>
void main()
{
    int i, N, X;
    float sum=0, mean;
    printf("Enter N: "); scanf("%d", &N);
    for (i=1; i<=N; i++) {
        printf("Enter X%d= ", i); scanf("%d", &X);
        sum = sum + X;
    }
    mean = sum/N;
    printf("Mean = %f\n", mean);
    getch();
}
```



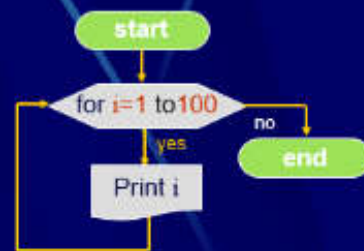
ผลลัพธ์

Enter N: 3
Enter X1= 5
Enter X2= 2
Enter X3= 10
Mean = 5.666667

Program-7

● โปรแกรมแสดงการกำหนดค่า 1, 2, 3, ..., 100

```
#include <stdio.h>
void main()
{
    int i;
    for (i=1; i<=100; i++) {
        printf("%d, ", i);
    }
    getch();
}
```

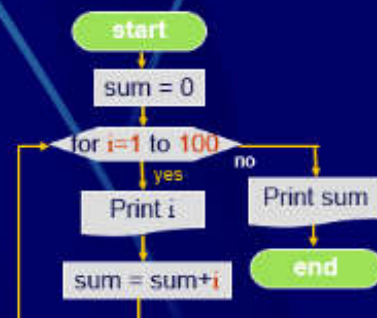


ผลลัพธ์
1, 2, 3, 4, 5, ..., 100

Program-8

● โปรแกรมแสดงการบวก 1+2+...+100

```
#include <stdio.h>
void main()
{
    int i, sum=0;
    for (i=1; i<=100; i++) {
        printf("%d+ ", i);
        sum = sum + i;
    }
    printf(" = %d\n", sum);
    getch();
}
```

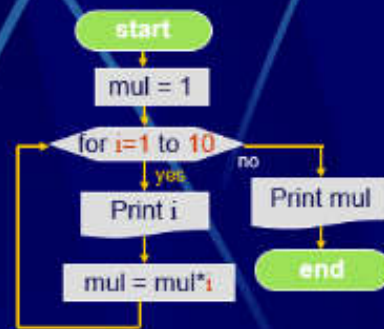


ผลลัพธ์
1+2+3+...+100 = 5050

Program-9

● โปรแกรมแสดงการคูณ $1 \times 2 \times \dots \times 10$

```
#include <stdio.h>
void main()
{
    int i;
    float mul=1;
    for (i=1; i<=10; i++) {
        printf("%d x ", i);
        mul = mul * i;
    }
    printf(" = %f\n", mul);
    getch();
}
```



ผลลัพธ์

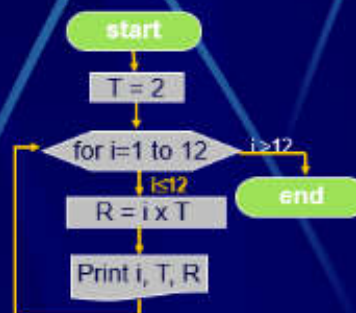
$1 \times 2 \times 3 \times \dots \times 10 = 3628800.0$

Program-10

● โปรแกรมแสดงการคำนวณ ตารางสูตรคูณแม่ 2

กำหนดให้ $T = 2$, $i = 1, 2, \dots, 12$, และ $R = i \times T$

```
#include <stdio.h>
void main()
{
    int i, T=2, R;
    for (i=1; i<=12; i++) {
        R = i * T;
        printf("%d x %d = %d\n", i, T, R);
    }
    getch();
}
```



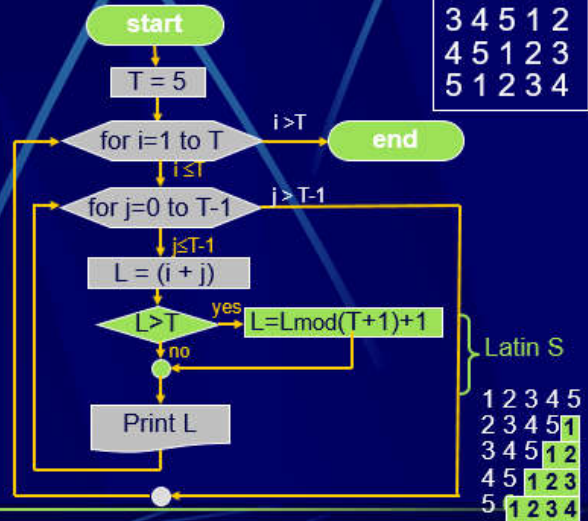
$1 \times 2 = 2$
 $2 \times 2 = 4$
 $3 \times 2 = 6$
 $4 \times 2 = 8$
 $5 \times 2 = 10$
 $6 \times 2 = 12$
 $7 \times 2 = 14$
 $8 \times 2 = 16$
 $9 \times 2 = 18$
 $10 \times 2 = 20$
 $11 \times 2 = 22$
 $12 \times 2 = 24$

Program-11

โปรแกรมแสดงการคำนวณ ตาราง Latin Square

ขนาด 5x5

```
#include <stdio.h>
void main()
{ int i, j, T=5, L;
  for (i=1; i<=T; i++) { /* row i */
    for (j=0; j<T; j++) { /* col j */
      L = (i+j);
      if (L>T) L = L%(T+1)+1;
      printf("%d ", L);
    }
    printf("\n"); /* new row */
  }
  getch();
}
```



หน่วยที่ 2 คำสั่งเลือกทำ

คำสั่ง if

ใช้ตรวจสอบเงื่อนไข ถ้าผลลัพธ์เป็นจริงจะทำชุดคำสั่งที่ตามมา

คำสั่ง if - else

ใช้สำหรับเลือกทำอย่างใดอย่างหนึ่ง

ถ้าเงื่อนไขเป็นจริงจะทำชุดคำสั่งที่ตามหลัง if ถ้าเป็นเท็จทำชุดคำสั่งที่ตามหลัง else

คำสั่ง if ซ้อน

ใช้สำหรับเลือกทำหลาย ๆ อย่างโดยการนำ if หรือ if-else มาซ้อนกัน

```
if(เงื่อนไขทางเลือก(
    คำสั่ง;
```

คำสั่งนี้ใช้ในกรณีที่มีทางเลือกการทำงานอยู่ทางเลือกเดียว
โดยถ้าตรวจสอบเงื่อนไขแล้วเป็นจริงก็จะทำตามคำสั่งนั้น

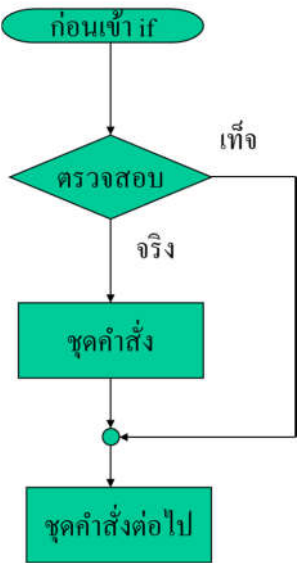
ตัวอย่าง

```
#include <stdio.h>
main()
{
    int x, y, z;
    y = 1; z = 5; x = y * z;
    if (x == (5
        x++;
        printf("x = %d ", x);
}
```

ถ้า x เท่ากับ 5 จริง จะทำดังนี้

ผลการทำงาน

x = 6;



ผังงานของคำสั่ง if

ตัวอย่าง

```
#include <stdio.h>

main()
{
    int age;
    printf("How old are you ");
    scanf("%d",&age);
    if (age > 70)
        printf("You are old \n");
    printf("Good bye !");
}
```

ข้อสังเกต
Good bye จะทำทุกครั้ง

ตัวอย่าง

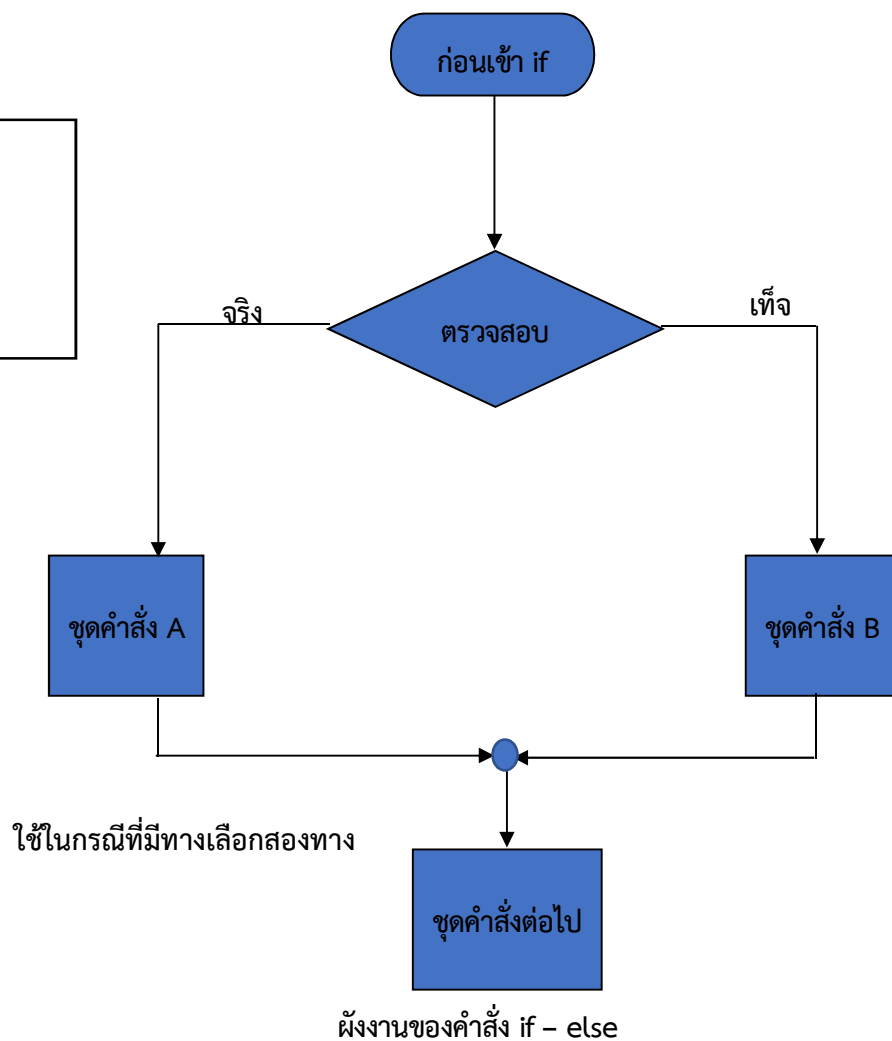
```
#include <stdio.h>
main()
{
    char ch1,ch2;
    ch1 = 'g';
    ch2 = 'k';
    printf("ch1 = %d\n",ch1);
    printf("ch2 = %d\n",ch2);
    if (ch2 > ch1)
        printf("ch2 is more than ch1\n");
}
```

ข้อสังเกต

จะเห็นว่าการเปรียบเทียบตัวอักษร ระบบจะนำรหัส ASCII มาเปรียบเทียบกัน

คำสั่ง if - else

```
if(เงื่อนไขทางเลือก(
คำสั่ง 1;
else
คำสั่ง 2;
```



ตัวอย่าง

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    int x, y;
```

```
    scanf("%d %d", &x, &y);
```

```
    if (x > y)
```

```
        printf("x is greater than y ");
```

```
    else
```

```
        printf("x is less than or equal to y ");
```

```
}
```

โปรแกรมจะรับตัวเลขสองค่า
มาเก็บในตัวแปร x และ y
จากนั้นจะนำมาเปรียบเทียบกัน

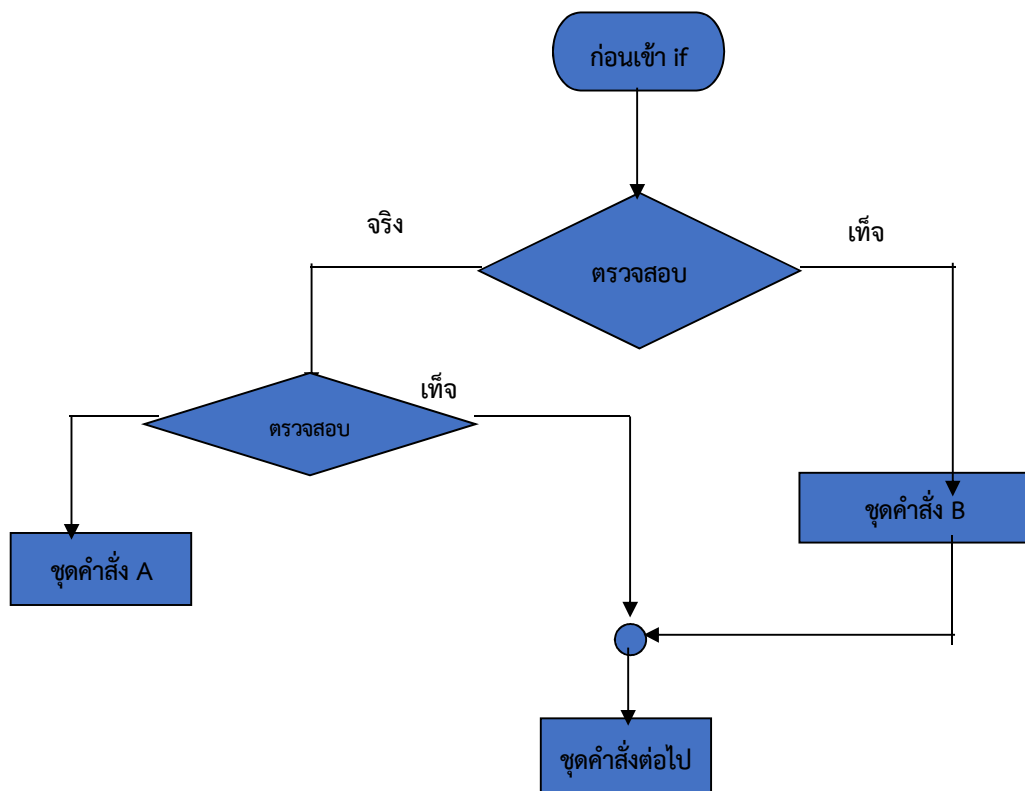
ตัวอย่าง

```
#include <stdio.h>
main()
{
    int x;
    printf("Enter integer value : ");
    scanf("%d",&x);
    if (x % 2 != 0)
        printf("\n%d is odd.\n ",x);
    else
        printf("\n%d is even.\n ",x);
}
```

หากรับค่า x เท่ากับ 7 จะพบว่านิพจน์
เปรียบเทียบ $x \% 2 \neq 0$ มีค่าเป็นจริง
จึงทำให้คำสั่ง ล่าง if ทำงาน

คำสั่ง if - else แบบซ้อน

```
if(เงื่อนไขทางเลือก1(
    คำสั่ง 1;
else if(เงื่อนไขทางเลือก2)
    คำสั่ง 2;
else if(เงื่อนไขทางเลือก3)
    คำสั่ง 3;
.
.
else
    คำสั่ง n;
```



ผังงานของคำสั่ง if - else แบบซ้อน

ตัวอย่าง รับค่าคะแนนแล้วแสดงเกรด

```
main()
{
    float score;    char grade;
    scanf("%f ", &score);
    if (score > 100)
        printf(" Score must be less than or equal to 100");
    else if (score >= 80)
        grade = 'G';
    else if (score >= 50)
        grade = 'P'
    else
        grade = 'F';
    printf("\n Grade = %c", grade);
}
```

ตัวอย่าง รับค่าแล้วตรวจสอบเป็นช่วง

```
main()
{
    int score;
    printf("Score : ");
    scanf("%d ", &score);
    if ((score >= 80) && (score <= 100))
        printf(" A\n");
    else if ((score >= 70) && (score <= 79))
        printf(" B\n");
    else if ((score >= 60) && (score <= 69))
        printf(" C\n");
    else if ((score >= 50) && (score <= 59))
        printf(" D\n");
    else
        printf("You got F !!!\n ");
}
```

คำสั่ง switch..case

ใช้สำหรับการเลือกทำที่มีทางเลือกหลายทาง

โปรแกรมจะตรวจสอบค่านิพจน์หลัง switch ว่า

มีค่าเท่ากับค่าคงที่หลัง case ค่าใด ก็จะกระโดดไป

ทำสแตตเมนต์หลังค่าคงที่ค่านั้น

แต่ถ้าหากไม่เท่ากับค่าคงที่ค่าใดเลยก็จะไปทำ

สแตตเมนต์หลัง default โดยค่า default นี้จะมีหรือ

ไม่มีก็ได้

```
switch (นิพจน์)
{
    case ค่าคงที่ 1 : statement A;
                      break;
    case ค่าคงที่ 2 : statement B;
                      break;
    .....
    case ค่าคงที่ n : statement N;
                      break;
    default :      statement;
}
```

variable : เป็นตัวแปรชนิด int หรือ char หรือเป็นผลที่ได้จากการคำนวณแล้วเป็น int หรือ char

constant_1, constant_2 : เป็นค่าคงที่ที่เป็นตัวแปรชนิดเดียวกับ variable

break : เป็นคำสั่งสำหรับออกจากการทำงานของ switch..case จะต้องใส่ไว้ทางตอนท้ายของ ทุก case

default : ถ้าหากค่าของ variable ไม่ตรงกับค่าคงที่ใดเลย จะต้องทำสแตตเมนต์นี้

ตัวอย่าง

```
#include <stdio.h>
main()
{
    int value;
    scanf("%d",&value );
    switch (value % 2)
    {
        case 0 : printf("Even integer \n");
        case 1 : printf("Odd integer\n");
    }
}
```

ถ้าค่า value % 2 เท่ากับ 0
จะแสดง
Even integer
Odd integer
ถ้าค่า value % 2 เท่ากับ 1
จะแสดง
Odd integer

ตัวอย่าง

```
scanf("%d",&value );
switch (value % 2)
{
    case 0 :
        printf("Even integer \n");
        break;
    case 1 :
        printf("Odd integer\n");
        break;
}
```

แก้ไขโดยใช้คำสั่ง break เพื่อให้หยุดและออกจาก switch

ตัวอย่าง

```
#include <stdio.h>
main(){
    char sex;
    printf("are you m(male) of f (female) ? ");
    scanf("%c",&sex);
    switch (sex)
    {
        case 'm' : printf("You are male \n"); break;
        case 'f' : printf("You are female\n"); break;
        default : printf("Try again !!!\n");
    }
}
```

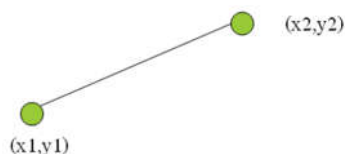
โจทย์ คำสั่งเลือกทำ

1) จงเขียนโปรแกรมสำหรับการกด ATM โดยป้อนตัวเลขเข้าไป แล้วให้จำนวนแบงค์ออกมา เช่น

INPUT 4800

OUTPUT	B1000	=	4
	B500	=	1
	B100	=	3

2) จงเขียนโปรแกรมหาระยะระหว่างจุดสองจุด อินพุตเป็น (x1,y1) และ (x2,y2)



3) จงเขียนโปรแกรมรับตัวเลขจำนวน 5 จำนวน เมื่อกรอกครบ แล้วแสดงผลว่าตัวเลขที่รับเข้ามาเป็นจำนวนคู่ และคี่อย่างละกี่จำนวน (กรณียังไม่ใช้ loop ให้รับ 5 จำนวน)

input : 5

#1 : 2

#2 : 5

#3 : 3

#4 : 1

#5 : 10

even : 2 odd: 3

4) จงเขียนโปรแกรมวัดความเหมาะสมระหว่างส่วนสูงกับน้ำหนักด้วยการหาค่าดัชนีมวลกาย (Body Mass Index หรือ BMI) ตามสูตรต่อไปนี้ แล้วบอกด้วยว่า ผอม ปกติ อ้วน

$$BMI = \frac{Weight (kg)}{Height^2 (m^2)}$$

5) จงเขียนโปรแกรมรับค่าเลขจำนวนเต็ม (N) จากคีย์บอร์ด และพิมพ์ตัวเลขตัวสุดท้ายของ N เป็นข้อความ กำหนดให้ 0 = Zero, 1 = One, 2= Two, 3=Three, 4 =Four, 5= Five, 6 =Six, 7= Seven, 8 = Eight, 9 = Nine

ตัวอย่างการแสดงผล

Please enter number : 1062

Two

6) จงเขียนโปรแกรมคำนวณภาษีของพนักงาน ให้โปรแกรมรับหมายประจำตัวพนักงานจำนวน 5 หลัก ฐานเงินเดือน และโอทีที่ได้ กำหนดให้ ถ้าเงินเดือนรวมตั้งแต่ 100000 บาท เสียภาษี 10% เงินเดือนตั้งแต่ 70000 บาท เสียภาษี 7% เงินเดือน 50000 บาท เสียภาษี 5% เงินเดือน 30000 บาท เสียภาษี 3% นอกนั้นเสีย 1%

Output เลขประจำตัว ยอดเงินรวม และยอดภาษีที่ต้องจ่าย

7) จงเขียนโปรแกรมคำสั่ง switch case รับค่าของคะแนนสอบกลางภาค 100 คะแนน และคะแนนสอบปลายภาค 100 คะแนน จากคีย์บอร์ดและตัดเกรดตามเงื่อนไขต่อไปนี้

คะแนน $80 \leq x \leq 100$ จะได้ grade = 'G' หมายถึง Good

คะแนน $50 \leq x < 80$ จะได้ grade = 'P' หมายถึง Pass

คะแนน $0 \leq x < 50$ จะได้ grade = 'F' หมายถึง Fail

(กำหนดให้โปรแกรมนี้อาจทำงานก็ต่อเมื่อคะแนนที่กรอก ไม่น้อยกว่า 0 หรือ มากกว่า 100)

ตัวอย่างการแสดงผล

Please enter midterm score : 80 (Enter)

Please enter final score : 90 (Enter)

Your Score = 85 %

Grade = G , Good

8) ให้แก้สมการ $ax^2 + bx + c = 0$ จงหาค่า x ว่ามีค่าเท่าไร โดย ให้โปรแกรมระบุ ว่า รับ a เป็นค่าใด รับ b เป็นค่าใด รับ c เป็นค่าใด แล้วให้โปรแกรมตอบว่าค่า x เป็นเท่าไร

9) ให้รับอักขระ 1 ตัว โดย

ถ้าตัวอักขระที่ป้อนเข้าไปนั้นเป็นตัวพิมพ์ใหญ่ให้แสดงผลเป็นตัวพิมพ์ใหญ่

ถ้าตัวอักขระที่ป้อนเข้าไปนั้นเป็นตัวพิมพ์เล็กให้แสดงผลเป็นตัวพิมพ์ใหญ่

แล้วพัฒนาโปรแกรมต่อ โดย ถ้าป้อนตัวอักขระเข้าไปแต่ไม่ใช่ตัวอักษรให้แสดงคำว่า error

10) นาย “ซีพลัสพลัส” ได้ไปซื้อเครื่องจับเวลามา 1 เครื่อง และเขาก็นำเครื่องจับเวลานั้นมาจับเวลาที่เขาใช้วิ่งออกกำลังกายไปใน 1 รอบสัปดาห์ และเมื่อเขาเริ่มวิ่งและจับเวลา เขาก็พบว่าเครื่องจับเวลาของเขานั้นจับเวลาในหน่วยของวินาที ปัญหาก็คือ นาย “ซีพลัสพลัส” อยากจะรู้ว่าเขานั้นได้วิ่งไปด้วยเวลาทั้งหมดเท่าไรในหน่วยของวัน ชั่วโมง นาที และ วินาที ตัวอย่างหน้าจอแสดงผล

```

*****
Enter Second of Times:: 166505
*****
166505 This Time is 1 Days
80105 This Time is 22 Hours
905 This Time is 15 Minutes
5 This Time is 5 Seconds
*****
Summation      = 166505
                = 1 Days 22 Hours 15 Minutes and 5 Seconds
*****

```

11) ให้ทำการรับปี ค.ศ. จากผู้ใช้โปรแกรม หลังจากนั้นให้ทำการแปลงเลขปี ค.ศ. ที่รับเข้ามานั้นให้กลายเป็นปี พ.ศ. หลังจากนั้นให้ทำการคำนวณผลรวมของเลขแต่ละหลักของปี ค.ศ. และ พ.ศ. ที่ได้มานั้น แสดงผลลัพธ์ สุดท้ายยกทางจอภาพ ดังต่อไปนี้

```

Input Christian Era = 2000
Change to Buddhist Era ==> 2543
Sum of A.D. = 2 + 0 + 0 + 0 = 2
Sum of B.E. = 2 + 5 + 4 + 3 = 14

```

12) จงเขียนโปรแกรมคำนวณหาค่าของ y เมื่อกำหนดเงื่อนไขดังนี้

$x = 1, 2$	$y = 2x + 1$
$x = 4, 6-8$	$y = 4x$
$x = 9-10, 12, 15-18$	$y = 20 - x$
$x = \text{other}$	$y = x$

13) จงเขียนโปรแกรมคำนวณหาค่าของ y เมื่อกำหนดเงื่อนไขดังนี้

$$y = \begin{cases} 2x & , & x = 1, 2 \\ 3x + 2 & , & 5 \leq x \leq 8 \\ \sqrt{x} & , & x = 3 \text{ หรือ } 9 \leq x \leq 12 \\ x^2 - 5 & , & x = 4 \\ x & , & x = \text{other} \end{cases}$$

14) จงเขียนโปรแกรมคำนวณค่าน้ำประปาโดยรับจำนวน Units ที่ใช้เข้าไปโดย (คำนวณในอัตราก้าวหน้า)

- 1) 0 - 10 Unit ละ 5 11 - 50 Unit ละ 10
51 - 100 Unit ละ 12 101 - 500 Unit ละ 15
501 - 1000 Unit ละ 18 1001 - 5000 Unit ละ 20
5001 ==> Unit ละ 21
- 2) 0 - 50 Unit ละ 5 51 - 150 Unit ละ 10
151 - 250 Unit ละ 15 251 ==> Unit ละ 20
- 3) [0, 10) Unit ละ 3.50 บาท [10, 50) Unit ละ 4.50 บาท
[50, 100) Unit ละ 6.00 บาท [100, xxxx] Unit ละ 7.00 บาท

15) ด้านซังน้ำหนัก กรมศุลกากร ณ ท่าอากาศยานกรุงเทพฯ หรือสนามบินดอนเมืองนั้น มีหน้าที่รับผิดชอบในเรื่องของการตรวจสอบการนำเข้าของสินค้า และทำการเรียกเก็บภาษีการนำเข้าสินค้า โดยจะมีเกณฑ์ที่ใช้ในการเรียกเก็บภาษีไม่เท่ากัน ซึ่งขึ้นกับประเภทของสินค้า และน้ำหนักโดยรวม หรือไม่ก็ขึ้นกับประเภท หรือลักษณะของสินค้า นายเอ ได้นำเข้าสินค้าประเภทเครื่องซังเข้ามาใช้งาน ในธุรกิจของตน โดยเครื่องซังนั้น กรมศุลกากรได้กำหนดเกณฑ์ในการเรียกเก็บภาษีไว้ดังนี้ คือ

- ถ้าเครื่องซังแสดงน้ำหนักไม่เกิน 20 กิโลกรัม เครื่องละ 200 บาท
- แสดงน้ำหนักเกิน 20 กิโลกรัม แต่ไม่เกิน 100 กิโลกรัม เครื่องละ 500 บาท
- แสดงน้ำหนักเกิน 100 กิโลกรัม แต่ไม่เกิน 2,000 กิโลกรัม เครื่องละ 1,000 บาท
- แสดงน้ำหนักเกิน 2,000 กิโลกรัม แต่ไม่เกิน 10,000 กิโลกรัม เครื่องละ 3,000 บาท
- แสดงน้ำหนักเกิน 10,000 กิโลกรัม เครื่องละ 10,000 บาท

กำหนดให้นักเรียนเขียนโปรแกรมเพื่อนำไปใช้งานที่ด้านศุลกากร โดยมีข้อกำหนด คือ โปรแกรมจะทำการรับน้ำหนักสูงสุดของเครื่องซัง (MaxWeight) เพื่อนำมาแยกประเภท พร้อมทั้งรับจำนวนการนำเข้า (Piece) เพื่อคำนวณหาค่าภาษี (TaxValue) ที่ต้องเรียกเก็บ

หน่วยที่ 3 คำสั่งควบคุมแบบการทำซ้ำ

คำสั่งควบคุมการทำซ้ำหรือคำสั่งวนซ้ำ เป็นคำสั่งที่ใช้ควบคุมการทำงานของโปรแกรมแบบวนซ้ำ การทำงานเดิม ๆ ตามเงื่อนไขที่กำหนด เช่น การวนซ้ำการทำงานเดิมเป็นจำนวน รอบ ทำงานซ้ำ ๆ จนกว่าเงื่อนไขจะเป็น 10 เท็จ สำหรับภาษาC จะมีชุดคำสั่งให้ทำงานเป็นรอบอยู่ รูปแบบด้วยกัน คือ 3

คำสั่ง .1.1for

ใช้กับลูปที่ทราบรอบการทำงานแน่นอน -

-ทำซ้ำจนกว่าเงื่อนไขที่กำหนดเป็นเท็จแล้วจึงออกนอกลูป

คำสั่ง .1.2while

-ใช้กับลูปที่ไม่ทราบจำนวนทำซ้ำที่แน่นอน

ทำซ้ำโดยตรวจสอบเงื่อนไขก่อนเข้าลูป -

การตรวจสอบเงื่อนไข
จะใช้ตัวดำเนินการทางตรรก

คำสั่ง .1.3do-while

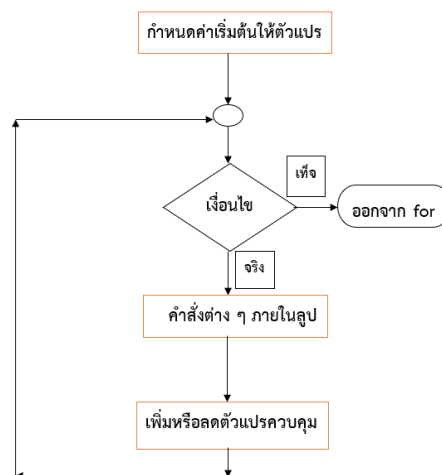
ทำซ้ำโดยตรวจสอบเงื่อนไขหลังจากทำงานในลูปไปแล้วหนึ่งครั้ง -

1. .1การทำซ้ำวนรอบที่แน่นอนด้วยคำสั่ง for

ประโยคคำสั่ง for มีรูปแบบการเขียนดังนี้ คือ

รูปแบบ :

```
for(exp1; exp2; exp3)
    statement;
หรือ
for(exp1; exp2; exp3)
{
    statement();
}
หรือ
for(initial;condition;change);
```



โดยที่ :	initial	หมายถึง	กำหนดค่าเริ่มต้นของตัวแปรที่เป็นเงื่อนไขตัวอย่างเช่น x = 0
	condition	หมายถึง	นิพจน์เงื่อนไขที่กำหนดขึ้นสำหรับการวนลูป
	change	หมายถึง	ส่วนของการเปลี่ยนแปลงค่าของตัวแปรที่ใช้เป็นเงื่อนไข

```
for (letter = 'a'; letter < 'z';letter++)
```

```
printf("%c ",letter);
```

ลักษณะการทำงานของวนซ้ำด้วยคำสั่ง for

1. คำสั่งภายในตัววนซ้ำ for จะทำงานเมื่อเป็นจริง (True)
2. การออกจากวนซ้ำเมื่อคำสั่งเงื่อนไขเป็นเท็จ (False)
3. การทำงานของวนซ้ำ จะเริ่มจากค่าเริ่มต้นที่กำหนด
4. จำนวนรอบที่ทำงานจะขึ้นอยู่กับค่านิพจน์ที่กำหนดไว้
5. การเพิ่มค่า counter ให้กับวนซ้ำส่งผลต่อจำนวนรอบที่ทำงาน เช่น ให้ทำซ้ำ 10 รอบก็สามารถเขียนชุดคำสั่ง ดังนี้

```
for (i=1; i <=10; i++)
```

จากคำสั่ง จะพบว่า

1. มีการกำหนดค่าเริ่มต้นของตัวนับจำนวนรอบเท่ากับ 1 (i = 1)
2. ตัวนับจำนวนรอบจะทำงาน 10 รอบ (i <= 10)
3. แต่ละรอบให้มีการเพิ่มจำนวนตัวนับจำนวนรอบทีละ 1 (i++)

ตัวอย่างโปรแกรมการทำงานของวนซ้ำด้วยคำสั่ง for: 01

```
1 #include <stdio.h>
2 main()
3 {
4     int number;
5     for(number = 1; number <=10; number++)
6         printf("%3d",number);
7 }
```

C:\Users\User\Desktop\test C62\Untitled1.exe

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Process exited after 3.847 seconds with return value 3
Press any key to continue . . .

ตัวอย่างโปรแกรมการทำงานของวนซ้ำด้วยคำสั่ง for: 02

```
1 #include <stdio.h>
2 main()
3 {
4     char ch;
5     for(ch = 'A'; ch <= 'Z'; ch++)
6         printf("  %c ",ch);
7
8 }
```

C:\Users\User\Desktop\test C62\Untitled1.exe

A	B	C	D	E	F	G	H	I	J	K
L	M	N	O	P	Q	R	S	T	U	V
W	X	Y	Z							

Process exited after 4.094 seconds with return value 91
Press any key to continue . . .

ตัวอย่างโปรแกรมการทำงานของวนซ้ำด้วยคำสั่ง for: 03

```

1  #include <stdio.h>
2  main()
3  {
4      int n;
5      for(n = 10; n <= 50; n+=10)
6          printf("\n%d",n);
7
8  }

```

เพิ่มครั้งละ 10

C:\Users\User\Desktop\test C62\Untitled1.exe

10
20
30
40
50

Process exited after 3.954 seconds with return value 3
Press any key to continue . . .

ตัวอย่างโปรแกรมการทำงานของวนซ้ำด้วยคำสั่ง for: 04

```

1  #include <stdio.h>
2  main()
3  {
4      int n = 5;
5      for( ; n > 0; n--)
6          printf("%d ",n);
7  }

```

C:\Users\User\Desktop\test C62\Untitled1.exe

5 4 3 2 1

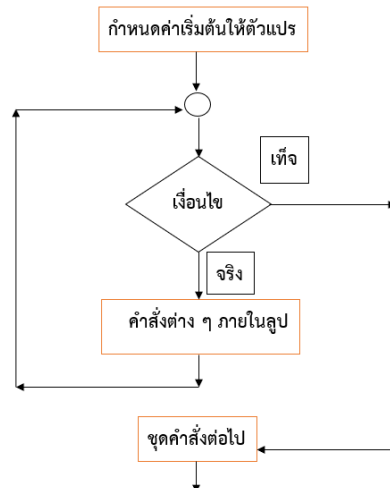
Process exited after 3.761 seconds with return value 4
Press any key to continue . . .

1.2 การวนซ้ำการทำงานเมื่อเงื่อนไขเป็นจริงด้วยคำสั่ง while

ประโยคคำสั่ง while มีรูปแบบการเขียนดังนี้ คือ

รูปแบบ :

```
while (เงื่อนไข)
    statement();
หรือ
while (เงื่อนไข)
{
    statement();
}
```



อาจไม่ทำในลูปเลยก็ได้ถ้าเงื่อนไขเป็นเท็จ

ลักษณะการทำงานของลูปwhile

1.ชุดคำสั่งภายในการวนซ้ำ while จะทำงานเมื่อเป็นจริง)True)

2.การออกจากลูปเมื่อคำสั่งเงื่อนไขเป็นเท็จ)false)

3.เงื่อนไขหรือนิพจน์ที่นำมาตรวจสอบ สามารถใช้ตัวดำเนินการเปรียบเทียบ หรือดำเนินการทางตรรกะได้ เช่น

```
while (i = 10
```

```
while (i >=10 && j <= 15
```

```
while (j > 10&& (b < 15|| C < 20
```

ตัวอย่างโปรแกรมการทำงานเมื่อเงื่อนไขเป็นจริงด้วยคำสั่ง while: 01

```

1  #include <stdio.h>
2  main()
3  {
4      int n = 1;
5      while (n <= 5 )
6      {
7          printf("\n%d",n);
8          n++;
9      }
10 }
```

C:\Users\User\Desktop\test C62\Untitled1.exe

```

1
2
3
4
5
-----
Process exited after 5.382 seconds with return value 2
Press any key to continue . . .
```

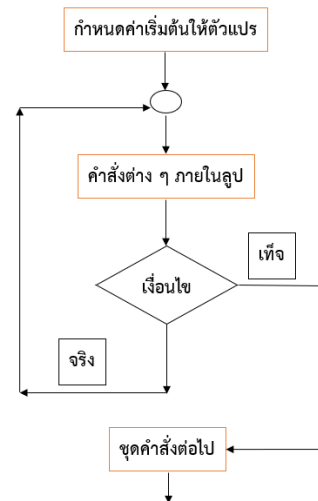
1.3. การวนลูปการทำงานอย่างน้อย 1 ครั้ง ด้วยคำสั่ง do...while

ประโยคคำสั่ง do...while มีรูปแบบการเขียนดังนี้ คือ

รูปแบบ :

```
do
    statement();
while (เงื่อนไข);
```

อาจไม่ทำในลูปเลยก็ได้ถ้าเงื่อนไขเป็นเท็จ



ตัวอย่างโปรแกรมการทำงานอย่างน้อย 1 ครั้ง ด้วยคำสั่ง do...while01 :

```

1  #include <stdio.h>
2  main()
3  {
4      int n = 1;
5      do
6      {
7          printf("\n%d",n);
8          n++;
9      }while (n <= 5);
10 }
```

C:\Users\User\Desktop\test C62\Untitled1.exe

```

1
2
3
4
5
-----
Process exited after 4.362 seconds with return value 2
Press any key to continue . . .
```

การวนลูปซ้อน

```
1 #include <stdio.h>
2 int n, m;
3 main()
4 {
5     for (n = 1; n <= 2; n++)
6         for (m = 1; m <= 3; m++)
7             printf("\n%d %d", n, m);
8 }
```

Select C:\Users\User\Desktop\test C62\Untitled1.exe

```
1 1
1 2
1 3
2 1
2 2
2 3

Process exited after 1.553 seconds with return value 3
Press any key to continue . . .
```

แบบฝึกหัด

- .1 วิชาฟิสิกส์แบ่งการเก็บคะแนนดังนี้ สอบกลางภาค %40 การบ้าน %20 สอบปลายภาค %40 ถ้าหากคะแนนทุกส่วนเต็ม จงเขียนโปรแกรมตัดเกรดของวิชานี้ 100 โดยให้กำหนดเงื่อนไขของเกรดขึ้นมาเอง
- .2 เงินฝากธนาคารมีดอกเบี้ย %5 ต่อปี ถ้าหากฝากเงิน ปีจะมีเงินเท่าไร 10 บาท อยากทราบว่ามีเงิน 2000, ถ้าปีจะมีเงินเกิน บาท 4000
- .3 จงเขียนโปรแกรมแก้สมการ $Ax^2 + Bx + C = 0$
- .4 จงเขียนโปรแกรมรับตัวเลข ค่า แล้วหาค่าเฉลี่ย 10
- .5 จงเขียนโปรแกรมรับตัวเลข ค่า แล้วบอกค่าสูงสุดและต่ำสุด 10
- .6 จงเขียนโปรแกรมรับตัวเลข แล้วบอกว่า เป็นจำนวนเฉพาะหรือไม่
- .7 จงเขียนโปรแกรมรับตัวเลข แล้วแจ้งว่าตัวเลขตั้งแต่ ถึงเลขนั้นมีจำนวนเฉพาะกี่ตัว พร้อมแสดงจำนวนเฉพาะ 1

ออกมาทางจอภาพ

โจทย์คำสั่งควบคุมการทำซ้ำ

โจทย์ loop ให้รับค่า : `1int` ตัวเป็นจำนวนเต็มบวก จากนั้นให้พิมพ์จำนวนที่หารด้วยจำนวนที่รับมาทั้งคู่ลงตัวตั้งแต่ 2 ถึง 1000 - 1

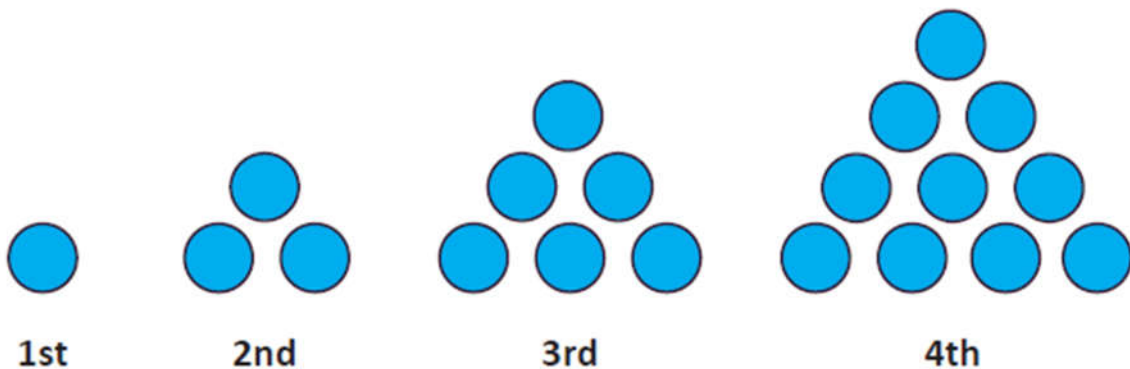
โจทย์ loop ให้รับค่าตัวอักษรภาษาอังกฤษหนึ่งตัว จากนั้นให้พิมพ์ตัวอักษรภาษาอังกฤษย้อนหลังจนถึงตัว : `2A` (ถ้าใส่ตัวพิมพ์เล็กก็ให้แปลงเป็นตัวพิมพ์ใหญ่)

โจทย์ loop3 : ให้เขียนโปรแกรมรับเลขจำนวนเต็ม ตัวเก็บไว้ในตัวแปร `1d` ไม่เกิน 20 จากนั้นให้พิมพ์รูปสี่เหลี่ยมผืนผ้าขนาด `d X d` โดยใช้ *

โจทย์ loop 4 : ให้เขียนโปรแกรมรับเลขจำนวนเต็ม ตัวเก็บไว้ในตัวแปร `1d` ไม่เกิน 20 จากนั้นให้พิมพ์รูปสามเหลี่ยมสูง `d` ยาว `d` โดยใช้ *

โจทย์ loop 5 : จำนวนเชิงสามเหลี่ยม (Triangular Number) ได้แก่ ... ,10 ,6 ,3 ,1

คือจำนวนที่สามารถสร้างรูปสามเหลี่ยมได้ดังแผนภาพ



ให้เขียนโปรแกรมเพื่อพิมพ์จำนวนเชิงสามเหลี่ยมทั้งหมดตั้งแต่ 2000 ถึง 1

โจทย์เพิ่มเติมคำสั่งควบคุมการทำงานซ้ำ

1. จงเขียนโปรแกรมรับค่า N แล้วตรวจสอบว่าตั้งแต่ 1 ถึง N มีจำนวนเฉพาะกี่จำนวน
2. เขียนโปรแกรม แปลง N เป็นเลขฐานสอง
3. รับข้อมูล N ค่า แต่ละค่าอยู่ระหว่าง 0-100 แล้วให้คอมพิวเตอร์แจ้งว่า ค่า MAX และ MIN คือค่าใด พร้อมบอกผลรวมและค่าเฉลี่ย เมื่อป้อนตัวเลขไม่ตรงกับช่วง 0-100 ให้โปรแกรมหยุดทำงาน
4. รับข้อความจากคีย์บอร์ด และ นำตัวอักษรแต่ละตัวมาตรวจสอบว่ามีสระ A E I O U อยู่กี่ตัว
5. iPhone20 ราคา 20,000 บาท แม่มีเงินอยู่ 500 บาท ธนาคารให้ดอกเบี้ยร้อยละ 3 ก็ปีแม่ถึงจะซื้อ iPhone20 ให้นักเรียนได้
6. ให้ฝากเงิน 10 ปี ปีละ N บาท เท่า ๆ กัน ดอกเบี้ยปีละ x% ถามว่าปีที่ 15 จะได้เงินเท่าใด
7. ให้รับค่า N แล้วแสดงผล * จำนวน N บรรทัด โดย * แต่ละบรรทัดมีจำนวนเท่ากับค่า N เช่น N=3

Output

```
*
**
***
```

8. ให้รับค่า N น้อยกว่าหรือเท่ากับ 4 แล้วให้แสดงผลดังนี้

Output

```
*
**
***
****
```

9. ให้รับค่า N โดยค่า N มีค่าไม่เกิน 24 แล้วให้แสดงผล ดังต่อไปนี้
เช่น N = 3 (แล้วลองพัฒนาต่อเป็น input N= 5)

Output

```
*      *
**     **
*****
**     **
*      *
```

10. ให้รับค่า N โดยค่า N มีค่าไม่เกิน 26 แล้วให้แสดงผลเป็นตัวอักษรภาษาอังกฤษ N บรรทัด โดยเรียงลำดับอักษรในแต่ละบรรทัด เช่น N=4

Output

```
A
AB
ABC
ABCD
```

11. ให้รับค่า N โดยค่า N มีค่าน้อยกว่าหรือเท่ากับ 200 แล้วให้แสดงผล ดังต่อไปนี้ เช่น N=4
A
BC
DEF
GHIJ

(โดยหากค่าที่รับเข้ามามีค่ามากกว่าจำนวนตัวอักษรภาษาอังกฤษให้วนค่า A ใหม่)

12.รับ N ตั้งแต่ 2-20 แล้ววาดสี่เหลี่ยมกลวงไป

2 <pre> ** ** </pre>	3 <pre> *** * * *** </pre>	5 <pre> ***** * * * * * * ***** </pre>
-----------------------------	-----------------------------------	---

13. ให้โปรแกรมแสดงผลดังต่อไปนี้ เช่น input 5

```

* _ * _ *
_ * _ * _
* _ * _ *
_ * _ * _
* _ * _ *

```

14. ให้โปรแกรมแสดงผลดังต่อไปนี้ เช่น input 5

```

      *
    * _ *
  * _ * _ *
* _ * _ * _ *
* _ * _ * _ * _ *
  * _ * _ * _ *
    * _ * _ *
      * _ *
        *

```

หน่วยที่ 4 โจทย์ประยุกต์

1. เขียนโปรแกรมรับค่าจำนวนเต็ม 1 จำนวน (n) และพิมพ์ผลลัพธ์ x^2 ตั้งแต่ 1 – n จำนวน

Input :	1		3		5	
Output :	x	x ²	x	x ²	x	x ²
	1	1	1	1	1	1
			2	4	2	4
			3	9	3	9
					4	16
					5	25

2. เขียนโปรแกรมรับค่าจำนวนเต็ม 1 จำนวน (n) และพิมพ์ผลลัพธ์ 2^n ตั้งแต่ 1 – n จำนวน

Input :	1		3		5	
Output :	n	2 ⁿ	n	2 ⁿ	n	2 ⁿ
	1	2	1	2	1	2
			2	4	2	4
			3	8	3	8
					4	16
					5	32

3. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป . (เมื่อเขียนได้แล้วให้ทำแบบกลับหัว)

input	3			
Output		1		
		1		1
	1	2		1
input	4			
Output			1	
		1		1
		1	2	1
	1	3	3	1

4. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป

Input	1		2		3
Output	1		1 1		1 1
			2		2 2
			1 1		3
					2 2
					1 1

5. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป

Input	1	2	3
Output	A	B B	C C
		A	B B
		B B	A
			B B
			C C

6. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป

Input :	1	3	5
Output :	0	0	0
		101	101
		21012	21012
			3210123
			432101234

7. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป

Input :	1	3	5
Output :	1	1	1
		121	121
		12321	12321
			1234321
			123454321

8. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป

Input :	1	3	5	6
Output :	1	1	1	1
		2 4	2 4	2 4
		3 6 9	3 6 9	3 6 9
			4 8 12 16	4 8 12 16
			5 10 15 20 25	5 10 15 20 25
				6 12 18 24 30 36

9. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป (สามเหลี่ยมปาสคาล)

```

Input :      1      3      5      6
Output :      1
              1 1      1 1      1 1
            1 2 1    1 2 1    1 2 1
              1 3 3 1    1 3 3 1
                1 4 6 4 1    1 4 6 4 1
                  1 5 10 10 5 1
  
```

10. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป

```

Input :      1      3      5      6
Output :      A
              A      A      A
            CB      CB      CB
          FED      FED      FED
            JIHG      JIHG
          ONMLK      ONMLK
            UTSRQP
  
```

11. เขียนโปรแกรมรับเลขจำนวนเต็ม 1 จำนวนความสูงหิ้งอกหินย้อย แล้วแสดงผลลัพท์ (หลังจากเรียนเรื่องอาร์เรย์แล้วให้พัฒนาโดยการรับจำนวนลูกของหินงอกหินย้อยแล้วรับความสูงของหินงอกหินย้อยแต่ละลูก)

```

Input :      1      2      3      4
Output :      V      \ /      \ /      \ /
              V      \ /      \ /
                V      \ /
                  V
  
```

12. เขียนโปรแกรมรับเลขจำนวนเต็ม 1 จำนวนความสูงภูเขา แล้วแสดงผลลัพท์ (หลังจากเรียนเรื่องอาร์เรย์แล้วให้พัฒนาโดยการรับจำนวนลูกของภูเขาแล้วรับความสูงของภูเขาแต่ละลูก)

```

Input :      1      2      3      4
Output :      ^      ^      ^      ^
              / \      / \      / \
                / \      / \
                  / \
  
```

13.เขียนโปรแกรมรับทิศทางลูกศร ถ้ารับเลข)1 ลูกศรหันหน้าไปทางซ้าย ถ้ารับเลข 2 ลูกศรหันหน้าไปทางขวาและ (หลังจากเรียนเรื่องอาร์เรย์แล้วให้พัฒนาโดยการรับจำนวนลูกศรและรับขนาดของลูกศรแต่ละลูก) รับขนาดของลูกศร

Input :	1	2	1
	3	4	2
Output :	/	\	/
	/	\	/
	/	\	/
	\	\	\
	\	/	\
		/	
		/	
		/	

14.แล้วแสดงสามเหลี่ยมตัวเลขและผลรวมตัวเลขที่ถูกล้อมกรอบ เช่น เขียนโปรแกรมรับเลขจำนวนเต็ม .

Input	1	2	3	6	9
Output	1	1	1	1	1
	No Answer	1 2	1 2	1 2	1 2
		No Answer	1 2 3	1 2 3	1 2 3
			No Answer	1 2 3 4	1 2 3 4
				1 2 3 4 5	1 2 3 4 5
				1 2 3 4 5 6	1 2 3 4 5 6
				16	1 2 3 4 5 6 7
					1 2 3 4 5 6 7 8
					1 2 3 4 5 6 7 8 9
					77

15. เขียนโปรแกรมบวกเลขฐาน x โดย x มีค่าตั้งแต่ เมื่อรันโปรแกรมจะให้ป้อนอินพุตค่าแรกเป็นเลข 16 ไปจนถึง 2 ฐานของตัวเลขจากนั้นป้อนตัวเลขเข้าไปสองค่า โปรแกรมจะแสดงผลบวกของเลขฐานนั้นออกมา

Base	16	5
Num1	12A	24
Num2	321	32
Output	44B	111

16. iPhone x ราคาเครื่องละ 40,000 บาท มีเงินอยู่ 500 บาท ธนาคารให้ดอกเบี้ยร้อยละ 3 ก็ปีถึงจะซื้อ iPhone x ได้

17. เขียนโปรแกรมรับเลขจำนวนเต็ม แล้วแสดงผลดังรูป (แสดงจำนวนตัวอักษร)

Input :	1	6	12	30
Output :	A	A	A	A
		BC	BC	BC
		DEF	DEF	DEF
			GHIJ	GHIJ
			KL	KLMNO
				PQRSTU
				VWXYZAB
				CD

18. หอคอยตัวอักษร

input	1	2	3	4
output	A	A	A	A
		B C	B C	B C
			F E D	F E D
				G H I J

ถ้าอักขระเกินค่า Z ให้วนมา A ใหม่

19. ชายคนหนึ่งอาบน้ำทุกๆ วัน โดยแต่ละวันจะใช้น้ำ ชัน แต่จะมีเงื่อนไขว่า 5

ถ้าวันนั้นเป็นวันสี่และตรงกับวันพุธจะอาบน้ำ ชัน 3

ถ้าวันนั้นเป็นวันสี่และตรงกับวันศุกร์จะอาบน้ำ ชัน 2

โดยโปรแกรมจะรับค่าดังนี้

(1จำนวนวันทั้งหมด

(2วันที่เริ่มนับวันแรกตรงกับวันอะไร กำหนดให้ เท่ากับวันเสาร์ 7 เท่ากับวันอาทิตย์ 1

โปรแกรมจะแสดงจำนวนชันที่ชายนั้นใช้อาบน้ำตามจำนวนวันที่ได้รับค่าไว้

20. เขียนโปรแกรมรับเลขจำนวนเต็ม 1 จำนวน แล้วแสดงผลลัพธ์ย้อนกลับ

Input :	38	5673	980123	47629145
Output :	83	3765	321089	54192674

หน่วยที่ 5 ตัวแปรอาร์เรย์และการประยุกต์

ข้อมูลชนิดอาร์เรย์

-12	23	45	65	12	27	86
-----	----	----	----	----	----	----



ตัวแปรอาร์เรย์เก็บจำนวนเต็ม 7 จำนวน

12.8	85.21	32.1	23.9	43.5
------	-------	------	------	------



ตัวแปรอาร์เรย์เก็บทศนิยม 5 จำนวน

แถวลำดับ (Array)

ตัวแปรประเภทอาร์เรย์ เป็นตัวแปรที่สามารถเก็บข้อมูลหลายๆ ค่าไว้ในตัวแปรชื่อเดียวกันได้ โดยระบบจะใช้พื้นที่หน่วยความจำต่อเนื่องกัน เพื่อเก็บข้อมูลชนิดเดียวกันหลายจำนวน

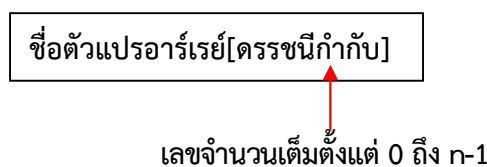


ตัวอย่าง

- `int n[10];`
ประกาศตัวแปรอาร์เรย์ชื่อ `n` มีขนาด 10 หน่วย แต่ละหน่วยเก็บเลขจำนวนเต็ม
- `char a[20];`
ประกาศตัวแปรอาร์เรย์ชื่อ `a` มีขนาด 20 หน่วย แต่ละหน่วยเก็บตัวอักษร
- `float g[5];`
ประกาศตัวแปรอาร์เรย์ชื่อ `g` มีขนาด 5 หน่วย แต่ละหน่วยเก็บเลขทศนิยม

***ถ้าหากประกาศตัวแปรเป็นสตริง ตัวแปรนั้นก็คืออาร์เรย์ของ `char`

การอ้างถึงสมาชิกในอาร์เรย์



ตัวอย่าง

`int n[5];`

`n[4] = 25;`

สามารถอ้างถึงสมาชิกแต่ละหน่วยของอาร์เรย์ `n[ ]` โดยใช้ `n[0], n[1], n[2], n[3]` และ `n[4]`

ตัวอย่าง

ถ้าหากมีข้อมูลกลุ่มหนึ่งเป็นคะแนนของนักศึกษา 8 คน สามารถเก็บได้ดังนี้

หมายเลข	X[0]	X[1]	X[2]	X[3]	X[4]	X[5]	X[6]	X[7]
คะแนน	18	20	35	84	21	45	65	74

ถ้าหากมีการอ้างถึงอาร์เรย์อาจเป็นดังต่อไปนี้

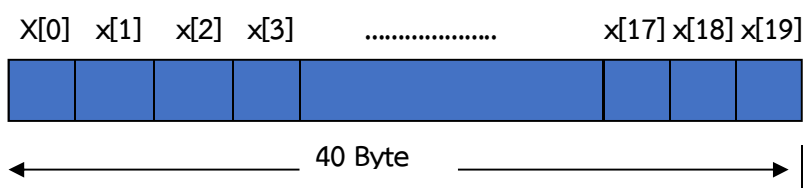
X[2]	อ้างถึงเซลล์ที่ 2 มีค่าเท่ากับ 35
X[2] + X[3]	นำเซลล์ที่ 2 บวกกับเซลล์ที่ 3 จะได้ 35 + 84 เท่ากับ 119
X[1+3]	อ้างเซลล์ที่ 4 มีค่าเท่ากับ 21
X[5] + 1	นำเซลล์ที่ 5 มาบวกด้วย 1 จะได้เท่ากับ 46

ขนาดหน่วยความจำของอาร์เรย์

ขึ้นอยู่กับประเภทของข้อมูลและจำนวนสมาชิกที่จองไว้

ตัวอย่าง

```
int x[20];
```



จองเนื้อที่ช่องละ 2 ไบต์ จำนวน 20 ช่อง รวมเป็น 40 ไบต์

```
char name[20];
```



ใช้พื้นที่ 20 ไบต์

ตัวอย่าง โปรแกรมรับข้อมูล 10 ค่า แล้วหาผลรวมของข้อมูลเหล่านั้น

```
#include <stdio.h>
```

```
main(){
```

```
    int num[10],sum,i;
```

```
    for(i = 0; i < 10; i++)
```

```
        scanf("%d",&num[i]);
```

```
    sum = 0;
```

```
    for(i=0;i<10;i++)
```

```
        sum = sum + num[i];
```

```
    printf("sum is %d\n",sum);
```

```
}
```

การกำหนดค่าเริ่มต้นให้อาร์เรย์

ชนิดของข้อมูล ชื่อตัวแปรอาร์เรย์[ขนาด] = {value-list};

ตัวอย่าง

```
int n[5] = {1,4,9,16,25};
```

```
char a[3] = {'A','B','C'};
```

```
int pw[ ] = {1,2,4,8,16,32,64,128};
```

```
char name[ ] = "COMPUTER";
```



ถ้าไม่ระบุขนาด โปรแกรมจะจองหน่วยความจำให้อย่าง

โปรแกรมอ่านค่าจากอาร์เรย์ และแสดงเป็นกราฟแท่ง

```
#define SIZE 10
```

```
main(){
```

```
    int i,j;
```

```
    int n[SIZE] = {19,3,15,7,11,9,13,5,17,1};
```

```
    printf("%s%13s%17s\n", "Element", "Value", "Histogram");
```

```
    for(i=0; i<=SIZE-1; i++)    {
```

```
        printf("%7d%13d    ", i, n[i]);
```

```
        for(j = 1; j<= n[i]; j++)
```

```
            printf("%c", '*');
```

```
        printf("\n");
```

```
    }
```

```
}
```

ตัวอย่าง

สมมติให้ i, j, k เป็นตัวแปรประเภท int และประกาศตัวแปร table เป็นอาร์เรย์เก็บจำนวนเต็ม

```
for (int k = 0; k < 9; k++)
```

```
    printf ("Value at %d = %d\n", k+1, table[k]);
```

```
table[i + j] = 0;
```

```
table[7 - table[j]] = j;
```

สิ่งที่ต้องระวัง

ในภาษาซีจะไม่มีการกำหนดให้ตรวจสอบขอบเขตของอาร์เรย์ โปรแกรมเมอร์จะต้องพยายามเขียนโปรแกรมที่เกี่ยวข้องกับสมาชิกของอาร์เรย์ภายในขอบเขตที่ประกาศอาร์เรย์ไว้ หากมีการอ้างอิงถึงสมาชิกอาร์เรย์นอกขอบเขตที่ได้ระบุไว้ เช่น table[12] สิ่งที่ได้คือการไปอ่านข้อมูลในพื้นที่ของหน่วยความจำที่อาจจะเก็บค่าของตัวแปรตัวอื่นหรือค่าอื่นใดที่ไม่อาจคาดเดาได้

ตัวอย่าง

ให้อ่านค่าของจำนวนเต็ม จำนวนจากคีย์บอร์ด และแสดงผลในลำดับที่กลับกัน 5

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
main () {
```

```
    int k;
```

```
    int table[SIZE];
```

```
    for (k = 0; k < SIZE; k++)
```

```
        scanf ("%d", &table[k]);
```

```
    for (k = SIZE-1; k >= 0; k--)
```

```
        printf ("%d\n", table[k]);
```

```
}
```

1 2 3 4 5

5 4 3 2 1

สมาชิกของอาร์เรย์อาจเป็นประเภทข้อมูลพื้นฐานใด ๆ ก็ได้ หรืออาจเป็นข้อมูลประเภท Enumeration เช่น

```
#define TSIZE      10
```

```
#define NAMESIZE   20
```

```
#define ADDRSIZE   30
```

```
enum month { JAN, FEB, MAR, APR, MAY,
             JUN, JUL, AUG, SEP, OCT,
             NOV, DEC }
```

```
typedef enum month Month;
```

```
int age[TSIZE];
```

```
float size[TSIZE+1];
```

```
Month date[8];
```

```
char name[NAMESIZE], address[ADDRSIZE];
```

แบบฝึกหัดเพิ่มเติม

1. จงเขียนฟังก์ชันตรวจสอบว่ากต y/Y หรือ n/N หรือไม่
2. เขียนฟังก์ชันรับข้อมูล 10 ค่าแล้วหาผลรวม
3. เขียนโปรแกรมหาค่า sin, cos, tan โดยมีมุมเป็นองศา ตั้งแต่ 0 ถึง 90 โดยให้กระโดดครั้งละ 5
4. เขียนโปรแกรมหาค่า $f(x) = x^2 + 1$
5. เขียนโปรแกรมหาพื้นที่ใต้กราฟของ $y(x) = x^2 - 3x + 2$
6. ถ้าหากมีข้อมูลอยู่ 10 ค่า จงเขียนโปรแกรมเรียงข้อมูลจากค่าน้อยที่สุดไปหาค่ามากที่สุด
7. จำนวนเฉพาะคือจำนวนเต็มบวก ซึ่งหารด้วยจำนวนอื่นไม่ลงตัว ยกเว้น 1 กับตัวมันเอง ตัวอย่างเช่น 7 เป็นจำนวนเฉพาะ แต่ 6 ไม่ใช่จงสร้างตารางจำนวนเฉพาะ n จำนวนแรก
8. จงเขียนโปรแกรมแปลงเลขฐานสิบเป็นเลขฐานสอง

อาร์เรย์สองมิติ

การประกาศตัวแปรอาร์เรย์สองมิติจะใช้ดัชนี 2 ตัว เพื่อระบุจำนวนสมาชิกในแต่ละหลัก และแต่ละแถว ดังนี้

ชนิดข้อมูล ชื่อตัวแปรอาร์เรย์[Row][Column]

ตัวอย่าง

```
int AB[2][3];
```

จะมีสมาชิกทั้งหมด 6 ตัว (2×3) การอ้างสมาชิกแต่ละตัวทำได้ดังนี้

แถวที่ 0 AB[0][0], AB[0][1], AB[0][2]

แถวที่ 1 AB[1][0], AB[1][1], AB[1][2]

การกำหนดค่าเริ่มต้นให้อาร์เรย์ มิติ 2

ตัวอย่าง

```
int sqr[3][3] = {
    1, 2, 3,
    4, 5, 6,
    7, 8, 9
};

int B[2][2] = { {1,2},{3,4} };
```

โปรแกรมอ่านข้อมูลจำนวนเต็มจากตัวแปรอาร์เรย์สองมิติ แล้วนำข้อมูลทั้งหมดมาบวกกัน

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    int i, j, sum;
```

```
    int b[5][4];
```

```
    sum = 0;
```

```
    for(i = 0; i < 5; i++)
```

```
        for(j = 0; j < 4; j++)
```

```
        {
```

```
            scanf("%d", &b[i][j]);
```

```
            sum = sum + b[i][j];
```

```
        }
```

```
    printf("The sum is %d\n",sum);
```

```
}
```

การเก็บข้อมูลในอาร์เรย์สองมิติ

```
main()
```

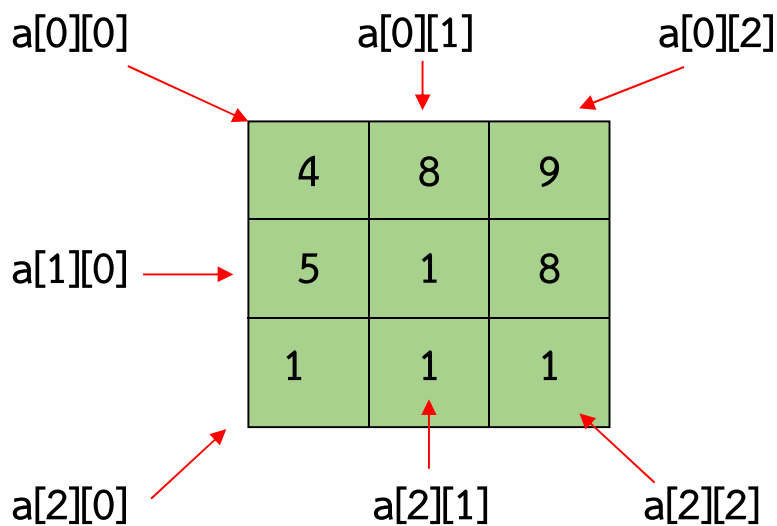
```
{
```

```
    int a[3][3];
    a[0][0] = 4;
    a[0][1] = 8;
    a[0][2] = 9;
```

```
    a[1][0] = 5;
    a[1][1] = 1;
    a[1][2] = 8;
```

```
    a[2][0] = 1;
    a[2][1] = 1;
    a[2][2] = 1;
```

```
}
```



การเก็บข้อมูลในอาร์เรย์สองมิติ

```
main()
```

```
{
```

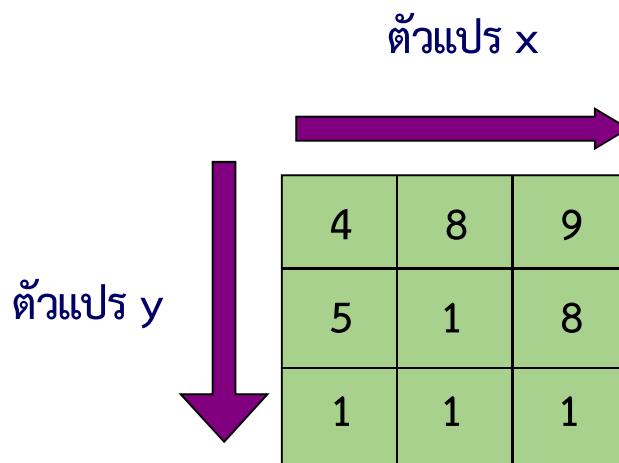
```
    int a[3][3];
    int x, y;
    for(y=0; y<3; y++)
```

```
{
```

```
        for(x=0; x<3; x++)
            printf("%d\t", a[y][x]);
        printf("\n");
```

```
}
```

```
}
```



การส่งตัวแปรอาร์เรย์เป็นอาร์กิวเมนต์

```
#include <stdio.h>
```

```
void funct(int [ ]);
```

```
void main( )
```

```
{
```

```
    int arrayp[20];
```

```
    .....
```

```
    funct(arrayp);
```

```
    .....
```

```
}
```

```
void funct(int arraya[ ])
```

```
{
```

```
    .....
```

```
}
```

เรียกใช้ฟังก์ชัน funct() โดยมี arrayp[] เป็นอาร์กิวเมนต์

โปรแกรมอ่านข้อมูลจำนวนเต็มจากตัวแปรอาร์เรย์ แล้วนำข้อมูลทั้งหมดมาบวกกัน

```
#include <stdio.h>
```

```
float sum(float [ ], int);
```

```
main()
```

```
{
```

```
    int i;
```

```
    float item[100];
```

```
    for(i = 0; i < 100; i++)
```

```
        scanf("%f", &item[i]);
```

```
    printf("Sum is %f\n",sum(item,100));
```

```
}
```

```
float sum(float a[ ], int n)
```

```
{
```

```
    int i;
```

```
    float s = 0.0;
```

```
    for(i = 0; i < n; i++)
```

```
        s = s + a[i];
```

```
    return (s);
```

```
}
```

โจทย์คำสั่งอาร์เรย์สองมิติ

โจทย์ array1 : รับเมตริกซ์ขนาด $m \times n$ 2 ชุด แล้วนำเมตริกซ์ 2 ชุดนั้นมาบวกกัน

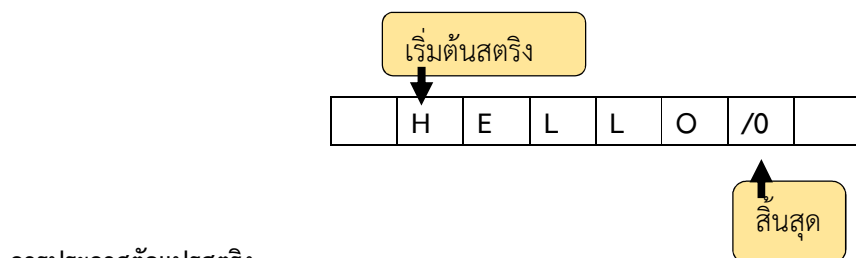
โจทย์ array2 : รับเมตริกซ์ขนาด $m \times n$ 1 ชุด แล้วแสดงผลเมตริกซ์ทรานสโพส

โจทย์ array3 : รับเมตริกซ์ขนาด $m \times n$ 2 ชุด แล้วนำเมตริกซ์ 2 ชุดนั้นมาคูณกัน

หน่วยที่ 6 ตัวแปรสตริง และขั้นตอนวิธีอักขระ

ข้อมูลแบบสตริง

เป็นตัวแปรแบบอักขระมาต่อเรียงกัน โดยใช้ตัวอักขระ null หรือ “\0” เป็นตัวสิ้นสุดสตริง



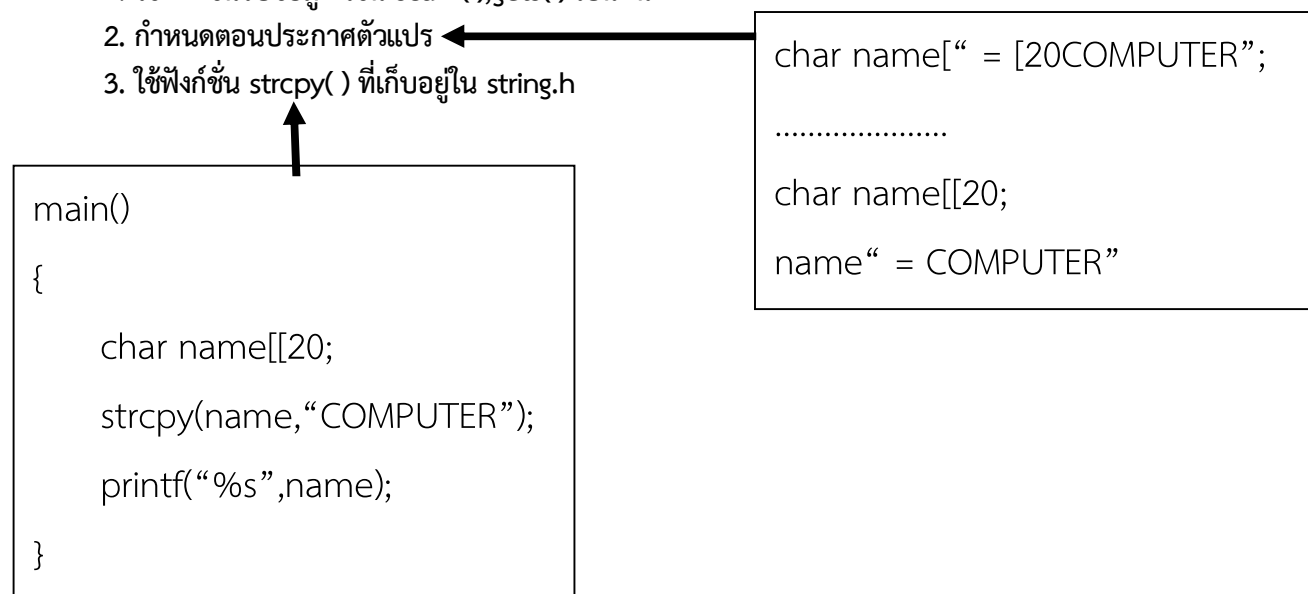
การประกาศตัวแปรสตริง

char ชื่อตัวแปร[n];

ขนาดของสตริงต้องมากกว่าจำนวนตัวอย่างน้อยหนึ่ง

การใส่ค่าในตัวแปรสตริง ทำได้ดังนี้

1. ใช้ฟังก์ชันรับข้อมูล เช่น scanf(), gets() เป็นต้น
2. กำหนดตอนประกาศตัวแปร
3. ใช้ฟังก์ชัน strcpy() ที่เก็บอยู่ใน string.h



ฟังก์ชันของตัวแปรสตริง

ฟังก์ชัน strcat()

นำสตริงสองตัวมาต่อกัน มีรูปแบบดังนี้
 strcat(สตริง1 , สตริง2);

ฟังก์ชัน strcmp()

นำสตริงสองตัวมาเปรียบเทียบกัน มีรูปแบบดังนี้
 strcmp(สตริง 1, สตริง 2);

ผลการเปรียบเทียบ	ค่าที่ส่งกลับ
สตริง1 < สตริง2	จำนวนลบ
สตริง1 = สตริง 2	ศูนย์
สตริง1 > สตริง 2	จำนวนบวก

ฟังก์ชัน strcpy()

คัดลอกสตริงต้นทางไปไว้ปลายทาง มีรูปแบบดังนี้
 strcpy(สตริงปลายทาง , สตริงต้นทาง);

ฟังก์ชัน strlen()

นับจำนวนอักขระในสตริง มีรูปแบบดังนี้
 strlen(สตริง);

```

1  #include <stdio.h>
2  int n, m;
3  main()
4  {
5      int i;
6      char str1[20];
7      strcpy(str1, "Have a nice day");
8      printf("%s\n", str1);
9      i = strlen(str1);
10     printf(" %d\n", i);
11 }
12

```

```

C:\Users\User\Desktop\test C62\Untitled1.exe
Have a nice day
15

Process exited after 3.852 seconds with return value 4
Press any key to continue . . .

```

โจทย์คำสั่งตัวแปรสตริงและขั้นตอนอักขระ

โจทย์ string1 : ให้เขียนโปรแกรมรับข้อความ ตัวอักษร จากนั้นให้หาว่า 100 ข้อความยาวไม่เกิน 1 มีตัวอักษรตัวพิมพ์เล็กและตัวพิมพ์ใหญ่กี่ตัว

โจทย์ string2 : ให้เขียนโปรแกรมรับข้อความ ตัวอักษร 100 ยาวไม่เกิน (จะมีสัญลักษณ์อะไรอยู่ด้วยก็ได้) ข้อความ 1 จากนั้นให้พิมพ์ข้อความนั้นโดยพิมพ์ออกมาเฉพาะตัวอักษรภาษาอังกฤษที่อยู่ในข้อความ

โจทย์ string3 : ให้เขียนโปรแกรมรับข้อความ ตัวอักษร จากนั้นให้พิมพ์ข้อความนั้น 100 ข้อความยาวไม่เกิน 1 ย้อนหลัง

โจทย์ string4 : ให้เขียนโปรแกรมรับข้อความ จากนั้น ตัว 1 ตัวอักษร และรับตัวอักษรอีก 100 ข้อความ ยาวไม่เกิน 1 ให้พิมพ์ข้อความเดิมโดยตัดตัวอักษรตัวดังกล่าวออกไป

โจทย์ string5 : ให้เขียนโปรแกรมรับข้อความ ตัวอักษร และให้ตัดตัวอักษรที่ซ้ำกันที่อยู่ 100 ข้อความ ยาวไม่เกิน 1 ติดกัน เช่น THAIILLANDD จะได้ THAILAND

โจทย์ string6 : เขียนโปรแกรมรับคำสตริง แล้วทำการตรวจสอบมีอักขระที่ติดกันซ้ำกันหรือไม่ ถ้ามีต้องแทรกตัว x ลงไประหว่างตัวที่ซ้ำกัน เช่น input : TEERAWAT output : TExERAWAT

โจทย์ string7 : เขียนโปรแกรมรับคำสตริง ถ้าเป็นตัวเล็กให้แปลงเป็นตัวใหญ่ ถ้าเป็นตัวใหญ่และอักขระอื่นๆ ให้คงไว้เหมือนเดิม

โจทย์ string8 : รหัสลับรหัสรัก ให้เก็บข้อความเป็น string โดยกำหนดให้คีย์ของการเข้ารหัสเป็น 1+ เช่น ถ้าข้อความเดิมเป็น i love u ให้พิมพ์ j mpwf v

โจทย์ string9 : เขียนโปรแกรมรับคำสตริง แล้วพิมพ์ผลลัพธ์แยกสตริง เป็น ชุด ตัวเว้นตัว 2

Input :	Computer	Olympic	String
Output :	Cmue	Oypc	Srn
	optr	lmi	tig

โจทย์ string10 : รหัสลับซีซาร์ รับเลขจำนวนเต็ม จำนวน เพื่อแสดงถึงจำนวนการเลื่อนตัวอักขระ และรับข้อความที่ 1 ถูกเข้ารหัส ให้เขียนโปรแกรมแสดงข้อความก่อนที่จะถูกเข้ารหัส

Input	3	5
	– Vruh g Nlhunhjddug –	NS BFW,
Output	– Soren Kierkegaard –	IN WAR,

โจทย์ string11 : อะนาแกรม คือ คำในภาษาอังกฤษที่ใช้ชนิดตัวอักษรและจำนวนของอักษรแต่ละชนิดเท่ากันทุกประการ ซึ่งอาจจะต่างกันที่ลำดับการใช้ตัวอักษรเท่านั้นที่อาจสลับกันหรือไม่ก็ได้ ตัวอย่างเช่น ANT กับ TAN เป็นอะนาแกรม เนื่องจากทั้งสองคำมีตัวอักษรใช้งานอยู่ ชนิด คือ 3A, N, และ T โดยตัวอักษรแต่ละชนิดมีใช้อยู่ละ ตัว 1 จงเขียนโปรแกรมเพื่ออ่านข้อมูลนำเข้า ที่มีคำภาษาอังกฤษอยู่สองบรรทัด บรรทัดละหนึ่งคำ แล้วตรวจสอบว่าคำทั้งสองเป็นอะนาแกรมซึ่งกันและกันหรือไม่ โดยพิมพ์ว่า yes เมื่อคำที่อ่านมาทั้งสอง เป็นอะนาแกรมซึ่งกันและกัน หรือพิมพ์คำว่า no เมื่อคำที่อ่านมา ไม่เป็นอะนาแกรมซึ่งกันและกัน

กำหนดให้คำภาษาอังกฤษดังกล่าว ประกอบด้วยตัวพิมพ์ใหญ่ จาก A ถึง Z หรือ อักษรตัวเล็กจาก a ถึง z เท่านั้น และไม่มีช่องว่างภายในคำ ทั้งนี้ตัวอักษรที่มีความหมายเดียวกัน ต่างแต่เพียงรูปแบบ เช่น)A กับ a หรือ D กับ d ให้ (นับเป็นชนิดเดียวกัน

Input : บรรทัดแรกเป็นจำนวนเต็ม n ($0 < n < 10,000$) ซึ่งบอกจำนวนคู่ของวลี
บรรทัดถัดมาอีก $2n$ บรรทัด เป็นวลีภาษาอังกฤษ โดยแต่ละบรรทัดมีความยาว
ไม่เกิน ตัวอักษร 20,000

Output : บรรทัดแรกเป็นคำตอบว่าคำทั้งสองในข้อมูลนำเข้าข้างต้นเป็นอะนาแกรมซึ่งกันและกันหรือไม่ โดยพิมพ์คำว่า yes เมื่อคำที่อ่านมาทั้งสองเป็นอะนาแกรมซึ่งกันและกัน และพิมพ์คำว่า no เมื่อไม่เป็นอะนาแกรมซึ่งกันและกัน

input	3 SmIlLe MILes Ton Ant Tom Marvolo Riddle I am Lord VOLDEMORT
output	yes no yes

โจทย์ string12 : อักษรมหาสนุก ให้รับสายอักขระ ชุดมีความยาวเท่ากัน แต่ในสายอักขระแต่ละตัวมีข้อมูลไม่ซ้ำ 2 2 กัน ให้นำสายอักขระตัวแรกมาวางและนำสายอักขระตัวที่ตามมาหาตัวที่ซ้ำกัน โดยวางซ้ำตัวที่เหมือนกันในแนวตั้ง เช่น

input	C T M R Q	B A Y M K
	X A B T E	N D X R Q
output	X	B A Y M K N
	A	D
	B	X
	C T M R Q	R
	E	Q

โจทย์ string13 : เขียนโปรแกรมรับสตริง)สายอักขระมาบวกกัน 2 จำนวน แล้วนำตัวเลขทั้ง 2 เป็นตัวเลข (

Input :	111	952	5
	55	123	9
Output :	166	1075	14

โจทย์ string14 : “ลายผ้ามาน”

ให้รับข้อมูลเป็น sting ที่มีความยาวตั้งแต่ 1-20

โดยลายผ้ามานเกิดจากการนำ input มาเรียงต่อกันตามความกว้างและความยาว โดยเมื่อสิ้นสุด)

ความยาวให้ขึ้นลายผ้าใหม่โดยเลื่อน sting ไปครึ่งละ อักษร กำหนดให้ความยาวผ้ามาน 1 = 80

ความยาว = 10 (

เช่น กำหนดความยาวผ้ามาน = 17 กว้าง = 4

Input : BHLMP 17 4 output : BHLMPBHLMPBHLMPBH

HLMPBHLMPBHLMPBHB

LMPBHLMPBHLMPBHBH

MPBHLMPBHLMPBHBHL

โจทย์ string15 : เขียนโปรแกรมรับคำสตริง แล้วตรวจสอบว่าสตริงที่รับเข้ามาเป็นพาลินโดมหรือไม่ ถ้าเป็น ให้แสดงข้อความ It is Palindrome ถ้าไม่ใช่ ให้แสดงว่า It is not a Palindrome

Input :	Civic	Madam	Computer
Output :	It is Palindrome	It si Palindrome	It is not a Palindrome

หน่วยที่ 7 พอยน์เตอร์ และข้อมูลโครงสร้าง

พอยน์เตอร์(Pointer) หรือตัวชี้ ตัวแปรประเภทนี้จะไม่เก็บข้อมูล แต่จะเป็นค่าตำแหน่งหรือแอดเดรส (Address) ของข้อมูล จะพบมากในโปรแกรมประเภทโครงสร้างข้อมูลเช่น Stack, Queue, Linked list เป็นต้น

การประกาศตัวแปรทั่วไป

```
int age = 23;  ← ตัวแปร age เก็บ 23
char ch = 'a'; ← ตัวแปร ch เก็บ a
```

การประกาศตัวแปรพอยน์เตอร์

ประเภทของข้อมูล *ชื่อตัวแปร

ตัวอย่างการประกาศตัวแปรพอยน์เตอร์

```
int a;
int *p;  ← ประกาศตัวแปร p เป็นพอยน์เตอร์
a = 10;
p = &a; ← นำค่าแอดเดรสที่เก็บ 10 ไปใส่ในตัวแปร p
```

การประกาศตัวแปรพอยน์เตอร์

<pre>int *pt_x;</pre>	สร้างตัวแปรพอยน์เตอร์ชนิด int ทำให้ pt_x ใช้เก็บตำแหน่งที่อยู่ของตัวแปรชนิด int เท่านั้น
<pre>float *pt_num;</pre>	สร้างตัวแปรพอยน์เตอร์ชนิด float ทำให้ pt_num ใช้เก็บตำแหน่งที่อยู่ของตัวแปรชนิด float เท่านั้น
<pre>char *pt_ch;</pre>	สร้างตัวแปรพอยน์เตอร์ชนิด char ทำให้ pt_ch ใช้เก็บตำแหน่งที่อยู่ของตัวแปรชนิด char เท่านั้น

ตัวอย่างแสดงค่าแอดเดรส

```
#include <stdio.h>
void main(){
    int age;
    int *pointer;
    age = 25;
    pointer = &age;
    printf("Address = %p\n",pointer); ← แสดงตำแหน่งค่าแอดเดรส
}
```

ให้นักเรียนพิมพ์โค้ดตัวอย่างแล้วบันทึกผลการทำงาน

```
#include<stdio.h>

Int main()
{
    int x; int *p;
    printf("Address = %p \n",p);
    p++;
    printf("Address = %p\n",p);
    x = 5;
    p = &x;
    printf("Address = %p\n",p);
    printf("Data = %d\n",*p);
    return 0;
}
```

อธิบายการทำงานและเขียนผลลัพธ์โปรแกรม

```
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
```

การกำหนดค่าและการอ่านค่าตัวแปรประเภทตัวชี้

การกำหนดค่าให้กับตัวแปรพอยน์เตอร์จะเป็นการกำหนดแอดเดรสของตัวแปรที่มีประเภทสอดคล้องกับประเภทของตัวแปรพอยน์เตอร์เท่านั้น โดยการใช้ Unary Operator & เป็นโอเปอเรเตอร์ที่อ้างถึงแอดเดรสของออบเจ็ค (Object) ใด ๆ

ตัวอย่างการกำหนดค่าและอ่านค่าตัวแปรประเภทตัวชี้

```
int x = 1, y = 2;
int *ip, *iq;
ip = &x;
y = *ip;
*ip = 0;
y = 5;
ip = &y;
*ip = 3;
iq = ip;
```

ตัวชี้และอาร์กิวเมนต์ของฟังก์ชัน (Pointer and Function Arguments)

เนื่องจากภาษาซีมีการส่งอาร์กิวเมนต์ให้กับฟังก์ชันแบบ By Value และฟังก์ชันสามารถคืนค่า (return) ค่าได้เพียงหนึ่งค่า หากต้องการให้ฟังก์ชันมีการเปลี่ยนแปลงค่าและคืนค่ากลับมายังฟังก์ชันที่เรียกใช้มากกว่าหนึ่งค่าจะต้องนำพอยน์เตอร์เข้ามาช่วย

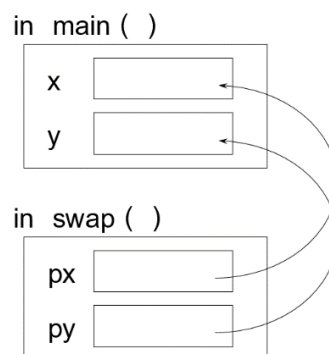
ตัวอย่างเช่น หากต้องการเขียนฟังก์ชันเพื่อสลับค่าของตัวแปร 2 ตัว ผลลัพธ์ที่ต้องการได้จากฟังก์ชันนี้จะมี 2 ค่าของตัวแปรที่ทำการสลับค่า หากอาร์กิวเมนต์เป็นตัวแปรธรรมดาจะไม่สามารถแก้ปัญหานี้ได้ จึงต้องใช้พอยน์เตอร์เข้ามาช่วย โดยการส่งค่าแอดเดรสของตัวแปรทั้ง 2 ให้กับฟังก์ชันที่จะสลับค่าของตัวแปรทั้ง 2 ผ่านทางตัวแปรพอยน์เตอร์ที่เป็นอาร์กิวเมนต์ของฟังก์ชัน

โปรแกรมตัวอย่างการสลับค่าตัวแปร 2 ตัวโดยผ่านฟังก์ชัน จะแสดงการส่งอาร์กิวเมนต์ในเป็นพอยน์เตอร์

```
#include <stdio.h>
void swap (int *, int *);
void main ( )
{
    int x = 5, y = 10;
    printf("Before swap : x = %d", x, ", y = %d\n", y);
    swap ( &x, &y);
    printf("After swap : x = %d", x, ", y = %d\n", y);
}
void swap (int *px, int *py)
{
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}
```

อาร์กิวเมนต์ที่เป็นประเภทพอยน์เตอร์จะช่วยให้ฟังก์ชันสามารถเปลี่ยนค่าให้กับตัวแปรที่ส่งเข้ามาได้ เนื่องจากอาร์กิวเมนต์นั้นจะเก็บแอดเดรสของตัวแปรที่ส่งเข้ามา เมื่อมีการเปลี่ยนแปลงค่าของอาร์กิวเมนต์ผ่าน Dereferencing Operator (*) ค่าของตัวแปรที่ส่งเข้ามาจะถูกเปลี่ยนค่าพร้อมกันในที่

รูปแสดงความสัมพันธ์ของการส่งอาร์กิวเมนต์แบบพอยน์เตอร์กับฟังก์ชัน



การใช้ตัวชี้กับอาร์เรย์

การทำงานใด ๆ ของอาร์เรย์สามารถใช้พอยน์เตอร์เข้ามาช่วย ซึ่งจะช่วยให้มีความเร็วในการทำงานสูงขึ้น สมมติว่ามีอาร์เรย์ `a` และพอยน์เตอร์ `pa` ดังนี้

```
int a[10];
```

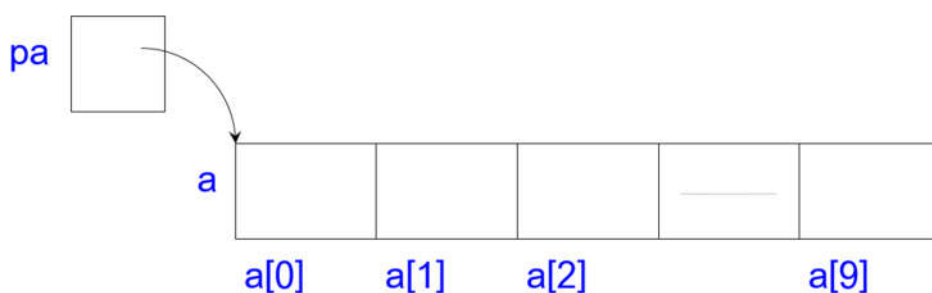
```
int *pa;
```

กำหนดให้พอยน์เตอร์ `pa` ชี้ไปยังอาร์เรย์ `a` ด้วยคำสั่ง

```
pa = &a[0]; /* หรือใช้คำสั่ง pa = a; */
```

`pa` จะเก็บค่าแอดเดรสเริ่มต้นของอาร์เรย์ `a`

รูป แสดงตัวชี้ชี้ไปยังแอดเดรสเริ่มต้นของอาร์เรย์

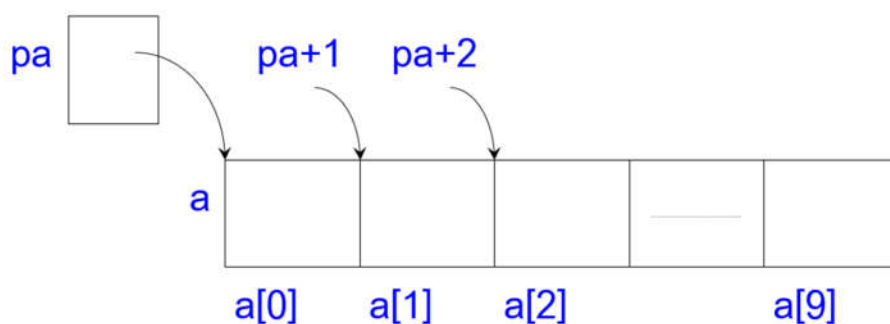


การนำไปใช้งานจะสามารถอ่านค่าอาร์เรย์ผ่านพอยน์เตอร์ได้ดังนี้

```
x = *pa;
```

จะเป็นการกำหนดค่าให้ `x` มีค่าเท่ากับ `a[0]` การเลื่อนไปอ่านค่าสมาชิกตำแหน่งต่าง ๆ ของอาร์เรย์ผ่านทางพอยน์เตอร์สามารถทำได้โดยการเพิ่มค่าพอยน์เตอร์ขึ้น 1 เพื่อเลื่อนไปยังตำแหน่งถัดไป หรือเพิ่มค่าขึ้น `N` เพื่อเลื่อนไป `N` ตำแหน่ง หรืออาจจะลดค่าเพื่อเลื่อนตำแหน่งลง

รูปแสดงการอ้างอิงตำแหน่งในอาร์เรย์ผ่านตัวชี้



กรณีที่ `pa` ชี้อยู่ที่ `a[0]` คำสั่ง

```
pa+1;
```

จะเป็นการอ้างอิงแอดเดรสของ `a[1]` หากเป็น `pa+i` เป็นการอ้างอิงแอดเดรส `a[i]` หากต้องการอ้างอิงข้อมูลภายในของสมาชิกของอาร์เรย์ตำแหน่งที่ `a[i]` จะใช้ `*(pa+i)`

การสั่งให้บวก 1 หรือบวก i หรือ ลบ i เป็นเหมือนการเลื่อนไปยังสมาชิกของอาร์เรย์ตำแหน่งที่ต้องการ เนื่องจากประเภทของข้อมูลแต่ละประเภทของอาร์เรย์ เช่น int, float, double และอื่น ๆ มีขนาดของข้อมูลที่แตกต่างกัน ทำให้ขนาดของสมาชิกภายในอาร์เรย์แต่ละประเภทมีขนาดแตกต่างกันด้วย การสั่งให้บวกหรือลบด้วยจำนวนที่ต้องการนั้นจะมีกลไกที่ทำหน้าที่คำนวณตำแหน่งที่ต้องการให้สอดคล้อง กับข้อมูลแต่ละประเภทโดยอัตโนมัติ

นอกจากนี้ยังสามารถใช้พอยน์เตอร์แทนอาร์เรย์ การอ้างโดยใช้ a[i] สามารถใช้ *(a+i) เนื่องจากทุกครั้งที่อ้างถึง a[i] ภาษาซีจะทำหน้าที่แปลงเป็น *(a+i) เพราะฉะนั้นการเขียนในรูปแบบใดก็ได้ผลลัพธ์ในการทำงาน เช่นเดียวกัน และการอ้างถึงแอดเดรส เช่น &a[i] จะมีผลเท่ากับการใช้ a+i

ในลักษณะเดียวกันการใช้งานพอยน์เตอร์ก็สามารถใช้คำสั่งในลักษณะอาร์เรย์ก็ได้ เช่น การอ้างถึง *(pa+i) สามารถเขียนด้วย pa[i] ก็ได้ผลเช่นเดียวกัน

สิ่งที่แตกต่างกันของอาร์เรย์และพอยน์เตอร์ คือ พอยน์เตอร์เป็นตัวแปร แต่อาร์เรย์ไม่ใช่ตัวแปร สมมติให้ a เป็นอาร์เรย์และ pa เป็นพอยน์เตอร์ การอ้างถึง pa = a หรือ pa++ จะสามารถคอมไพล์ได้ แต่จะไม่สามารถใช้คำสั่ง a = pa หรือ a++ ได้

เมื่อมีการส่งชื่อของอาร์เรย์ให้แก่ฟังก์ชัน จะเป็นการส่งตำแหน่งแอดเดรสของสมาชิกตัวแรกของอาร์เรย์ให้แก่ฟังก์ชัน ดังนั้นพารามิเตอร์ในฟังก์ชันนั้นจะเป็นตัวแปรประเภทพอยน์เตอร์

ตัวอย่าง ฟังก์ชันที่รับพารามิเตอร์เป็นพอยน์เตอร์ โดยอาร์กิวเมนต์ที่ส่งมาเป็นอาร์เรย์

```
int strlen (char *s)
{
    int n;
    for ( n = 0; *s != '\0'; s++ )
        n++;
    return n;
}
```

จะเห็นว่า s เป็นพอยน์เตอร์ ในฟังก์ชันจะมีการตรวจสอบข้อมูลว่ามีค่าเท่ากับ '\0' หรือไม่ และมีการเลื่อนตำแหน่งทีละ 1 ค่า (นับว่าข้อมูลมีความยาวเพิ่มขึ้นทีละ 1) โดยใช้ s++ การเรียกใช้ฟังก์ชัน strlen สามารถทำได้หลายลักษณะ

```
strlen ("hello world");    /* string constant */
strlen (array);            /* char array[10] */
strlen (ptr);              /* char *ptr;    */
```

นอกจากนี้ยังสามารถประกาศพารามิเตอร์ภายในฟังก์ชัน strlen ได้ใน 2 ลักษณะ คือ char *s แบบในตัวอย่าง หรืออาจจะใช้ char s[] ก็ได้ โดยทั่วไปจะใช้ในลักษณะแรก เพราะช่วยในรู้ได้ทันทีว่า s เป็นตัวแปรพอยน์เตอร์ และยังสามารถส่งส่วนใดส่วนของอาร์เรย์ให้แก่ฟังก์ชันก็ได้ โดยไม่จำเป็นต้องส่งสมาชิกตัวแรกก็ได้เช่นกัน

การคำนวณกับแอดเดรส

ให้ p เป็นพอยน์เตอร์ชี้ไปยังอาร์เรย์ใด ๆ คำสั่ง p++ เป็นการเลื่อน p ไปยังสมาชิกถัดไป และคำสั่ง p += i เป็นการเลื่อนพอยน์เตอร์ไป i ตำแหน่งจากตำแหน่งปัจจุบัน นอกจากนี้ยังสามารถใช้เครื่องหมายความสัมพันธ์ (Relational Operator) เช่น ==, !=, <, >= และอื่น ๆ ทำงานร่วมกับพอยน์เตอร์ได้ สมมติให้ p และ q ชี้ไปยังสมาชิกของอาร์เรย์เดียวกัน

$$p < q$$

จะเป็นจริงเมื่อ p ชี้ไปที่สมาชิกที่อยู่ก่อนหน้าสมาชิกที่ q ชี้อยู่ การเปรียบเทียบในลักษณะจะใช้ได้ต่อเมื่อ p และ q ชี้ไปที่อาร์เรย์เดียวกันเท่านั้น

นอกจากนี้ยังสามารถใช้การลบหรือการบวกกับพอยน์เตอร์ได้เช่นเดียวกัน แต่สิ่งที่ควรระวังคือ การทำเช่นนั้นจะต้องอยู่ในขอบเขตขนาดของอาร์เรย์เท่านั้น

ตัวอย่างฟังก์ชัน strlen () ปรับปรุงให้กระชับขึ้น

```
int strlen (char *s)
{
    char *p = s;
    while (*p != '\0')
        p++;
    return p-s;
}
```

เนื่องจาก s ชี้อยู่ที่ตำแหน่งเริ่มต้น โดยมี p ชี้ไปที่ s เช่นเดียวกัน แต่จะมีการเลื่อน p ไปทีละหนึ่งตำแหน่ง จนกว่าค่าที่ตำแหน่งที่ p ชี้อยู่จะเท่ากับ '\0' เมื่อนำ p ค่าสุดท้ายมาลบกับ s ที่ตำแหน่งเริ่มต้นก็จะได้ความยาวของข้อมูลที่ส่งเข้ามา

ตัวชี้ตัวอักษรและฟังก์ชัน (Character Pointer and Function)

การทำงานกับข้อความหรือที่เรียกว่า สตริง (String) เป็นการใช้ข้อมูลตัวอักษรหลาย ๆ ตัว หรืออาร์เรย์ของข้อมูลประเภท char หรืออาจจะใช้พอยน์เตอร์ชี้ไปยังข้อมูลประเภท char การทำงานกับค่าคงที่สตริง (String Constant) สามารถเขียนภายในเครื่องหมาย “ ”

“I am a string”

เมื่อมีการใช้ค่าคงที่สตริงจะมีการพื้นที่ในหน่วยความจำเท่ากับความยาวของค่าคงที่สตริงบวกด้วย 1 เนื่องจากลักษณะการเก็บข้อมูลประเภทข้อความในหน่วยความจำจะมีการปะตัวอักษร null หรือ '\0' ต่อท้ายเสมอเพื่อให้รู้ว่าเป็นจุดสิ้นสุดของข้อมูล การจองพื้นที่ดังกล่าวจะเหมือนการจองพื้นที่ของข้อมูลประเภทอาร์เรย์ เป็นอาร์เรย์ของ char ค่าคงที่สตริงที่พบเห็นได้เสมอได้แก่ข้อความที่ใช้ในฟังก์ชัน printf () เช่น

```
printf ( “Hello, world\n” );
```

ฟังก์ชัน `printf ( )` จะรับพารามิเตอร์เป็นพอยน์เตอร์ชี้ไปยังแอดเดรสของข้อมูลที่ตำแหน่งเริ่มต้นของอาร์เรย์ และนำข้อความนั้นแสดงออกทางอุปกรณ์แสดงข้อมูลมาตรฐาน

ในการเขียนโปรแกรมจะสามารถใช้พอยน์เตอร์ชี้ไปค่าคงที่สตริงใด ๆ ก็ได้ เช่น

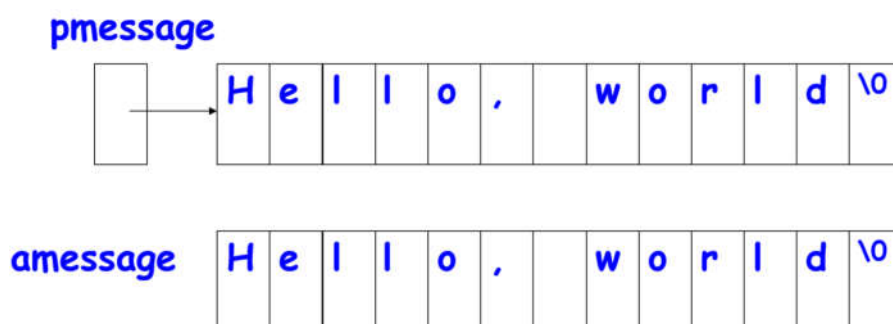
```
char *pmessage = "Hello, world";
```

`pmessage` จะเป็นพอยน์เตอร์ประเภท `char` ชี้ไปที่อาร์เรย์ของตัวอักษร จะแตกต่างจากการใช้อาร์เรย์ทั่วไป เช่น

```
char amessage[ ] = "Hello, world";
```

ลักษณะของอาร์เรย์เช่น `amessage` จะมีการจองพื้นที่ใช้กับอาร์เรย์ขนาด 13 ตัวอักษรรวมทั้ง null ส่วนลักษณะของพอยน์เตอร์ที่ชี้ไปยังค่าคงที่สตริง จะมีการจองพื้นที่ให้กับค่าคงที่สตริงขนาด 13 ตัวอักษรเช่นเดียวกัน แต่จะมีการจองพื้นที่ให้กับพอยน์เตอร์และทำการชี้พอยน์เตอร์นั้นไปยังพื้นที่ของค่าคงที่สตริงที่จองเอาไว้

รูปแบบ การจองพื้นที่ให้กับอาร์เรย์และตัวชี้ชี้ไปยังค่าคงที่สตริง



ตัวอย่าง ฟังก์ชัน `strcpy ( )` ทำหน้าที่สำเนาข้อความจากตัวแปรหนึ่งไปยังอีกตัวแปรหนึ่งเขียนในลักษณะอาร์เรย์

```
void strcpy ( char *s, char *t )
{
    int i=0;
    while ( ( s[i] = t[i] ) != '\0' )
        i++;
}
```

ตัวอย่าง ฟังก์ชัน `strcpy ( )` เขียนในลักษณะพอยน์เตอร์

```
void strcpy ( char *s, char *t )
{
    while ( ( *s = *t ) != '\0' ) {
        s++;
        t++;
    }
}
```

ตัวอย่าง ฟังก์ชัน strcpy () เขียนในลักษณะพอยน์เตอร์แบบสั้น

```
void strcpy ( char *s, char *t )
{
    while ( ( *s++ = *t++ ) != '\0' ) ;
}
```

โจทย์ตัวแปรตัวชี้

โจทย์ pointer : 1 รับตัวเลข 10 ตัวลงอาร์เรย์ และแสดงผลรวม

โจทย์ pointer : 2 รับข้อความ และให้คัดลอกข้อความจากตัวแปร ไปไว้อีกตัวแปร พร้อมแสดงผล

โจทย์ pointer : 3 รับข้อความ 2 ข้อความ และให้ตอบว่าเป็น anagram กันหรือไม่ (YES/NO)

โจทย์ pointer : 4 รับข้อความ (a-z , A-Z) และเปลี่ยนตัวอักษรให้เป็นพิมพ์ใหญ่

โจทย์ pointer : 5 รับข้อความ และให้ตรวจสอบว่าเป็น parindrome หรือไม่ (YES/NO)

โจทย์ pointer6 : รับค่าให้ 2 อาร์เรย์ ขนาด 5 ช่อง นำอาร์เรย์มาบวกกัน และแสดงผลแต่ละช่องของอาร์เรย์ ทั้ง 2 เป็นตัวเลข 1 ตำแหน่ง

โจทย์ pointer7 : รับข้อความแล้วตรวจสอบข้อความ (ตัวพิมพ์เล็ก) ว่ามีสระอยู่ทั้งหมดกี่ตัว

โจทย์ pointer8 : กำหนดค่าให้ array ขนาด 10 ตัวเลข และทำการเรียงลำดับจากน้อยไปหามาก

โจทย์ pointer9 : รับค่าตัวเลข และให้นับตำแหน่งที่มีเลขคู่ ว่ามีทั้งหมดกี่ตัว

โจทย์ pointer10 : รับข้อความ รับค่าค้นหา ค้นหาข้อความ ถ้ามีให้บอกตำแหน่งตัวอักษรแรก ถ้าไม่มีให้แสดง 0

ตัวแปรโครงสร้าง (Struct)

ตัวแปรโครงสร้าง เป็นการสร้างกลุ่มของประเภทข้อมูลที่เหมือนกัน หรือต่างกัน เข้าไว้ด้วยกันเป็นประเภทข้อมูลใหม่ที่เราเรียกว่า ตัวแปรโครงสร้าง หรือ ที่เรียกกันว่าเป็นการเก็บข้อมูลแบบเรคคอร์ด ที่ภายในเรคคอร์ด จะมีหลายๆ ข้อมูล เช่น รหัสหนังสือ ชื่อหนังสือ ชื่อผู้แต่ง ราคา เป็นต้น เพื่อสะดวกในการเรียกใช้งานในด้านต่างๆ

```
รูปแบบ struct name_struct {
    member1;
    member2;
    ...
    member...;
} variable_struct ;
```

name_struct คือ ชื่อโครงสร้าง
 struct name_struct คือ ชื่อประเภทข้อมูลชนิดใหม่
 member คือ ประเภทข้อมูล ที่เหมือนกัน หรือ ต่างกันก็ได้
 สามารถมีได้หลาย member ใน struct เดียวกัน
 variable_struct คือ ตัวแปรโครงสร้าง

ตัวอย่างที่ 1

```
struct book {
    int b_id;
    char b_name[60];
    char b_author[50];
    float b_price;
} a , bookrecord[5];

struct book b;
```

book คือ ชื่อโครงสร้าง
 struct book คือ ชื่อประเภทข้อมูลชนิดใหม่
 a คือตัวแปรโครงสร้างแบบ เรคคอร์ด 1, bookrecord คือ
 ตัวแปรโครงสร้างแบบอาร์เรย์
 b คือตัวแปรโครงสร้างแบบ เรคคอร์ด 1

รหัสหนังสือ	ชื่อหนังสือ	ชื่อผู้แต่ง	ราคา
b_id	b_name	b_author	b_price

วิธีนำข้อมูลลงตัวแปรโครงสร้าง

แบบที่ 1 a.b_id = 1001; หรือ scanf("%d",&a.b_id);
 scanf("%s",a.b_name);
 scanf("%s",a.b_author);
 a.b_price = 159.50; หรือ scanf("%f",&a.b_price);

แบบที่ 2 bookrecord[0].b_id = 2001; หรือ scanf("%d",&bookrecord[0].b_id);
 scanf("%s", bookrecord[0].b_name);
 scanf("%s", bookrecord[0].b_author);
 bookrecord[0].b_price = 499.25; หรือ scanf("%f",&bookrecord[0].b_price);

การแสดงผลข้อมูลแบบโครงสร้าง

```
printf("book id : %d %s %s %0.2f\n",a.b_id,a.b_name,a.b_author,a.b_price);
```

```
book id : 1001 Harry Potter J. K. rowing 1256.70
```

```
printf("book name %s price %0.2f\n",a.b_name,a.b_price);
```

```
book name Harry Potter price 1256.70
```

```
printf("book name %s\n",bookrecord[0].b_name);
```

```
book name Harry Potter
```

```
printf("book id %d name %s\n",bookrecord[0].b_id,bookrecord[0].b_name);
```

```
book id : 1001 name Harry Potter
```

```
struct book {
```

```
    int b_id;
```

```
    char b_name[60];
```

```
    char b_author[50];
```

```
    float b_price;
```

```
    } bookrecord[5]={ {1001,"Harry Potter","J. K. rowing ",1256.70},
                      {1002,"Unfortunate Events","Lemony Snicket ",1344.47},
                      {1003,"JAVA language","Teerawat Prakobpol",189.05},
                      {1004,"Microprocessor","Teerawat Prakobpol",342.00},
                      {1005,"Basic C language","Teerawat Prakobpol",225.05}
    };
```

```
printf("id   name \t\t author \t\t price\n");
```

```
for(i=0 ; i<5 ;i++)
```

```
{ printf("%d ",bookrecord[i].b_id);
```

```
  printf("%-20s",bookrecord[i].b_name);
```

```
  printf("%-20s",bookrecord[i].b_author);
```

```
  printf("%10.2f\n",bookrecord[i].b_price);
```

```
}
```

/t คือ แทนการกดปุ่ม tab

%-20s คือ การจอง ตำแหน่ง แล้วพิมพ์ชิดซ้าย 20

ถ้าไม่มี จะชิดขวา -

id	name	author	price
1001	Harry Potter	J. K. rowing	1256.70
1002	Unfortunate Events	Lemony Snicket	1344.47
1003	JAVA language	Teerawat Prakobpol	189.05
1004	Microprocessor	Teerawat Prakobpol	342.00
1005	Basic C language	Teerawat Prakobpol	225.05

ตัวอย่างที่ 2

```

struct date {
    int day;
    int month;
    int year;
};

struct student {
    int std_id;
    char std_name[40];
    struct date std_birth;
    struct date std_checkin;
    int class;
};

struct student s;
```

struct date , struct student คือ ชื่อประเภทข้อมูลชนิดใหม่
ใน struct student จะมี struct date อยู่ในโครงสร้างด้วย
เป็นโครงสร้างที่ซ้อนโครงสร้างอีกที

struct student คือ ประเภทข้อมูล เช่นเดียวกับ int char float

student								
รหัสนักเรียน	ชื่อนักเรียน	วันเกิด / date			วันที่เข้าศึกษา / date			ระดับชั้น
		วัน	เดือน	ปี	วัน	เดือน	ปี	

การใช้ typedef เพื่อนิยามประเภทข้อมูลใหม่

รูปแบบ typedef ประเภทข้อมูลที่ต้องการนิยาม ชื่อประเภทข้อมูลใหม่;

แบบเดิม struct student s;

แบบนิยาม typedef struct student STD;

STD คือ ประเภทข้อมูล เช่นเดียวกับ struct student

```

STD s , stdrecord[5];          // struct student นิยามให้เป็นประเภทข้อมูลใหม่ ว่า STD
scanf("%d",&s.std_id);
scanf("%s",s.std_name);
scanf("%d",&s.std_birth.day);
scanf("%d",&s.std_birth.month);
scanf("%d",&s.std_birth.year);
scanf("%d",&s.std_checkin.day);
scanf("%d",&s.std_checkin.month);
scanf("%d",&s.std_checkin.year);
scanf("%d",&s.std_class);

```

จากตัวอย่างที่ 2 ให้เขียนโปรแกรมเพิ่มเติมข้อมูลนักเรียนเริ่มต้นต่อไปนี้

นำข้อมูลลง stdrecord 5 ชุดดังนี้

10001Chris Hemsworth 11 12 2552 4 2550 5 3

10002 Tom Cruise 3 4 2552 6 5 2551 4

10003Taylor Swift 2552 12 13 1 5 2549 1

10004Hugh Grant 9 9 2551 1 5 2550 1

10005Kristen Stewart 1 2549 5 7 2552 4 9

โจทย์โครงสร้าง

- .1เขียนโปรแกรมเพื่อค้นหานักเรียนที่มีอายุน้อยที่สุด
- .2เขียนโปรแกรมเพื่อค้นหานักเรียนที่เริ่มเข้าศึกษาเป็นลำดับสุดท้าย
- .3เขียนโปรแกรมเพื่อค้นหาข้อมูลนักเรียนทั้งหมด จาก รหัสนักเรียน
- .4เขียนโปรแกรมเพื่อค้นหาข้อมูลนักเรียนทั้งหมด จาก เดือนเกิด (พิมพ์ออกมาทุกคนที่เกิดเดือนเดียวกัน)
- .5เขียนโปรแกรมเพื่อให้แสดงข้อมูลนักเรียนทุกคน ดังนี้ รหัสนักเรียน ชื่อนักเรียน อายุ
ให้แสดง อายุ (2562 1 1 นับถึง) วัน เดือน ปี

หน่วยที่ 8 ฟังก์ชันและการประยุกต์

1. การสร้างฟังก์ชัน
2. การประกาศตัวแปร .
3. Library function
4. การประยุกต์ฟังก์ชัน
5. ฟังก์ชันเรียกซ้ำ (Recursive function)

ฟังก์ชันแรกในภาษาซี (main)

```
#include "stdio.h"
void main()
{
    int x,y,z;
    x = 100;
    y = 65;
    z = x + y;
    printf("%d",z);
}
```



Test.exe

เรียกใช้ฟังก์ชัน test()

```
#include "stdio.h"
void test()
{
    int x, y, z;
    x = 100;
    y = 65;
    z = x + y;
    printf("%d",z);
}
void main()
{
    test();
}
```

ฟังก์ชันในภาษาซี

- ฟังก์ชันมาตรฐาน (Standard Function)

เป็นฟังก์ชันที่อยู่ในโปรแกรม เก็บไว้ใน header file ผู้ใช้จะต้องรู้ว่าฟังก์ชันนั้น ๆ เก็บอยู่ในไฟล์ใด

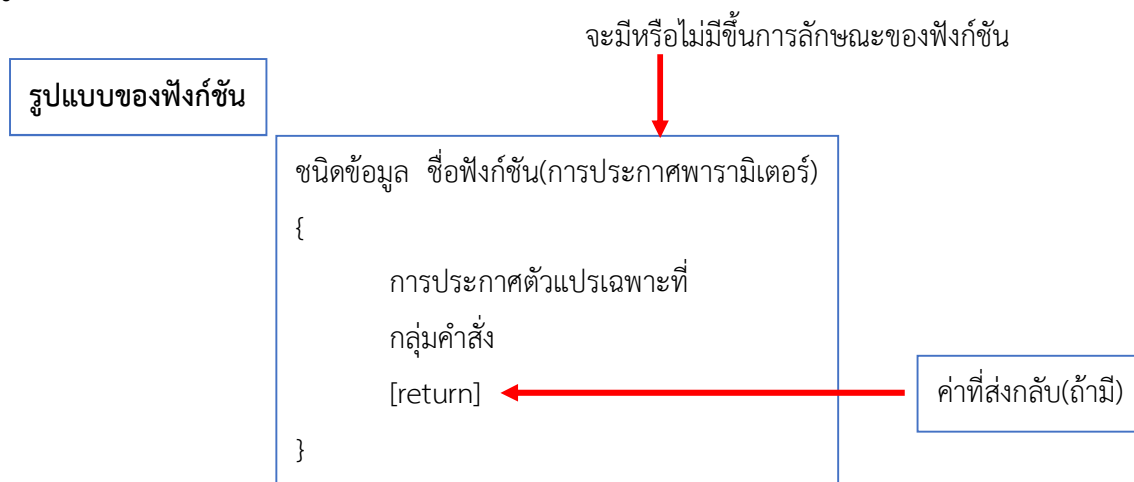
- ฟังก์ชันที่เขียนขึ้นเอง (user defined function)

เขียนขึ้นเพื่อให้ทำงานตามที่ต้องการ เช่น

```
void cal_sum()
{
    int x,sum = 0;
    for(x=1; x<=10;x++)
        sum = sum + x;
}
```

ฟังก์ชัน (function)

ฟังก์ชันเป็นกลุ่มคำสั่งที่ใช้ในการแบ่งโปรแกรมเป็นส่วนย่อย ๆ เพื่อทำงานอย่างใดอย่างหนึ่ง ฟังก์ชันนี้อาจมีการส่งข้อมูลไปและกลับระหว่างฟังก์ชันได้



ประเภทของฟังก์ชันที่สร้างขึ้นเอง

ถ้าหากแยกตามประเภทการส่งค่าไปกลับจะแยกได้ดังนี้

- ฟังก์ชันที่ไม่มีการส่งค่าไปและรับค่ากลับ
- ฟังก์ชันที่มีการส่งค่าไปแต่ไม่มีการรับค่ากลับ
- ฟังก์ชันที่มีการส่งค่าไปและรับค่ากลับ

เช่น ให้หาค่าที่เข้ามาทาง

เช่น สั่งให้พิมพ์ข้อความ

ตัวอย่างเช่นส่งค่าไปให้คำนวณ

ฟังก์ชันที่ไม่มีทั้งการส่งค่าไปและรับค่ากลับ

ฟังก์ชันประเภทนี้จะให้ชนิดของข้อมูลเป็น void และภายในวงเล็บใช้ void เพื่อบอกว่าไม่มีการส่งและรับค่า และจะไม่ใช่คำสั่ง return

รูปแบบของฟังก์ชัน

```

void funct_name(void)
{
    local variable declaration;
    statement();
}

```

ฟังก์ชันที่มีการส่งค่าไปแต่ไม่มีการรับกลับ

ฟังก์ชันประเภทนี้จะให้ชนิดของข้อมูลเป็น void ภายในวงเล็บจะมีอาร์กิวเมนต์ และจะไม่ใช้คำสั่ง return

รูปแบบของฟังก์ชัน

```
void funct_name(type1 arg1, type2 arg2,.....,typeN argN)
{
    local variable declaration;
    statement();
}
```

```
void ADD(int a, int b)
{
    int x;
    x = a+b;
    printf("sum = %d",x);
}
```

ฟังก์ชันที่มีการส่งค่าไปและมีการรับกลับ

ฟังก์ชันประเภทนี้จะให้ชนิดของข้อมูลเป็นประเภทที่จะส่งกลับ ภายในวงเล็บจะมีอาร์กิวเมนต์ และจะใช้คำสั่ง return ส่งค่ากลับมาให้ฟังก์ชัน

รูปแบบของฟังก์ชัน

```
int ADD(int a, int b)
{
    int x;
    x = a+b;
    return x;
}
```

ข้อมูล
ประเภทเดียวกัน

```
type funct_name(type1 arg1, type2 arg2,.....,typeN argN)
{
    local variable declaration;
    statement();
    return (value);
}
```

ตัวอย่างโปรแกรม สร้างฟังก์ชันมาสองฟังก์ชัน ไม่มีการส่งและรับค่า

<pre>#include <stdio.h> void one(void); void two(void); void main(void) { one(); two(); }</pre>	 มีการประกาศ Prototype	<pre>void one() { int a = 5, b = 7; printf("A = %d, B = %d\n",a,b); } void two() { float p = 4.5, q = 3.5; printf("P = %6.2f, Q = %6.2f\n",p,q); }</pre>
---	---	---

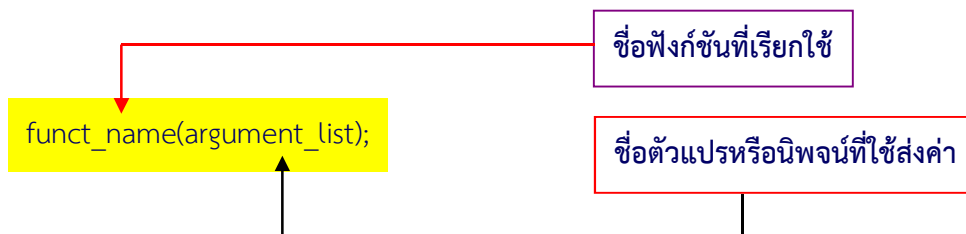
จะสังเกตเห็นว่าไม่มีพารามิเตอร์ ไม่มี return

ตัวอย่างโปรแกรม สร้างฟังก์ชันมาสองฟังก์ชัน ไม่มีการส่งและรับค่า

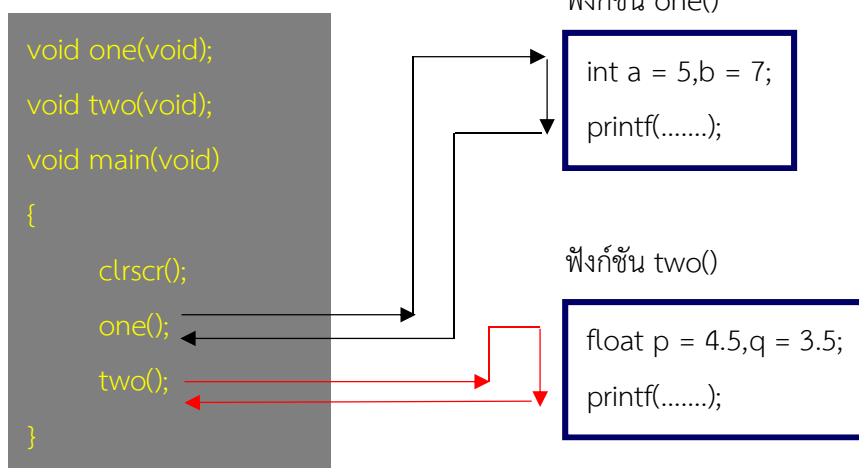
<pre>#include <stdio.h> void one() { int a = 5, b = 7; printf("A = %d, B = %d\n",a,b); } void two() { float p = 4.5, q = 3.5; printf("P = %6.2f, Q = %6.2f\n",p,q); }</pre>	อีกแบบ	<pre>void main(void) { one(); two(); }</pre>
---	--	---

ถ้าหากสร้างฟังก์ชันก่อน main จะไม่ประกาศ Prototype ก็ได้

การเรียกใช้ฟังก์ชัน รูปแบบ



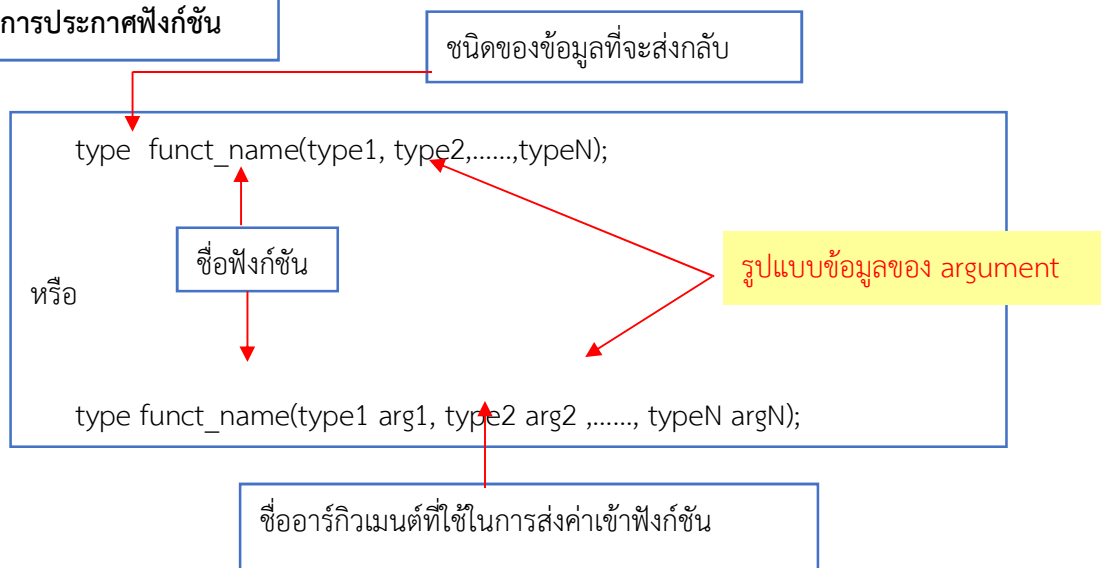
โปรแกรมหลัก



การประกาศรูปแบบฟังก์ชัน (function prototype)

คล้ายกับการประกาศตัวแปร เพื่อบอกว่าในโปรแกรมของเรา มีฟังก์ชันอะไรที่สร้างขึ้นเอง ฟังก์ชันนั้น ๆ มีการเรียกใช้อย่างไร ใช้ตัวแปรอย่างไร ตำแหน่งที่ประกาศ ควรอยู่ก่อน main()

รูปแบบการประกาศฟังก์ชัน



ตัวอย่างการประกาศรูปแบบฟังก์ชัน

ตัวอย่าง

1. float add(int, float);
2. int sum(int p, int q); หรือ int sum(int, int);
3. void move(int, float, int);
4. float calculate(float, float);
5. void point(int*, int*, float);

ฟังก์ชันนี้ใช้อาร์กิวเมนต์ที่เป็นตัวแปร
แบบพอยน์เตอร์

การประกาศตัวแปรของฟังก์ชัน

ฟังก์ชันนี้ไม่มีการคืนค่า จะรับอาร์กิวเมนต์เป็น int, float และ int เช่น move(5, 2.70, 12);

ตัวแปรแบบทั่วไป (global variables)

เป็นตัวแปรที่ทุกส่วนของโปรแกรมสามารถเรียกใช้ได้ บางครั้งจะเรียกว่าตัวแปรเอ็กเทอร์นัล (External variables)

ตัวแปรเฉพาะที่ (local variables)

เป็นตัวแปรที่สร้างขึ้นภายในฟังก์ชัน ใช้ในฟังก์ชันนั้น ๆ เมื่อออกจากฟังก์ชันค่าตัวแปรจะหายไป การใช้ตัวแปรประเภทนี้จะทำให้ฟังก์ชันต่าง ๆ สามารถใช้ชื่อตัวแปรชื่อเดียวกันได้ ตัวแปรประเภทนี้บางครั้งเรียกว่า ออโตเมติก (automatic variable)

ตัวอย่างการใช้ตัวแปรแบบทั่วไปร่วมกับฟังก์ชัน

```
#include <stdio.h>

int a;

void Ex()
{
    a = 5;
    printf("%d\n",a);
}
```

```
main()
{
    a = 3;
    printf("%d\n",a);
    Ex();
    printf("%d\n",a);
    return 0;
}
```

3

5

5

ตัวอย่างการใช้ตัวแปรเฉพาะที่

```

#include <stdio.h>

int a;

void Ex()
{
    int a;

    a = 5;

    printf("%d\n",a);
}

main()
{
    a = 3;

    printf("%d\n",a);

    Ex();

    printf("%d\n",a);

    return 0;
}

```

3

5

3

ตัวอย่างฟังก์ชันที่คืนค่าเป็นเลขจำนวนเต็ม

```

#include <stdio.h>

int x,y;

int add_num(int a, int b)
{
    int m;

    m = a + b;

    return m;
}

void main()
{
    printf("INPUT  X ");
    scanf("%d",&x);
    printf("INPUT  Y ");
    scanf("%d",&y);
    printf("%d + %d =%d\n",x,y,add_num(x,y) );
}

```

ส่งตัวแปร x,y ไปให้ a กับ b

ตัวอย่างการฟังก์ชันที่ส่งผ่านตัวแปรหลายประเภท

```
#include <stdio.h>

void test_f(int a, int b, double f, char c, char s[20])
{
    printf("Integer %d, %d\n", a, b);
    printf("Float %f\n", f);
    printf("Char %c\n", c);
    printf("String %s\n", s);
}

void main()
{
    test_f(23, 34, 3.564, 'p', "Yellow River");
}
```

ผลการทำงาน

Integer 23, 34

Float 3.564000

Char p

String Yellow River

ฟังก์ชันมาตรฐาน

ฟังก์ชันทางคณิตศาสตร์ `#include <math.h>`

ฟังก์ชัน $\sin(x)$, $\cos(x)$, $\tan(x)$

หาค่าของฟังก์ชันโดยรับค่ามุมเป็นเรเดียน

ฟังก์ชัน $\sqrt{x}$

หาค่ารากที่สองของ x

ฟังก์ชัน $\exp(x)$

ฟังก์ชันหาค่า e^x โดย e มีค่าประมาณ 2.718282

ฟังก์ชัน $\text{pow}(x,y)$

ใช้หาค่า x^y โดย x เป็นค่าคงที่หรือตัวแปรที่มีค่ามากกว่า 0 y เป็นค่ายกกำลัง

ฟังก์ชัน $\log(x)$, $\log_{10}(x)$

หาค่า $\log$ ฐาน n และค่า $\log$ ฐาน 10

ต้องรู้ว่ารับข้อมูลเข้าเป็นอะไร
และคืนค่าประเภทใดออกมา

โปรแกรมหาค่าจากฟังก์ชันตรีโกณมิติ

```
#include "stdio.h"
#include "math.h"
main()
```

อาจใช้ M_PI

```
{
    double r, pi = 3.14159;
    r = pi/180;
    printf("%f\n",sin(r));
    printf("%f\n",cos(r));
    printf("%f\n",tan(r));
}
```

360 องศา = 2π
 x องศา = $(\pi/180) \cdot x$

ผลการรัน
 0.017452
 0.999848
 0.017455

ฟังก์ชันที่พบบ่อย

ฟังก์ชัน clrscr()

ใช้ลบจอภาพ

ฟังก์ชัน gotoxy(x,y)

กำหนดตำแหน่งของเคอร์เซอร์

ฟังก์ชัน abs(x)

หาค่าสัมบูรณ์ของ x

ฟังก์ชัน atoi(s)

เปลี่ยนค่าสตริง s ให้เป็นเลขจำนวนเต็ม

ฟังก์ชัน atof(s)

เปลี่ยนค่าสตริง s ให้เป็นเลขทศนิยม

ตัวอย่างการหาค่าของ $f(x) = x^2 + 3x + 1$

```
#include <stdio.h>

int f_x(int x);

main()
{
    int i;
    printf(" x      x + 3x + 1 \n");
    for(i = 1; i <= 10; i++)
        printf("%d      %d \n",i,f_x(i) );
    getch();
}
```

```
int f_x(int x)
{
    int y;

    y = x*x + 3*x + 1;

    return y;
}
```

ตัวอย่าง แสดงการทำงานของโปรแกรมการบวกเลขจำนวนจริง จำนวนที่รับจากผู้ใ้ 2

```
#include <stdio.h>

double InputDouble ( ) {
    double x;
    printf ( "\nInput real value : " );
    scanf ( "%2f ", &x );
    return ( x );
}

double SumDouble ( double x, double y ) {
    return ( x + y );
}

void PrintOut ( double x ) {
    printf ( "\n Result of sum is : %2f", x );
}

void main ( ) {
    double a1, a2, sumVal;
    a1 = InputDouble();
    a2 = InputDouble();
    sumVal = SumDouble ( a1, a2 );
    PrintOut ( sumVal );
}
```

ตัวอย่าง แสดงการทำงานของโปรแกรมการบวกเลขจำนวนจริง จำนวนที่รับจากผู้ใช้ ในลักษณะที่มีการ 2 ประกาศโปรโตไทป์

```
#include <stdio.h>

double InputDouble ( );
double SumDouble ( double , double );
void PrintOut ( double );
void main ( )
{
    double a1, a2, sumVal;
    a1 = InputDouble( );
    a2 = InputDouble( );
    sumVal = SumDouble ( a1, a2 );
    PrintOut ( sumVal );
}
```

การเรียกใช้ฟังก์ชัน 4.3

การเรียกใช้ฟังก์ชันที่มีการคืนค่า จะใช้รูปแบบดังต่อไปนี้

ค่าที่รับ = ฟังก์ชัน (อาร์กิวเมนต์)

ตัวอย่าง

a1 ต้องมีชนิดเป็น double เนื่องจากค่าที่จะส่งคืนกลับมาจากฟังก์ชันมีชนิดเป็น double

a1 = InputDouble ();

ใช้คู่กับโปรโตไทป์

double InputDouble ();

ตัวอย่าง

a1 และ a2 ต้องมีชนิดเป็น double เพื่อให้ตรงกับชนิดตัวแปรของอาร์กิวเมนต์ที่ประกาศในโปรโต

sumVal = SumDouble (a1,a2);

ใช้คู่กับโปรโตไทป์

double InputDouble ();

ตัวอย่าง

PrintOut(sumVal);

ใช้คู่กับโปรโตไทป์

void PrintOut (double);

ประกาศให้รู้ว่าฟังก์ชันนี้ไม่มีการคืนค่า

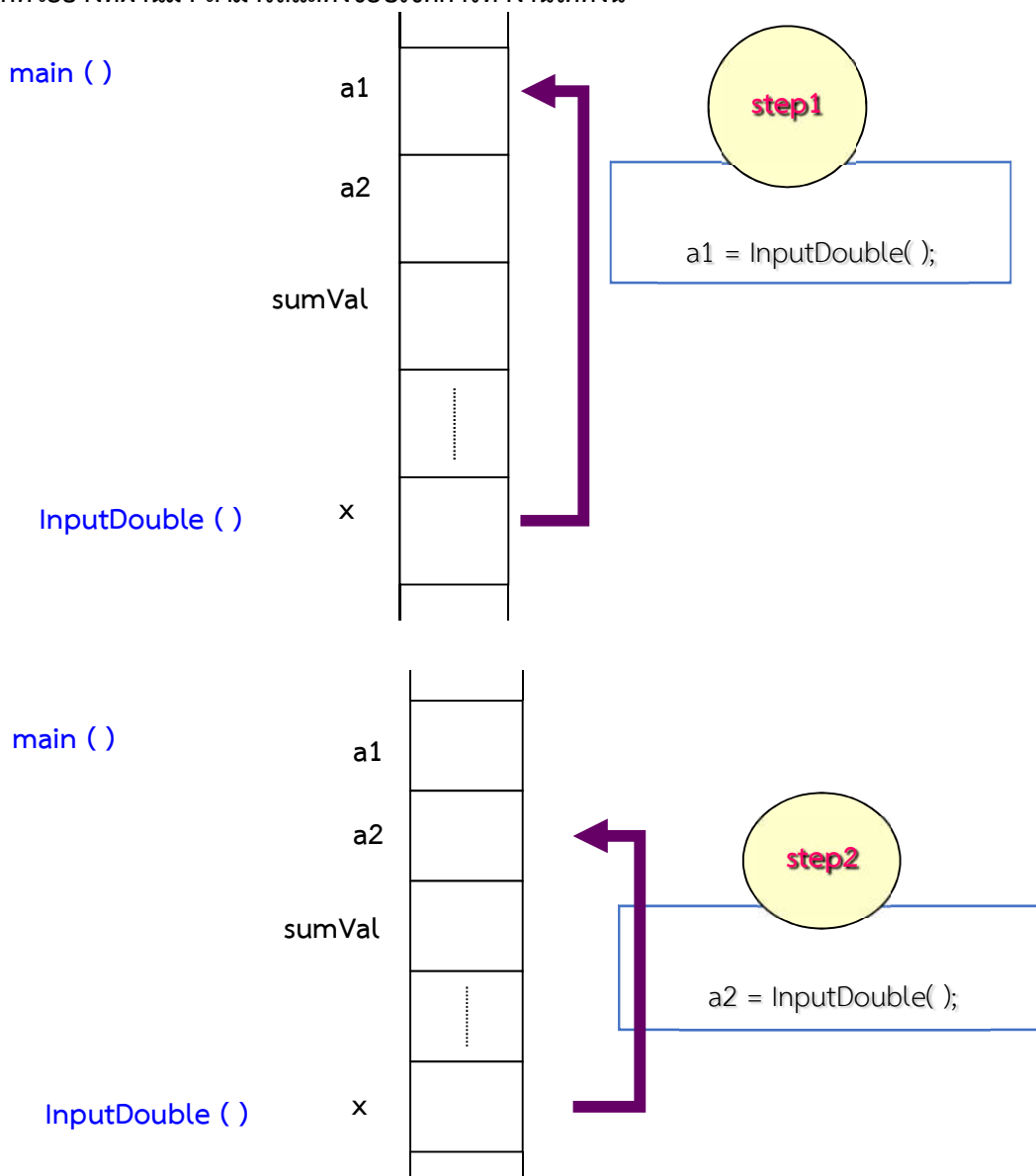
ขอบเขต (Scope)

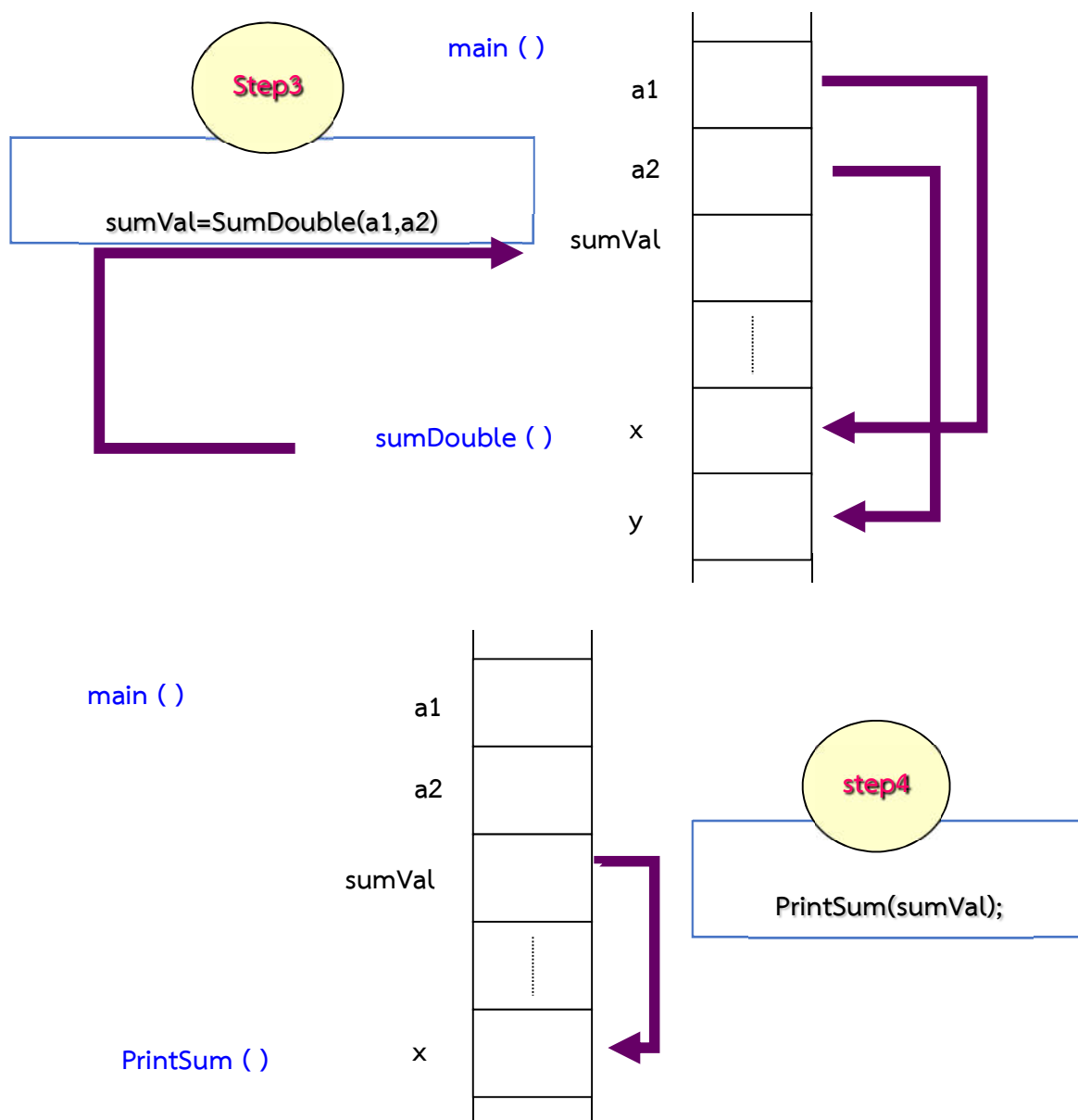
การทำงานของโปรแกรมภาษาซีจะทำงานที่ฟังก์ชัน `main ( )` ก่อนเสมอ เมื่อฟังก์ชัน `main ( )` เรียกใช้งานฟังก์ชันอื่น ก็จะมีการส่งคอนโทรล (Control) ที่ควบคุมการทำงานไปยังฟังก์ชันนั้น ๆ จนกว่าจะจบฟังก์ชันหรือพบคำสั่ง `return`

เมื่อมีการเรียกใช้งานฟังก์ชันจะมีการจองพื้นที่หน่วยความจำสำหรับตัวแปรที่ต้องใช้ภายในฟังก์ชันนั้น และเมื่อสิ้นสุดการทำงานของฟังก์ชันก็จะมีการคืนพื้นที่หน่วยความจำส่วนนั้นกลับสู่ระบบ การใช้งานตัวแปรแต่ละตัวจะมีขอบเขตของการใช้งานขึ้นอยู่กับตำแหน่งที่ประกาศตัวแปรนั้น

ตัวอย่าง

จากตัวอย่างที่ผ่านมา สามารถแสดงขอบเขตการทำงานได้ดังนี้





จะเห็นว่าตัวแปร `x` ที่ประกาศในแต่ละขั้นตอนจะทำงานอยู่ภายในฟังก์ชันที่มีการประกาศค่าเท่านั้น และใช้พื้นที่ในการเก็บข้อมูลคนละส่วนกัน

ขอบเขตการทำงานของตัวแปรแต่ละตัวจะกำหนดอยู่ภายบล็อกของคำสั่งภายในเครื่องหมายปีกกา (`{ }`) หรือการประกาศในช่วงของการประกาศฟังก์ชัน เรียกตัวแปรเหล่านี้ว่า ตัวแปรโลคอล (Local Variable)

นอกจากนี้สามารถประกาศตัวแปรไว้ที่ภายนอกฟังก์ชันบริเวณส่วนเริ่มของโปรแกรมจะเรียกว่า ตัวแปรโกลบอล (Global Variable) ซึ่งเป็นตัวแปรที่สามารถเรียกใช้ที่ตำแหน่งใด ๆ ในโปรแกรมก็ได้ ยกเว้นในกรณีที่มีการประกาศตัวแปรที่มีชื่อเดียวกันตัวแปรโกลบอลภายในบล็อกหรือฟังก์ชัน

ลองเขียนโปรแกรมวนลูปรับตัวเลข แล้วคืนค่าเป็นตัวเลข

```
char ch;
```

```
ch = getch() - '0';
```

ค่าคงที่ของฟังก์ชันทางคณิตศาสตร์

สัญลักษณ์	ความหมาย
M_E	e
M_LOG2E	$\log_2 e$
M_LOG10E	$\log_{10} e$
M_LN2	$\ln 2$
M_LN10	$\ln 10$
M_PI	pi

ฟังก์ชันเรียกซ้ำ (Recursive Function)

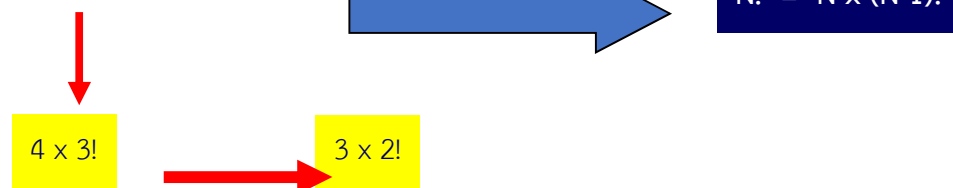
เป็นฟังก์ชันที่สร้างขึ้นเอง โดยภายในฟังก์ชันจะมีการเรียกชื่อของตัวเอง บางครั้งจะเรียก ว่าฟังก์ชันแบบเรียกตัวเอง โปรแกรมประเภทนี้จะต้องมีจุดที่ให้ออกจากฟังก์ชันได้

ตัวอย่าง

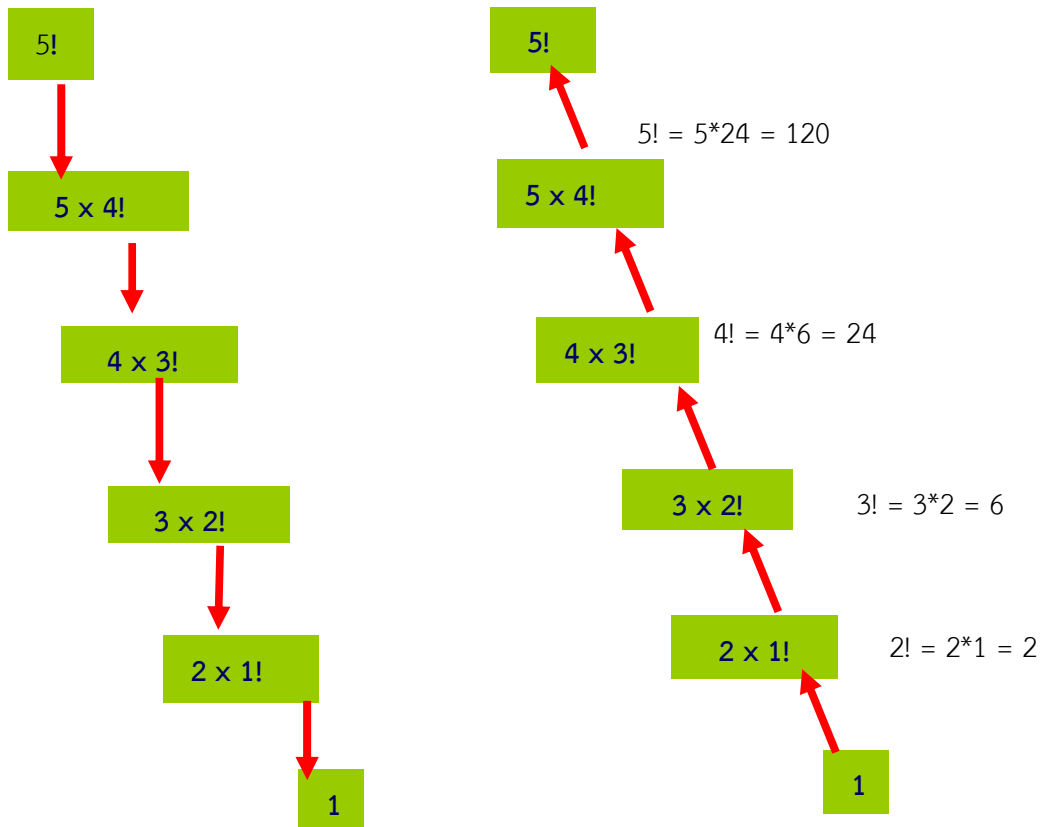
$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

สามารถมองได้เป็น

$$5! = 5 \times 4!$$



การคืนค่าของฟังก์ชัน



ตัวอย่างการฟังก์ชันหาค่า factorial

```
#include <stdio.h>

long factorial(long)

main()
{
    int i;

    for(i = 1 ; i<= 10; i++)

        printf("%2d! = %ld\n",i,factorial(i));
```

```
long factorial(long number)
{
    if (number <= 1)
        return 1;
    else
        return(number*factorial(number-1));
```

ฟังก์ชันสุ่มตัวเลข

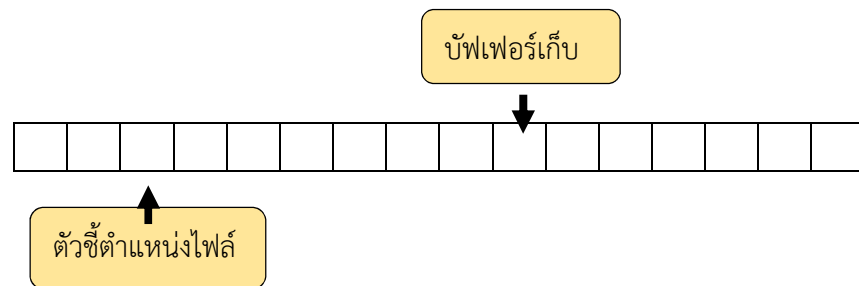
```
#include "stdlib.h"
#include "time.h"
main()
{
    srand(time(NULL));
    printf("%d\n",rand()%10);
    getch();
}
```

แบบฝึกหัดเพิ่มเติม

1. จงเขียนฟังก์ชันตรวจสอบว่ากต y/Y หรือ n/N หรือไม่
2. เขียนฟังก์ชันรับข้อมูล 10 ค่าแล้วหาผลรวม
3. เขียนโปรแกรมหาค่า sin, cos, tan โดยมีมุมเป็นองศา ตั้งแต่ 0 ถึง 90 โดยให้กระโดดครั้งละ 5
4. เขียนโปรแกรมหาค่า $f(x) = x^2 + 1$
5. เขียนโปรแกรมหาพื้นที่ใต้กราฟของ $y(x) = x^2 - 3x + 2$

หน่วยที่ 9 การติดต่อไฟล์

หลักการทำงานร่วมกับไฟล์



การประมวลผลไฟล์นั้น ภาษาซีจะไม่กระทำโดยตรง แต่จะกระทำผ่านบัฟเฟอร์ ซึ่งเป็นหน่วยความจำหลักที่ภาษาซีกันไว้สำหรับทำงานกับไฟล์

ประเภทของไฟล์

1. เทกซ์ไฟล์ (Text file) เป็นไฟล์ข้อความ เก็บเป็นรหัส ASCII สามารถอ่านขึ้นมาบนคอมพิวเตอร์ได้
2. ไบนารีไฟล์ (Binary file) เก็บในรูปแบบเลขฐานสอง

การประมวลผลไฟล์

การเปิดไฟล์

ในการทำงานร่วมกับไฟล์สิ่งแรกที่จะต้องทำคือต้องประกาศตัวแปรไฟล์พอยเตอร์จากนั้นใช้คำสั่งเปิดไฟล์โดยระบุว่าจะเปิดเพื่ออะไร โดยการใช้ฟังก์ชัน `fopen()` ในการเปิดไฟล์ โดยมีรูปแบบการใช้งานดังนี้

`fp = fopen("data.txt", "w");` เปิดไฟล์เพื่อเขียน

`fp = fopen("data.txt", "a++");` เปิดไฟล์เพื่อเขียนต่อท้ายข้อมูลเก่า

`fp = fopen("data.txt", "r");` เปิดไฟล์เพื่ออ่านข้อมูล

ตัวอย่างโปรแกรมการประมวลผลไฟล์ การเปิดไฟล์ 01 :

```

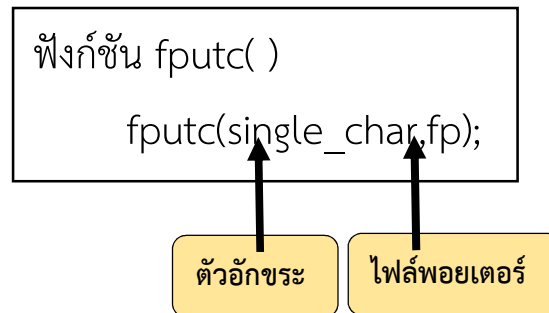
1  #include "stdio.h"
2  void main()
3  {
4      FILE *fp;
5      if( (fp = fopen("A:test.dat","w")) == NULL)
6      {
7          printf("Error in open file ");
8          exit(1);
9      }
10 }
```

การเขียนลงบนไฟล์

ในการเขียนข้อมูลลงบน text file คุณสามารถใช้ฟังก์ชันต่างๆ ได้ในภาษา C โดยทั่วไปแล้วจะมี ฟังก์ชันที่ 3 เราจะใช้

คือ

การเขียนตัวอักษรลงไฟล์



fputc() ใช้สำหรับเขียนตัวอักษรลงไปในไฟล์

fputs() ใช้สำหรับเขียนข้อความที่เป็น string ลงไปในไฟล์

fprintf() ใช้สำหรับเขียนข้อความที่เป็น string ลงไปในไฟล์เช่นกัน แต่สามารถจัดรูปแบบการเขียนได้

ตัวอย่างโปรแกรมการประมวลผลไฟล์ การเขียนลงบนไฟล์ 01 :

```

1  #include <stdio.h>
2  int main()
3  {
4      FILE *fp;
5      fp = fopen("example.txt", "w");
6      fputs("The first line of writing to the text file.\n", fp);
7      fputs("The second line.\n", fp);
8      fprintf(fp, "The third line with number %f.\n", 5.43);
9      fclose(fp);
10     return 0;
11 }

```

example - Notepad

The first line of writing to the text file.
The second line.
The third line with number 5.430000.

Windows (CRLF) Ln 1, Col 1 100%

การเขียนสตริงลงไฟล์

fputs(ตัวอักษร,ไฟล์พอยเตอร์(

```
fputs("Data Data Data\n",fp);
fputs("ComputerComputer\n",fp);
fputs("I LOVE YOU \n",fp);
fputs("Test Test Test\n",fp);
```

การเข้าถึงไฟล์ หรือ การอ่านจากไฟล์

การอ่านอักขระทีละตัวจากไฟล์

single_char = fgetc(fp)

ไฟล์พอยเตอร์

char c; c = fgetc(fp); printf("%c\n",c);	c = fgetc(fp); printf("%c\n",c);
c = fgetc(fp); printf("%c\n",c);	c = fgetc(fp); printf("%c\n",c);

การอ่านไฟล์จนกว่าจะจบไฟล์

การตรวจสอบตำแหน่งสิ้นสุดไฟล์)EOF - End Of File) จะใช้ฟังก์ชัน feof()
ฟังก์ชันนี้จะคืนค่า 0 ถ้าพอยน์เตอร์ไม่อยู่ในตำแหน่งสิ้นสุดไฟล์

```
char c;
while(!feof(fp))
{
    c = fgetc(fp);
    printf("%c",c);
}
```

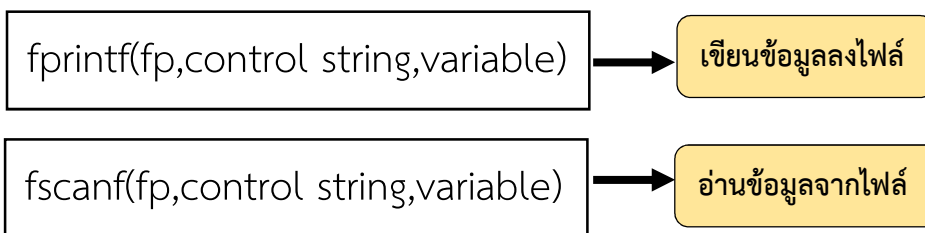
การอ่านสตริง

ใช้ฟังก์ชัน fgets

fgets(บัฟเฟอร์,จำนวนตัว,ไฟล์พอยน์เตอร์(;

```
char buffer[128];
fgets(buffer,30,fp);
printf("1 = %s\n",buffer);
fgets(buffer,30,fp);
printf("2 = %s\n",buffer);
fgets(buffer,30,fp);
printf("3 = %s\n",buffer);
```

การอ่านอักขระทีละตัวจากไฟล์



```
char name[30];
int age;
int h,w;
fscanf(fp,"%s %d %d %d\n",name,&age,&h,&w);
printf("Name = %s\n",name);
printf("age = %d\n",age);
printf("weight = %d\n",w);
printf("height = %d\n",h);
```

โจทย์คำสั่งการติดต่อไฟล์

โจทย์ file ข้อความจากนั้นจึงพิมพ์ใส่ในไฟล์ชื่อว่า 1 ให้รับข้อความ : 1text.txt

โจทย์ file ลงในไฟล์ชื่อ 2000 ถึง 1 ให้พิมพ์จำนวนเฉพาะตั้งแต่ : 2prime.txt

โจทย์ file ให้อ่านไฟล์ชื่อ : 3fileจากนั้นให้แสดงผล 2

โจทย์ file คน ประกอบด้วย 5 ให้นักเรียนรับข้อมูลของนักเรียน : 4ช่วยข้อมูล ชื่อ นามสกุล อายุ เขียนลงในไฟล์ชื่อ profile.txt

โจทย์ file ให้นักเรียนอ่านไฟล์จากชื่อ : 5fileแล้วหาผลรวมของจำนวนในไฟล์นั้น 2